PHYB57

全部课程资料合订本

Syllabus · Lectures C1—C21 · Tutorials 1—7 · 附录：Crib Sheet + 90 分钟模拟卷

共 253 页 · 2026-11-13 · PDF 左侧书签可直接跳转

目录

Syllabus		
PHYB57 F24 Syllabus		3
Lectures 课件		
Lecture C1	Introduction to Python as a Calculator • Overview:	7
Lecture C2	Variables and Assignment in Python Overview:	18
Lecture C3	Python Functions Key Concepts:	37
Lecture C4	If–Else Statements in Python •Branching Statements Overview:	52
Lecture C5	For–Loops in Python •Definition: A for–loop is a sequence of instructi	57
Lecture C8	Complexity and Big–O Notation •Definition of Complexity:	65
Lecture C8–1	Complexity and Big–O Notation •Definition of Complexity:	74
Lecture C9	Understanding Number Representation Systems Decimal System (Base–10)	83
Lecture C11	TXT Files in Python •Text Files Overview:	91
Lecture C12	2D Plotting in Python •Overview:	105
Lecture C16_1	Least Squares Regression •What is Least Squares Regression?	114
Lecture C16_2	Python Methods for Least Squares Regression •Key Python Libraries:	124
Lecture C17	Interpolation Problem Statement Overview:	130
Lecture C19	Root Finding Problem Statement Definition:	140
Lecture C20	Numerical Differentiation: Problem Overview Concept: Numerical differe	150
Lecture C21	Riemann's Integral in Numerical Methods •Definition: Riemann's Integra	160
Tutorials 习题课		
Tutorial 1	TUTORIAL 1 Welcome to your first PHYB57 tutorial! We'll be meeting 3PM	174
Tutorial 2	TUTORIAL 2 Today we're going to discuss data structures — lists, tuple	180
Tutorial 3	TUTORIAL 3 Welcome to Tutorial 3! Today, we will build upon the variab	187
Tutorial 4	TUTORIAL 4 Welcome to Tutorial 4! Today, we're going to discuss algori	196
Tutorial 5	TUTORIAL 5 Welcome to Tutorial 5! Today, we will explore how to work w	202
Tutorial 6	TUTORIAL 6 Welcome to Tutorial 6! Today, we're discussing least square	211
Tutorial 7	TUTORIAL 7 Welcome to Tutorial 7! Today, we will cover interpolation,	223
附录 复习材料		
Crib Sheet (1 page)		245
模拟卷 90 min（含答案）		246

PHYB57 F2024 - Introduction to Scientific Computing

Instructor: Kristen Menou kristen.menou@utoronto.ca

E-mail communication (preferred to Quercus messages)

Please send all e-mail correspondences from your university account and include your student number. In the email subject, include the prefix "PHYB57: "

Text Book

We will use the textbook "Python Programming And Numerical Methods: A Guide For Engineers And Scientists". There is no need to purchase it as there is a freely-available online version that will be extensively used:

https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/Index.html

Office hours:

Monday 1-2pm - default time is 5 mins per student

There will be regularly scheduled office hours during which the instructor will be available to engage in individual or group consultations as determined by the needs of the students. Should additional office hours be required, students are encouraged to make such requests, specifying a few preferred time slots that would be convenient for them.

Learning outcomes

- Develop proficiency in Python for numerical modeling and data analysis.
- Understand computational models and their applications in physics.
- Gain skills in scientific visualization and the interpretation of data.

Lectures

In-person lectures will be on Mondays 10-11am and Thursdays 11am - 12noon.

Note that the lectures will start 10 minutes past the hour. Please make sure you are on time.

Please refrain from eating lunch during the lectures.

Practicals

Mondays, 3-4pm or 4-5pm (students will be assigned to one of two 1hr-long sessions)

Attendance is mandatory for all practicals. During the practicals, exercises assigned to all students as practice will be solved/reviewed.

Grading Scheme

Assessment Title	Percent (%)
Homework (x4)	10
Pop quizzes (lectures & practicals)	10
Term test 1	20
Term test 2	20
Final Exam	40
Total Percentage	**100**

Academic Integrity in PHYB57

> Students in this course are obligated to understand what constitutes plagiarism. UTSC demands high standards of integrity and ethical conduct. We will follow a strict adherence to the University's Code of Behaviour for Academic Matters. You can find it in https://www.utsc.utoronto.ca/vpdean/academic-integrity-matters

There are also certain specific guidelines that apply to this course. While collaboration with fellow students is encouraged, it is prohibited to present any work that is not one's own original production. In the event of working with a partner, it is mandatory to indicate this on all assignments submitted. Failure to do so will be considered as a violation of academic integrity. It is important to note that even when working in partnership, each individual is required to produce their own distinct document and submitting identical documents will also be considered as a violation.

Furthermore, it is a violation of academic integrity to solicit the assistance of friends, family members or any other third party in completing assignments. Similarly, the hiring of external entities to produce assignments on one's behalf is strictly prohibited and shall be treated as a severe violation of academic integrity.

As a general rule, the use of the internet for research purposes is permissible, however, the sharing or distribution of any course-related materials such as lectures, assignments, quizzes or exams is strictly prohibited. This includes posting such material on public spaces such as discussion forums, and sharing it in private spaces such as instant messaging apps or chat groups. Any such violations will be treated as a severe breach of academic integrity.

The use of AI tools will be permitted, as detailed during the first lecture.

Accessibility

This course welcomes and accommodates students with a diverse range of learning styles and needs. If you have any disability or health condition that may require accommodations, please do not hesitate to contact the instructor or the AccessAbility Services Office for assistance. Your enquiries will be treated with the utmost confidentiality.

Problem Sets:

Handed out in class approximately every other week (total of 4). Due one week later.

Policy on collaboration: You are welcome to discuss the problems with fellow students, but you must write your own solutions, individually.

Policy on late problem set returns: In order to be fair to those who turn assignments in on time, points will be deducted on assignments turned in late.

Tentative Lecture Schedule:

Week 1

Introduction and overview. Chapter 1 - Python Basics

Week 2

Chapter 2 & 3 - Variables, Data Structures, Functions

Week 3

Chapter 4 & 5 - Branching & Iteration

Week 4

Chapter 8 & 9 - Complexity & Representation of Numbers

Week 5

Chapter 11 & 12 - Reading, Writing & Visualization

Week 6

Chapter 16 - Least Square Regression

Reading Week - No lecture or tutorial on February 23rd

Week 7

Chapter 17 - Interpolation

Week 8

Chapter 19 - Root Finding

Week 9

Chapter 20 - Numerical Differentiation

Week 10

Chapter 21 - Numerical Integration

Week 11

Chapter 22 - ODE IVPs

Week 12

Chapter 23 - - ODE BVPs

Introduction to Python as a Calculator

- **Overview:**
  - Python can be used as a powerful calculator to perform basic arithmetic operations similar to a standard graphing calculator.
  - Operations include addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (**).
- **Order of Operations:**
  - Python follows the standard mathematical order of operations: Parentheses, Exponents, Multiplication and Division (from left to right), Addition and Subtraction (from left to right).
  - Parentheses can be used to change the order of execution.
- **Example Calculation:**

```python
(3*4)/(2**2 + 4/2)  # Output: 2.0
```

Using Python's Math Module

- **Accessing Mathematical Functions:**
  - Python provides various arithmetic functions through the math module, which needs to be imported using import math.
  - Common functions include:
    - math.sqrt(x) - Square root
    - math.sin(x) - Sine function
    - math.exp(x) - Exponential function
    - math.log(x) - Natural logarithm
    - math.pi - Value of π
- **Example Calculations:**

```python
import math
math.sin(math.pi/2)  # Output: 1.0
math.exp(math.log(10))  # Output: 10.000000000000002
```

- **Tips:**
  - Use TAB completion in Jupyter notebooks or Ipython to auto-complete function names.
  - View function documentation by appending a question mark ? after the function name, e.g., math.factorial?.

Handling Different Data Types and Complex Numbers

- **Data Types in Python:**
  - **int**: Integer values (e.g., 1, 42)
  - **float**: Floating-point numbers (e.g., 3.14, 2.718)
  - **complex**: Complex numbers (e.g., 2 + 5j)
- **Working with Complex Numbers:**
  - Python uses j to denote the imaginary unit.
  - You can create complex numbers directly or using the complex() function.

```python
2 + 5j  # Output: (2+5j)
complex(2, 5)  # Output: (2+5j)
```

- **Scientific Notation:**
  - Python supports scientific notation for large numbers, using e to denote powers of 10.

```python
3e0*3.65e2*2.4e1*3.6e3  # Output: 94608000.0
```

Introduction to Managing Python Packages

Key Points:

- **Importance of Packages in Python:**

Python's strength lies in the vast number of community-developed packages/modules that provide pre-coded functions and algorithms.

- **Role of Package Management:**

Managing packages efficiently is crucial to fully leverage Python's capabilities, as it allows for easy installation, updating, and removal of packages.

- **Primary Tool - pip:**

pip is the most common package manager used in Python, automating the process of package management.

Common pip Commands

Key Commands:

- **Installing a Package:**
  - Command: pip install package_name
  - To install a specific version: pip install package_name==1.5
- **Upgrading a Package:**
  - Command: pip install --upgrade package_name
  - Shortcut: pip install -U package_name
- **Uninstalling a Package:**
  - Command: pip uninstall package_name
- **Listing Installed Packages:**
  - Command: pip list
- **Displaying Package Information:**
  - Command: pip show package_name

Advanced Package Management

Installing Packages from Source:

- **When to Use:**

Occasionally, a package may not be available on PyPI, requiring manual installation fr
source files.

- **Process:**
  - Download and uncompress the source file.
  - Navigate to the directory containing the setup.py script.
  - Run the command: python setup.py install
- **Considerations for Windows Users:**

Use a command prompt window to execute the installation command.

Introduction to Jupyter Notebook

Overview:

- **Jupyter Notebook** is an open-source web application that allows for the creation and sharing of documents containing live code, equations, visualizations, and narrative text.
- Commonly used for:
  - Data cleaning and transformation
  - Numerical simulation
  - Statistical modeling
  - Data visualization
  - Machine learning

Advantages:

- Combines code and rich text elements such as equations, links, images, and tables in one document.
- Facilitates the creation of interactive content like blogs, papers, and books.
- Ideal for sharing ideas and results with others.

Working with Jupyter Notebooks

Starting a Jupyter Notebook:

- Run the command jupyter notebook in your terminal to launch the notebook interface in your default web browser.
- The interface displays a dashboard showing files, running notebooks, and options to create new notebooks.

Notebook Interface:

- The interface includes a toolbar and a menu that provide access to various tools and functions.
- **Key Features:**
  - **Cells:** The basic unit where you can write and execute code or add text.
  - **Cell Types:**
    - **Code cells** for writing and running Python code.
    - **Markdown cells** for adding descriptive text, using rich text formatting.

Basic Operations:

- Execute a cell by pressing Shift + Enter.
- Cells can be rearranged, inserted, or deleted as needed.

Managing and Closing Jupyter Notebooks

Shutting Down a Notebook:

- Simply closing the browser tab does not stop the notebook; the server continues running in the background.
- To fully shut down a notebook, close the associated terminal where it was launched.

Proper Closure:

- To completely close a notebook:
  - Go to the dashboard.
  - Check the box next to the running notebook.
  - Select the shutdown option from the toolbar.

Conclusion:

- With the basics covered, you're ready to start using Jupyter Notebook to develop and share Python projects.

Introduction to Logical Expressions and Operators in Python

Logical Expressions:

- A logical expression evaluates to either True or False.
- Python uses the boolean data type for logical expressions.
  - True is equivalent to 1
  - False is equivalent to 0
- **Example:**
  - Expression: 3 < 4
  - Python evaluates it as True.

Comparison Operators:

- Used to compare values in logical expressions.
- Key Operators:
  - > : Greater than
  - >= : Greater than or equal to
  - < : Less than
  - <= : Less than or equal to
  - != : Not equal
  - == : Equal

Example of Comparison:

- Given a = 4 and b = 2:
  - a > b evaluates to True
  - a == b evaluates to False

Logical Operators and Order of Operations

Logical Operators:

- Operate on two or more logical expressions.
- **Key Operators:**
  - and: True if both expressions are true.
  - or: True if at least one expression is true.
  - not: Inverts the truth value of an expression.

Truth Table Examples:

- **AND Operator:**
  - P and Q is True if both P and Q are True.
- **OR Operator:**
  - P or Q is True if at least one of P or Q is True.

Order of Operations:

- Arithmetic operations are executed first, followed by comparison operations, and finally logical operations.
- **Tip:**
  - Use parentheses to clarify order, e.g., (1 + 3) > (2 + 5) is more readable.

Python Quirks:

- Python allows expressions like (3 > 2) + (5 > 4), which resolves to 2.
- Any non-zero number is considered True in logical operations.

Variables and Assignment in Python

Overview:

- **Variables** are containers for storing data values. In Python, they act as references to information stored in memory.
- The **assignment operator** = is used to assign values to variables.
  - Example: x = 1 assigns the value 1 to the variable x.
- Variables in programming are unlike mathematical variables; they do not represent a truth statement but rather a storage location for data.

Key Concepts:

- Variables are named using alphanumeric characters and underscores (_).
- The first character of a variable name must be a letter or underscore.
- Variable names are **case-sensitive** (e.g., x and X are different).
- Avoid using names of built-in functions as variable names (e.g., help = 2).

Example:

```python
x = 1
y = 2
y = y * 3  # y now behaves as 2, and y * 3 equals 6
```

Assignment vs. Mathematical Equality:

- In Python, x = 2 assigns the value 2 to x. It's different from the mathematical statement x = 2, which is a truth assertion.
- Python assignments always go left: the right-hand value is stored in the left-hand variable.

Important Operations:

- Example: x = x + 1 in programming:
  - It updates x by adding 1 to its current value.
  - If x was 1 before, it becomes 2 after the operation.

Managing Variables:

- Use %whos to list all variables in memory.
- Clear a specific variable using del variable_name.
- Clear all variables with %reset.

Example:

```python
x = 1
x = x + 1  # Now, x equals 2
del x  # x is removed from memory
```

Best Practices in Variable Naming and Usage

Naming Conventions:

- Variables should have meaningful names that reflect their purpose.
  - Example: Use distance for the distance between two points instead of x.
- Avoid spaces in variable names; use underscores or camelCase for readability.

Caution:

- Reassigning a variable does not retain any memory of how it was assigned.
  - Changing x after y = x + 1 does not affect y.
- Be mindful of overwriting variables or functions (e.g., reassigning help).

Example:

```python
distance = 5
numRabbits = 12
```

TIP: Always assign results to variables to keep track of operations and avoid confusion.

Strings in Python

What is a String?

- A string is a sequence of characters enclosed within single or double quotation marks.
- Examples: "Hello World", 'Python', "12345".

Key Characteristics:

- **Type Verification**: Strings can be verified using the type() function.
  - Example:

```python
w = "Hello World"
type(w)  # Output: <class 'str'>
```

- **Empty Strings**: You can create an empty string and verify it.
  - Example:

```python
empty_str = ""
type(empty_str)  # Output: <class 'str'>
```

String Length and Indexing:

- **Length**: Use len() function to find the number of characters in a string.
  - Example:

```python
len(w)  # Output: 11
```

- **Indexing**: Strings are indexed starting from 0. You can access any character using its index.
  - Example:

```python
w[6]  # Output: 'W'
```

Advanced String Operations in Python

String Slicing:

- **Basic Slicing**: Extract a substring using the syntax [start:end].
  - Example:

```python
w[6:11]  # Output: 'World'
```

- **Negative Indexing**: Use negative indices to slice from the end of the string.
  - Example:

```python
w[-5:]  # Output: 'World'
```

String Manipulation Methods:

- **Concatenation**: Combine two strings using the + operator.
  - Example:

```python
str_a = "Hello"
str_b = "World"
print(str_a + " " + str_b)  # Output: 'Hello World'
```

- **Upper Case Conversion**: Convert all characters in a string to upper case.
  - Example:

```python
w.upper()  # Output: 'HELLO WORLD'
```

- **String Replacement**: Replace a substring within a string.
  - Example:

```python
w.replace("World", "Python")  # Output: 'Hello Python'
```

Formatted Strings (f-Strings):

- **Basic Usage**: Insert variables into strings easily.
  - Example:

```python
name = "Python"
print(f"I love {name}!")  # Output: 'I love Python!'
```

- **Expression Evaluation**: Directly evaluate expressions inside strings.
  - Example:

```python
print(f"{3 * 4}")  # Output: '12'
```

Lists in Python

- **Definition and Characteristics**
  - Python lists are versatile data structures that can store a sequence of items.
  - Lists are defined using square brackets [ ], with elements separated by commas.
  - A list can contain any type of data, including numbers, strings, and even other lists (nested lists).
- **Examples**
  - A simple list: list_1 = [1, 2, 3]
  - A list with mixed data types: list_3 = [1, 2, 3, 'Apple', 'orange']
  - Nested lists: list_4 = [[1, 2, 3], ['Hello', 'World']]
- **Accessing Elements**
  - Elements are accessed using indices, similar to strings.
  - Example: list_3[2] retrieves the 3rd element of list_3, which is 3.

Operations on Lists

- **Basic Operations**
  - **Length**: Use len(list) to get the number of elements in a list.
  - **Concatenation**: Combine lists using the + operator.
  - **Appending**: Add elements to a list using append().
  - **Inserting**: Insert elements at a specific position using insert().
  - **Deleting**: Remove elements by index using the del statement.

- **Examples**
  - Adding lists: list_1 + list_2 results in [1, 2, 3, 'Hello', 'World'].
  - Appending: list_1.append(4) changes list_1 to [1, 2, 3, 4].
  - Inserting: list_1.insert(2, 'center') changes list_1 to [1, 2, 'center', 3, 4].
  - Deleting: del list_1[2] removes the element at index 2, resulting in [1, 2, 3, 4].

Advanced List Operations

- **Checking Membership**
  - Use the in operator to check if an element exists in a list.
  - Example: 5 in list_5 returns True if 5 is in list_5.
- **Converting Sequences to Lists**
  - The list() function can convert other sequence types (e.g., strings) into lists.
  - Example: list('Hello World') converts the string into a list of characters: ['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd'].
- **Usage in Data Handling**
  - Lists are frequently used in Python for handling and manipulating data, making them a fundamental tool in programming.

Tuples in Python

- **Definition**: A *tuple* is a sequence data structure in Python, typically defined using parentheses ( ). Elements within a tuple are separated by commas.
- **Key Characteristics**:
  - **Immutability**: Once a tuple is created, its elements cannot be modified.
  - **Heterogeneity**: Tuples can store a heterogeneous sequence of elements, meaning they can contain elements of different data types.
- **Comparison with Lists**:
  - Tuples are immutable, while lists are mutable.
  - Tuples often store a fixed sequence of diverse elements, whereas lists usually contain a sequence of homogeneous elements.

Example:

```python
tuple_1 = (1, 2, 3, 2)
print(tuple_1[1:3])  # Output: (2, 3)
```

Operations and Uses of Tuples

- **Common Operations**:
  - **Indexing and Slicing**: Similar to lists, tuples support indexing and slicing.
    - Example: tuple_1[1:4] gives (2, 3, 2)
  - **Length**: len(tuple_1) returns the number of elements.
  - **Counting Elements**: tuple_1.count(2) returns the count of a specific element.

- **Use Cases**:
  - **Data Integrity**: Tuples are ideal when you need a data structure that should not be modified.
  - **Unpacking**: Tuples can be easily unpacked into variables.
    - Example: a, b, c = tuple_1
  - **Heterogeneous Data**: Used to store pairs of related but different data types, like (fruit, quantity) pairs.

Example:

```python
fruits = ('apple', 3)
print(f"Fruit: {fruits[0]}, Quantity: {fruits[1]}")  # Output: Fruit: apple, Quantity: 3
```

Practical Examples and Tuple Constraints

- **Immutability in Practice**:
  - Attempting to modify a tuple element results in a TypeError.
  - Example:

```python
tuple_1 = (1, 2, 3)
tuple_1[2] = 4  # Raises TypeError: 'tuple' object does not support item assignment
```

- **Packing and Unpacking**:
  - Tuples support *packing* (combining multiple values into a tuple) and *unpacking* (assigning tuple elements to variables).

- **Real-World Scenario**:
  - Example: Storing coordinates (x, y) as a tuple for geometric operations.
  - Example: Packing items into a tuple point = (10, 20) and then unpacking x, y = point.

Conclusion:

- Tuples are a crucial data structure in Python, offering immutability and a convenient way to handle related but varied data types.

Sets in Python

- **Definition**:
  - Sets in Python are an unordered collection of unique elements, defined using curly braces {}.
  - They do not allow duplicate elements, making them ideal for operations requiring uniqueness.
- **Creation**:
  - Example: set_1 = set([1, 2, 2, 3, 2, 1, 2]) results in {1, 2, 3}.
  - A set can be created from a list, tuple, or string by passing it to the set() function.
- **Key Properties**:
  - Unordered: No guaranteed order of elements.
  - Immutable: Elements inside the set cannot be changed, but the set itself is mutable (i.e., elements can be added or removed).

Basic Operations on Sets

- **Finding Unique Elements**:
  - Example: set([1, 2, 2, 3, 2, 1, 2]) → {1, 2, 3}
  - Example: set('Banana') → {'B', 'a', 'n'}
- **Mathematical Operations**:
  - **Union**: Combines elements from both sets without duplicates.
    - Example: set_1.union(set_2) where set_1 = {1, 2, 3} and set_2 = {2, 4, 5, 6} results in {1, 2, 3, 4, 5, 6}.
  - **Intersection**: Returns elements that are present in both sets.
    - Example: set_1.intersection(set_2) results in {2}.
  - **Difference**: Returns elements that are in one set but not the other.
    - Example: set_1.difference(set_2) results in {1, 3}.
  - **Symmetric Difference**: Elements that are in either set, but not in both.
    - Example: set_1.symmetric_difference(set_2) results in {1, 3, 4, 5, 6}.

Advanced Set Operations and Applications

- **Subset and Superset**:
  - **Subset**: Check if a set is a subset of another.
    - Example: set_1.issubset({1, 2, 3, 4, 5}) returns True because all elements of set_1 are in the larger set.
  - **Superset**: Check if a set contains another set.
    - Example: {1, 2, 3, 4, 5}.issuperset(set_1) returns True.
- **Applications**:
  - Sets are used to filter unique items, perform fast membership testing, and implement common mathematical operations in Python programs.
  - Examples include removing duplicates from a list, performing data analysis to find common items across multiple datasets, and working with collections where order is not important.

Dictionaries in Python

- **Definition**:
  - A **dictionary** in Python is a collection of key-value pairs where each key is unique.
  - Unlike sequences (lists or tuples), dictionaries are **unordered** and indexed by keys rather than by a numerical index.
- **Key Features**:
  - **Keys**: Can be strings, numbers, or tuples but cannot be lists.
  - **Values**: Can be of any data type.
  - **Syntax**: Defined using {}, with key-value pairs separated by colons.

```python
dict_1 = {'apple': 3, 'orange': 4, 'pear': 2}
```

Accessing Elements:

- Access a value by referring to its key:

```python
dict_1['apple']  # Output: 3
```

Working with Dictionaries

- **Common Operations**:
  - **Getting all keys**: dict_1.keys()
  - **Getting all values**: dict_1.values()
  - **Getting dictionary length**: len(dict_1)

- **Creating and Modifying Dictionaries**:
  - **Empty Dictionary**: Can be created and filled later.

```python
school_dict = {}
school_dict['UC Berkeley'] = 'USA'
```

  - **Converting Tuples to Dictionary**:

```python
dict([("UC Berkeley", "USA"), ('Oxford', 'UK')])
```

- **Checking Membership**:
  - Check if a key exists using the in keyword:

```python
"UC Berkeley" in school_dict  # Output: True
```

  - To check if a key does not exist, use not in:

```python
"Harvard" not in school_dict  # Output: True
```

Practical Examples and Use Cases

- **Practical Usage**:
  - Dictionaries are highly useful for **mapping** data where a unique identifier (key) maps to a specific value.
  - Examples include storing **user data** by ID, **configuration settings**, and **look-up tables**.
- **Examples**:
  - **Inventory System**:

```python
inventory = {'apple': 50, 'banana': 30, 'orange': 20}
```

  - **Gradebook**:

```python
grades = {'Alice': 'A', 'Bob': 'B+', 'Charlie': 'A-'}
```

Advantages:

- **Efficient Lookups**: Access elements in constant time.
- **Flexible Data Handling**: Can store complex data structures as values.

Introduction to Numpy Arrays

Overview:

•Numpy is a fundamental Python module for numerical computing, widely used in scientific computing.

•Key features of Numpy:

- **N-dimensional Array Object:** Efficiently handles large datasets with a powerful array object.
- **Broadcasting:** Facilitates operations on arrays of different shapes.
- **Integration with C/C++ and Fortran:** Enhances performance by integrating with lower-level languages.
- **Linear Algebra, Fourier Transform, and Random Number Capabilities:** Essential tools for scientific computing.

Creating Numpy Arrays:

•Importing Numpy: import numpy as np (community convention uses “np” as the alias).

•Converting a list to an array: np.array([1, 4, 3]) results in array([1, 4, 3]).

•2D Arrays: Use nested lists, e.g., np.array([[1, 4, 3], [9, 2, 7]]).

Array Attributes:

•shape: Returns a tuple of the array dimensions. E.g., (2, 3) for a 2x3 matrix.

•size: Returns the total number of elements in the array.

Numpy Array Operations

Generating Arrays:

- **Using np.arange:** Creates arrays with a defined start, end, and step value.
  - Example: np.arange(1, 2000, 1) generates [1, 2, 3, ..., 1999].
- **Using np.linspace:** Creates arrays with a specified number of evenly spaced elements.
  - Example: np.linspace(3, 9, 10) generates [3, 3.67, 4.33, ..., 9].

Array Indexing:

- 1D Array: Access elements using indices, e.g., x[1] for the second element.
- 2D Array: Access elements using row and column indices, e.g., y[0, 1] for the element in the first row, second column.

Common Array Operations:

- **Arithmetic Operations:** Perform element-wise operations between arrays and scalars.
  - Example: b + 2 adds 2 to each element of array b.
- **Matrix Multiplication:** Element-by-element multiplication using *.
  - Example: b * d multiplies corresponding elements of arrays b and d.

Python Functions

Key Concepts:

- **Function Definition**: A function in Python is a block of reusable code that performs a specific task. It is defined using the def keyword, followed by the function name, parameters in parentheses, and a colon.
- **Components of a Function**:
  1. **Function Header**: Starts with def, includes the function name and parameters.
  2. **Function Body**: Contains the statements that execute the function's task, indented for structure.
  3. **Return Statement**: Optional; returns the output of the function.

Example:

```python
def my_adder(a, b, c):
    """Function to sum three numbers"""
    return a + b + c
```

Usage:

```python
result = my_adder(1, 2, 3)  # Returns 6
```

Best Practices:

- Use descriptive names for functions and variables.
- Comment extensively to clarify the function's purpose and logic.
- Regularly save your work.

Understanding Function Parameters

Input Arguments:

- Functions can accept multiple input arguments, which are variables provided by the user or another function.
- Example:

```python
def multiply(x, y):
    return x * y
result = multiply(4, 5)  # Returns 20
```

Output Parameters:

- Functions may return multiple values as a tuple.
- Example:

```python
def my_trig_sum(a, b):
    out1 = np.sin(a) + np.cos(b)
    out2 = np.sin(b) + np.cos(a)
    return out1, out2
c, d = my_trig_sum(2, 3)
```

Error Handling:

- Python functions assume input arguments will be of the expected type. If not, errors like TypeError may occur.
- Example of error:

```python
def my_adder(a, b, c):
    return a + b + c

result = my_adder(1, 2, "three")  # Raises TypeError
```

Advanced Function Features

Multiple Outputs:

- Functions can return multiple outputs, typically as a tuple, allowing the user to unpack the values into separate variables.
- Example

```python
def calculate(a, b):
    return a + b, a * b

sum_result, product_result = calculate(5, 10)
```

Default Parameters:

- Functions can have default parameter values, which are used if no argument is provided during the function call.
- Example:

```python
def greet(name="World"):
    print(f"Hello, {name}!")

greet()  # Prints "Hello, World!"
greet("Alice")  # Prints "Hello, Alice!"
```

Composition of Functions:

- Functions can be composed, meaning the output of one function can be used as the input for another.
- Example:

```python
def add(a, b):
    return a + b

def square(x):
    return x * x

result = square(add(2, 3))  # Returns 25
```

Understanding Local Variables in Python

- **Local Variables** are variables that are declared inside a function.
- These variables are **accessible only within that function** and have their own memory block separate from the rest of the program.
- The value of a local variable is not affected by or does not affect variables outside the function, even if they share the same name.

Example:

```python
def my_adder(a, b, c):
    out = a + b + c
    print(f'The value of out within the function is {out}')
    return out

out = 1
d = my_adder(1, 2, 3)
print(f'The value of out outside the function is {out}')
```

Output:

- Inside the function: out = 6
- Outside the function: out = 1

Key Points:

- When the function is executed, a **new memory block** is created.
- Variables inside the function do not interfere with variables outside it.

Understanding Global Variables in Python

- **Global Variables** are declared outside of functions and are accessible anywhere in the program.
- They can be read and modified both inside and outside functions, using the global keyword.

Common Issues:

- Without declaring a variable as global, attempts to modify it inside a function can lead to errors like UnboundLocalError.

Example Without Global Keyword:

```python
n = 42

def func():
    print(f'Within function: n is {n}')
    n = 3
    print(f'Within function: n is now {n}')

func()
```

Error: UnboundLocalError: local variable 'n' referenced before assignment

Solution Using Global Keyword:

```python
n = 42

def func():
    global n
    print(f'Within function: n is {n}')
    n = 3
    print(f'Within function: n is now {n}')

func()
print(f'Outside function: n is {n}')
```

Output:

- Inside function: n = 42, then n = 3
- Outside function: n = 3

Practical Implications and Best Practices

Why Separate Memory Blocks?

- **Efficiency**: Prevents functions from interfering with each other in large projects.
- **Modularity**: Allows independent development and debugging of functions without concern for variable name conflicts.

Best Practices:

1. **Minimize Global Variables**: Overuse can lead to complex dependencies and harder-to-debug code.
2. **Use Functions Effectively**: Leverage local variables within functions to encapsulate functionality and maintain clear scope boundaries.
3. **Global Keyword Use**: Use cautiously and only when necessary to modify global variables inside functions.

Nested Functions in Python

- **Definition**:
  - A nested function is a function defined inside another function (the parent function).
  - The nested function can only be called by the parent function.

- **Why use nested functions?**:
  - Useful when a task needs to be performed multiple times within the parent function but is not required outside.
  - Helps in organizing the code and preventing redundancy.

- **Example**:

```python
def parent_function():
    def nested_function():
        # Code for nested function
    # Code that calls the nested function
```

Memory and Scope in Nested Functions

- **Memory block**:
  - A nested function has its own separate memory block, independent of the parent function.
  - This allows variables with the same name to exist in both the parent and nested functions without conflict.

- **Practical Example**:

```python
def my_dist_xyz(x, y, z):
    def my_dist(x, y):
        return np.sqrt((x[0]-y[0])**2 + (x[1]-y[1])**2)
    d0 = my_dist(x, y)
    d1 = my_dist(x, z)
    d2 = my_dist(y, z)
    return [d0, d1, d2]
```

- Here, my_dist() is a nested function that computes distances between 2D points.

Advantages of Using Nested Functions

- **Code clarity**:
  - Organizing complex functions into smaller, task-specific sub-functions reduces clutter and enhances readability.

- **Error reduction**:
  - Nested functions reduce the risk of mistakes (like formula errors), as computations are centralized within a single subfunction.

- **Cleaner code**:
  - Without a nested function, the parent function may become cluttered with repeated logic, increasing the chances of errors and making it harder to maintain.

Lambda Functions in Python

- **Definition**: A *lambda function* is an anonymous function, meaning it has no name and is typically used for short, simple operations.

- **Syntax**: Lambda functions are written using the lambda keyword followed by any number of arguments, and an expression to be evaluated.

```python
lambda arguments: expression
```

- **Use Case**: These functions are commonly used when a simple function is needed for a single task, especially in scenarios such as sorting, filtering, or applying functions in a concise manner.

Example:

- To square a number:

```python
square = lambda x: x**2
print(square(2))  # Output: 4
```

Practical Applications of Lambda Functions

- **Lambda with Sorting**: Lambda functions are often used in sorting custom objects. For example, sorting a list of tuples based on the second element:

```python
sorted([(1, 2), (2, 0), (4, 1)], key=lambda x: x[1])
# Output: [(2, 0), (4, 1), (1, 2)]
```

Lambda for Simple Math Operations: Lambda functions allow quick mathematical operations without the need to define full functions.

```python
my_adder = lambda x, y: x + y
print(my_adder(2, 4))  # Output: 6
```

Advantages:

- Compact code.
- Useful in functions like map(), filter(), and sorted() where a small function is required.

Limitations and Best Practices

- **Limitations**:
  - Lambda functions are limited to one expression and are not suitable for complex operations.
  - Lack of a name makes debugging harder for more complicated functions.

- **Best Practices**:
  - Use lambda functions for simple tasks such as mathematical operations, data sorting, or filtering.
  - Avoid using them for large or complex functions where traditional function definitions are more appropriate.

Functions as Arguments

- **Key Points**:
  - In Python, functions are treated as first-class objects, meaning they can be passed as arguments to other functions.
  - This capability allows for greater flexibility in coding, making functions interchangeable.

  - **Example**: The max function can be assigned to a variable f, and using f is equivalent to calling the max function.

```python
f = max
print(f([2, 3, 5]))  # Output: 5
```

Custom Function with Function Argument

- **Key Points**:
  - You can define custom functions that accept other functions as arguments.
  - **Example**: A function my_fun_plus_one takes a function f and a value x as input, applies f to x, and adds 1 to the result

```python
import numpy as np

def my_fun_plus_one(f, x):
    return f(x) + 1

print(my_fun_plus_one(np.sin, np.pi/2))  # Output: 2.0
print(my_fun_plus_one(np.sqrt, 25))  # Output: 6.0
```

- **Concept**: This enables the combination of various mathematical operations in a streamlined way.

.

Lambda Functions as Input

- **Key Points**:
  - Lambda functions, which are small anonymous functions, can also be passed as arguments to other functions.

  - **Example**: Passing a lambda function to my_fun_plus_one for simple mathematical operations.

```python
print(my_fun_plus_one(lambda x: x + 2, 2))  # Output: 5
```

- **Advantage**: This reduces the need to define separate named functions for small operations, enhancing code efficiency and readability.

If-Else Statements in Python

- **Branching Statements Overview**:
  - An **If-Else Statement** (or simply **If Statement**) is used to control the flow of code.
  - It executes a block of code only if a given logical condition is met.
  - If the condition is true, a specific code block is executed; otherwise, the program skips to the next block.

- **Basic Syntax**:

```python
if condition:
    code_block
else:
    alternative_code_block
```

- Python evaluates the logical expression (condition). If True, the code block executes. If False, it moves to the else block.

- **Indentation**:
  - All lines within the if or else blocks must have the same level of indentation, typically four spaces.

Extended If-Else and Elif Statements

- **Extended If-Else**:
  - You can add multiple conditions using elif (short for else-if).
  - Python checks each condition in sequence, and once it finds a True condition, it executes the corresponding block and exits.

- **Syntax**:

```python
if condition_1:
    code_block_1
elif condition_2:
    code_block_2
elif condition_3:
    code_block_3
else:
    default_code_block
```

- **Key Points**:
  - The elif statement allows checking additional conditions.
  - You can have as many elif statements as needed, but only one else statement.
  - Once a True condition is found, no further conditions are evaluated.

Example - Thermostat Function

- **Task**: Write a function my_thermo_stat(temp, desired_temp) that adjusts a thermostat based on the temperature difference.
- **Functionality**:
  - If the current temperature is more than 5 degrees lower than the desired temperature, return "Heat".
  - If it’s more than 5 degrees higher, return "AC".
  - Otherwise, return "Off".
- **Code Example**:

```python
def my_thermo_stat(temp, desired_temp):
    if temp < desired_temp - 5:
        return 'Heat'
    elif temp > desired_temp + 5:
        return 'AC'
    else:
        return 'Off'
```

- **Sample Usage**:

```python
print(my_thermo_stat(65, 75))  # Output: Heat
print(my_thermo_stat(75, 65))  # Output: AC
print(my_thermo_stat(65, 63))  # Output: Off
```

Ternary Operators in Python

- **Definition**: Ternary operators, also known as conditional expressions, provide a concise way to write simple if-else logic in a single line of code.
- **Syntax**:

```python
expression_if_true if condition else expression_if_false
```

- **Example**:

```python
is_student = True
person = 'student' if is_student else 'not student'
print(person)  # Output: student
```

- **Benefits**:
  - Simplifies code for basic conditionals.
  - Useful for making code more concise and readable.
  - Often employed in list comprehensions and similar constructs.

Equivalent Block for Ternary Operator

- **Ternary Operator Equivalent in Python**:

The example from the previous slide can be expressed in a more traditional if-else structure:

```python
is_student = True
if is_student:
    person = 'student'
else:
    person = 'not student'
print(person)  # Output: student
```

- **Comparison**:
  - The ternary operator offers a more compact representation, reducing the number of lines and increasing readability for simple conditionals.
  - The traditional if-else block is better suited for more complex conditional logic.

For-Loops in Python

- **Definition**: A for-loop is a sequence of instructions that is repeated for each element in a collection or range.
  - Also called a “definite loop” because it iterates over a predetermined sequence.
- **General Syntax**:

```python
for looping_variable in sequence:
    code block
```

- **Example**: Summing numbers from 1 to 3 using a for-loop

```python
n = 0
for i in range(1, 4):
    n += i
print(n)  # Output: 6
```

- **How it Works**:

1. The range(1, 4) generates a sequence [1, 2, 3].
2. In each iteration, the variable i is updated with the next value from the sequence.
3. The value of n is updated by adding i to it in each iteration.
4. The loop terminates when the sequence ends, resulting in the sum n = 6.

Advanced For-Loops and Applications

- **Iterating over Strings**:
  - Example: Print each character in the string “banana”

```python
for c in "banana":
    print(c)
```

- **Working with Lists**:
  - Example: Summing specific elements from a list using step indexing.

```python
a = [2, 3, 1, 3, 3]
s = 0
for i in range(0, len(a), 2):
    s += a[i]
print(s)  # Output: 6
```

- **Using Dictionaries**:
  - Example: Iterating over keys and values in a dictionary.

```python
dict_a = {"One": 1, "Two": 2, "Three": 3}
for key, value in dict_a.items():
    print(key, value)
```

- **Nested For-Loops**:
  - Example: Summing all elements in a 2D array using nested loops.

```python
x = [[5, 6], [7, 8]]
s = 0
for i in range(len(x)):
    for j in range(len(x[i])):
        s += x[i][j]
print(s)  # Output: 26
```

While Loops in Python

Definition:

- A while loop in Python is a control flow structure that repeatedly executes a block of code as long as the specified condition (logical expression) evaluates to True.

Syntax:

```python
while <condition>:
    # Code block to be repeated
```

Key Concept:

- Before each iteration, Python checks if the condition is true.
- If true, the code block executes and the process repeats.
- If false, the loop ends.

Use Case:

- While loops are particularly useful when the number of iterations is unknown beforehand.

Example of a While Loop

Task:

Determine how many times 8 can be divided by 2 before the result is less than 1.

```python
i = 0
n = 8

while n >= 1:
    n /= 2
    i += 1

print(f'n = {n}, i = {i}')
```

Explanation:

1.i tracks the number of divisions.

2.n starts at 8, and while n >= 1, it is divided by 2.

3.Loop continues until n becomes less than 1.

4.Output: After 4 iterations, n = 0.5 and i = 4.

Infinite Loops and Practical Tips

Infinite Loops:

- If the condition in a while loop always evaluates to True and nothing in the loop changes this condition, the loop runs indefinitely, causing an infinite loop.

Example of Infinite Loop:

```python
n = 0
while n > -1:
    n += 1
```

This loop will never stop as n is always greater than -1.

Prevention:

- Ensure the loop's condition can eventually become False by modifying variables within the loop.
- You can manually interrupt infinite loops by pressing ctrl + c on PC or cmd + c on Mac

Comprehensions in Python

- **Definition**:
  - Comprehensions in Python are a compact and efficient way to create sequences (lists, dictionaries, sets) from other sequences.
  - They provide a more concise syntax compared to traditional loops, which reduces code complexity and improves readability.
- **Common Uses**:
  - List comprehension, dictionary comprehension, and set comprehension.
- **Basic Structure**:

```python
[Output for Input_sequence if Condition]
```

- **Example**:
  - Without comprehension:

```python
x = range(5)
y = []
for i in x:
    y.append(i**2)
print(y)  # [0, 1, 4, 9, 16]
```

  - With comprehension:

```python
y = [i**2 for i in range(5)]
print(y)  # [0, 1, 4, 9, 16]
```

Advanced List Comprehension

- **Conditional List Comprehension**:
  - You can add conditions within the comprehension to filter elements.
  - Example: Only storing even numbers:

```python
y = [i**2 for i in range(5) if i % 2 == 0]
print(y)  # [0, 4, 16]
```

- **Nested Loops in List Comprehension**:
  - Nested loops can also be used within comprehensions to handle more complex logic.
  - Example:

```python
y = [i + j for i in range(5) for j in range(2)]
print(y)  # [0, 1, 1, 2, 2, 3, 3, 4, 4, 5]
```

Dictionary Comprehension

- **Definition**:
  - Similar to list comprehension, but used for creating dictionaries.

- **Basic Structure**:

```python
{key: expression for key, value in iterable}
```

- **Example**:
  - Constructing a dictionary where values are cubed:

```python
x = {'a': 1, 'b': 2, 'c': 3}
new_dict = {key: value**3 for key, value in x.items()}
print(new_dict)  # {'a': 1, 'b': 8, 'c': 27}
```

Complexity and Big-O Notation

- **Definition of Complexity**:
  - Complexity of a function describes the relationship between the size of the input and the difficulty of running the function.
  - The input size is often denoted as *n* and can represent tangible factors like the length of an array.

- **Basic Operations**:
  - Operations such as additions, multiplications, and function calls are considered to assess complexity.
  - The number of basic operations is used as an approximation of a function's run time.

- **Big-O Notation**:
  - A common way to express the growth rate of an algorithm as input size increases.
  - It only considers the highest power term and ignores constants to focus on large-scale behavior.
  - Example: A function requiring 4n^2 + n + 1 basic operations is simplified to O(n^2).

Examples of Algorithm Complexity

- **Quadratic Complexity O(n^2):**
- Example: A nested loop structure where the total number of operations grows proportionally to n^2, such as:

```python
def f(n):
    out = 0
    for i in range(n):
        for j in range(n):
            out += i * j
    return out
```

- Total operations: n^2, leading to a complexity of O(n^2).

- **Linear Complexity O(n):**
- Example: An iterative Fibonacci function where the number of operations grows linearly with input size:

```python
def my_fib_iter(n):
    out = [1, 1]
    for i in range(2, n):
        out.append(out[i - 1] + out[i - 2])
    return out
```

- This function grows with O(n), as the for-loop runs n times.

Recursive Complexity and Exponential Time

- **Exponential Complexity O(2^n):**
  - Example: Recursive Fibonacci function where each call leads to two more recursive calls:

```python
def my_fib_rec(n):
    if n < 2:
        return 1
    else:
        return my_fib_rec(n-1) + my_fib_rec(n-2)
```

  - The number of recursive calls doubles with each increment in n, leading to an exponential growth rate of O(2^n).

- **Logarithmic Complexity O(log n):**
  - Example: A function that divides the input in half each time:

```python
def my_divide_by_two(n):
    out = 0
    while n > 1:
        n /= 2
        out += 1
    return out
```

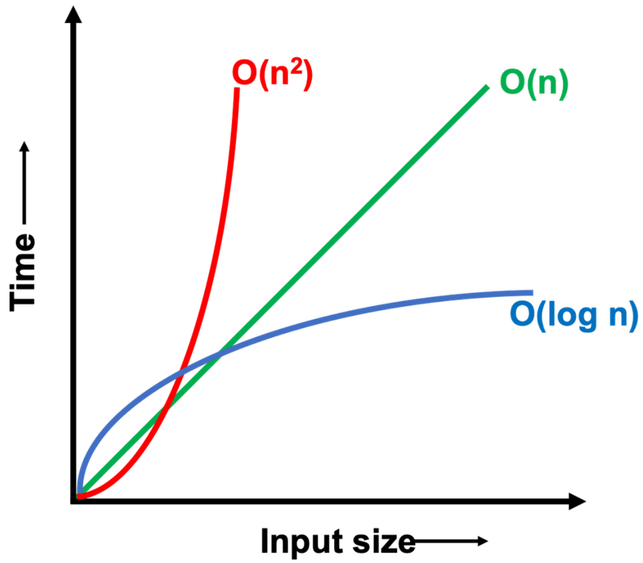
  - The complexity is O(log n) since the input size is halved during each iteration.

Why Complexity Matters

- **Complexity** measures how an algorithm's runtime or space requirements grow with input size.

- **Different types of complexity**:
  - Logarithmic: O(log n)
  - Linear: O(n)
  - Quadratic: O(n^2)
  - Exponential: O(2^n)

- **Impact of complexity on performance**:
  - **Exponential growth**: Adding resources barely increases the size of solvable problems.
  - **Polynomial time**: More manageable as problem size increases.
  - **Logarithmic algorithms**: Can handle significantly larger problems as resources scale.



Visual Representation:

- The relationship between input size and computation time (using logarithmic, linear, quadratic, and exponential growth) is shown through this graph.

Algorithm Scaling and Performance

Scaling Example: Exponential vs Polynomial Time

- **Exponential Algorithm (e.g., O(2^n)):**
  - Doubling computational power increases the solvable problem size by only one unit.
  - For large inputs, exponential algorithms quickly become infeasible.
- **Polynomial Algorithm (e.g., O(n^c)):**
  - Doubling resources can increase the solvable problem size by a factor of $\sqrt[c]{2}$.
  - More efficient for larger problems.

Real-world Impact:

- **Fibonacci Example:**
  - Recursive solution (exponential time) vs iterative solution (polynomial time).
  - In practice, polynomial algorithms are chosen for performance-critical applications.

NP-Complete Problems and Engineering

NP-Complete Problems:

- **Definition**: Problems for which no polynomial time algorithm is known.

- **Examples**:
  - **Traveling Salesman Problem (TSP)**
  - **Set Cover Problem**

- **Significance**:
  - These problems have significant applications in **logistics** and **operations research**.
  - They are crucial in fields like **encryption**, where the difficulty of solving NP-Complete problems secures data (e.g., in banking systems).

Profiling in Python

- **What is Profiling?**

Profiling in Python involves measuring the time and resources used by code to identify bottlenecks and optimize performance.

- **Purpose of Profiling:**
  - Helps developers focus on improving parts of code that consume most time or resources.
  - Useful in optimizing the performance of numerical and computational programs.

- **Common Python Profiling Tools:**
  - %time and %timeit: Magic commands used in Jupyter notebooks to check execution time of code.
  - Python’s built-in cProfile: Provides detailed reports on function calls and time spent.

Magic Commands for Time Measurement

- **Single Line Timing:**
  - %time measures the execution time of a single line of code.
  - Example: %time sum(range(200))

Output: CPU times: user 6 μs, sys: 1 μs, total: 7 μs | Wall time: 9.06 μs

- **Code Block Timing:**
  - %%time measures the execution time of all code within a Jupyter cell.
  - Example:

```python
%%time
s = 0
for i in range(200):
    s += i
```

- **Repetitive Timing:**
  - %timeit: Runs code multiple times and provides an average time.
  - Example: %timeit sum(range(200))

Output: 1.24 μs ± 70.6 ns per loop (1000000 loops each)

Advanced Profiling with Line Profiler

- **What is Line Profiling?**
  - Used to measure the time taken by individual lines of code.
  - Helps identify specific bottlenecks within a function.

- **Example: Using** line_profiler**:**
  - After installing the line_profiler package, use %lprun to profile specific functions.
  - Code Example:

```python
%load_ext line_profiler
%lprun -f slow_sum slow_sum(1000, 10000)
```

```
Timer unit: 1e-06 s

Total time: 6.1411 s
File: <ipython-input-20-cc5de53096ac>
Function: slow_sum at line 1

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     1                                           def slow_sum(n, m):
     2
     3      1001        301.0      0.3      0.0      for i in range(n):
     4                                                   # we create a size m array of random numbers
     5      1000      87876.0     87.9      1.4          a = np.random.rand(m)
     6
     7      1000        439.0      0.4      0.0          s = 0
     8                                                   # in this loop we iterate through the array
     9                                                   # and add elements to the sum one by one
    10  10001000    2463579.0      0.2     40.1          for j in range(m):
    11  10000000    3588901.0      0.4     58.4              s += a[j]
```

- **Output Analysis:**
  - Line-by-line summary of execution time.
  - Shows that lines 10 and 11 are responsible for the majority of the runtime (adding elements one by one to a sum).

Complexity and Big-O Notation

- **Definition of Complexity**:
  - Complexity of a function describes the relationship between the size of the input and the difficulty of running the function.
  - The input size is often denoted as *n* and can represent tangible factors like the length of an array.

- **Basic Operations**:
  - Operations such as additions, multiplications, and function calls are considered to assess complexity.
  - The number of basic operations is used as an approximation of a function's run time.

- **Big-O Notation**:
  - A common way to express the growth rate of an algorithm as input size increases.
  - It only considers the highest power term and ignores constants to focus on large-scale behavior.
  - Example: A function requiring 4n^2 + n + 1 basic operations is simplified to O(n^2).

Examples of Algorithm Complexity

- **Quadratic Complexity O(n^2):**
- Example: A nested loop structure where the total number of operations grows proportionally to n^2, such as:

```python
def f(n):
    out = 0
    for i in range(n):
        for j in range(n):
            out += i * j
    return out
```

- Total operations: n^2, leading to a complexity of O(n^2).

- **Linear Complexity O(n):**
- Example: An iterative Fibonacci function where the number of operations grows linearly with input size:

```python
def my_fib_iter(n):
    out = [1, 1]
    for i in range(2, n):
        out.append(out[i - 1] + out[i - 2])
    return out
```

- This function grows with O(n), as the for-loop runs n times.

Recursive Complexity and Exponential Time

- **Exponential Complexity O(2^n):**
  - Example: Recursive Fibonacci function where each call leads to two more recursive calls:

```python
def my_fib_rec(n):
    if n < 2:
        return 1
    else:
        return my_fib_rec(n-1) + my_fib_rec(n-2)
```

  - The number of recursive calls doubles with each increment in n, leading to an exponential growth rate of O(2^n).

- **Logarithmic Complexity O(log n):**
  - Example: A function that divides the input in half each time:

```python
def my_divide_by_two(n):
    out = 0
    while n > 1:
        n /= 2
        out += 1
    return out
```

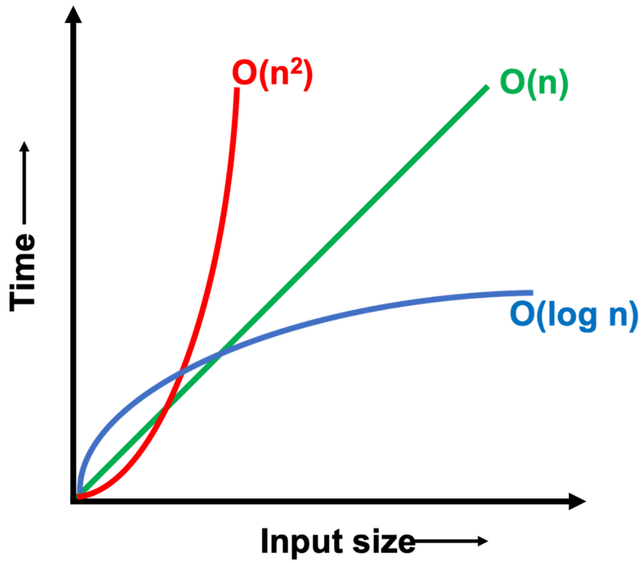
  - The complexity is O(log n) since the input size is halved during each iteration.

Why Complexity Matters

- **Complexity** measures how an algorithm's runtime or space requirements grow with input size.

- **Different types of complexity**:
  - Logarithmic: O(log n)
  - Linear: O(n)
  - Quadratic: O(n^2)
  - Exponential: O(2^n)

- **Impact of complexity on performance**:
  - **Exponential growth**: Adding resources barely increases the size of solvable problems.
  - **Polynomial time**: More manageable as problem size increases.
  - **Logarithmic algorithms**: Can handle significantly larger problems as resources scale.



Visual Representation:

- The relationship between input size and computation time (using logarithmic, linear, quadratic, and exponential growth) is shown through this graph.

Algorithm Scaling and Performance

Scaling Example: Exponential vs Polynomial Time

- **Exponential Algorithm (e.g., O(2^n)):**
  - Doubling computational power increases the solvable problem size by only one unit.
  - For large inputs, exponential algorithms quickly become infeasible.
- **Polynomial Algorithm (e.g., O(n^c)):**
  - Doubling resources can increase the solvable problem size by a factor of $\sqrt[c]{2}$.
  - More efficient for larger problems.

Real-world Impact:

- **Fibonacci Example:**
  - Recursive solution (exponential time) vs iterative solution (polynomial time).
  - In practice, polynomial algorithms are chosen for performance-critical applications.

NP-Complete Problems and Engineering

NP-Complete Problems:

- **Definition**: Problems for which no polynomial time algorithm is known.

- **Examples**:
  - **Traveling Salesman Problem (TSP)**
  - **Set Cover Problem**

- **Significance**:
  - These problems have significant applications in **logistics** and **operations research**.
  - They are crucial in fields like **encryption**, where the difficulty of solving NP-Complete problems secures data (e.g., in banking systems).

Profiling in Python

- **What is Profiling?**

Profiling in Python involves measuring the time and resources used by code to identify bottlenecks and optimize performance.

- **Purpose of Profiling:**
  - Helps developers focus on improving parts of code that consume most time or resources.
  - Useful in optimizing the performance of numerical and computational programs.

- **Common Python Profiling Tools:**
  - %time and %timeit: Magic commands used in Jupyter notebooks to check execution time of code.
  - Python’s built-in cProfile: Provides detailed reports on function calls and time spent.

Magic Commands for Time Measurement

- **Single Line Timing:**
  - %time measures the execution time of a single line of code.
  - Example: %time sum(range(200))

Output: CPU times: user 6 μs, sys: 1 μs, total: 7 μs | Wall time: 9.06 μs

- **Code Block Timing:**
  - %%time measures the execution time of all code within a Jupyter cell.
  - Example:

```python
%%time
s = 0
for i in range(200):
    s += i
```

- **Repetitive Timing:**
  - %timeit: Runs code multiple times and provides an average time.
  - Example: %timeit sum(range(200))

Output: 1.24 μs ± 70.6 ns per loop (1000000 loops each)

Advanced Profiling with Line Profiler

- **What is Line Profiling?**
  - Used to measure the time taken by individual lines of code.
  - Helps identify specific bottlenecks within a function.

- **Example: Using** line_profiler**:**
  - After installing the line_profiler package, use %lprun to profile specific functions.
  - Code Example:

```python
%load_ext line_profiler
%lprun -f slow_sum slow_sum(1000, 10000)
```

```
Timer unit: 1e-06 s

Total time: 6.1411 s
File: <ipython-input-20-cc5de53096ac>
Function: slow_sum at line 1

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     1                                           def slow_sum(n, m):
     2
     3      1001        301.0      0.3      0.0      for i in range(n):
     4                                                   # we create a size m array of random numbers
     5      1000      87876.0     87.9      1.4          a = np.random.rand(m)
     6
     7      1000        439.0      0.4      0.0          s = 0
     8                                                   # in this loop we iterate through the array
     9                                                   # and add elements to the sum one by one
    10  10001000    2463579.0      0.2     40.1          for j in range(m):
    11  10000000    3588901.0      0.4     58.4              s += a[j]
```

- **Output Analysis:**
  - Line-by-line summary of execution time.
  - Shows that lines 10 and 11 are responsible for the majority of the runtime (adding elements one by one to a sum).

Understanding Number Representation Systems

Decimal System (Base-10)

- Most familiar number system, composed of digits 0 to 9.
- Numbers are represented as a sum of powers of 10.

- **Example**:

147.3 in base-10 is expanded as:

1 x 10^2 + 4 x 10^1 + 7 x 10^0 + 3 x 10^{-1}

Base-N Systems

- Any number system can use a base N , where N is the number of unique digits in the system.
- Example (Base-3):

121 (base-3) = 1 x 3^2 + 2 x 3^1 + 1 x 3^0 = 16 (base-10)

Binary Numbers (Base-2)

- A key number system for computers, using only digits 0 and 1.
- Each digit is a coefficient of a power of 2, also known as a bit.

Conversion Examples

- **11 (Base-10)** in binary:

11 (base-10) = 1011 (base-2)

- **37 (Base-10)** in binary:

37 (base-10) = 100101 (base-2)

Binary Arithmetic

Binary Addition and Multiplication

- **Addition Example:**

Adding 37 and 17 in binary:

100101 + 10001 = 110110 , which is 54 in decimal.

- **Multiplication Example:**

Multiplying 37 by 17 in binary:

100101 x 10001 = 1001110101 , which is 629 in decimal.

Why Binary is Useful in Computers

•Operations on binary numbers are implemented using logic gates (AND, OR, NOT), which are fast for computers to execute.

Limitations

•Binary numbers are stored in fixed bit lengths (e.g., 32-bit systems), restricting the range of representable numbers.

Floating Point Numbers

What are Floating Point Numbers?

- Floating point numbers are used to represent real numbers in a computer, providing a wide range of values while using a fixed number of bits.
- They are crucial for scientific and engineering calculations due to the need for high precision and a broad range of values.

Components of Floating Point Numbers:

- **Sign (s):** Determines whether the number is positive or negative.
- **Exponent (e):** The power of 2, indicating the scale of the number.
- **Fraction (f):** The significant or mantissa, a value between 1 and 2, indicating the precision of the number.

IEEE 754 Double Precision Format:

- Python maps floats to the IEEE754 double precision format, which uses **64 bits**:
  - **1 bit for the sign**, **11 bits for the exponent**, and **52 bits for the fraction**.
- The exponent has a bias of 1023, meaning e - 1023 normalizes the exponent.

Calculations and Precision

Representation of a Floating Point Number:

- A float can be represented as:

n = (-1)^s x 2^(e-1023) x (1 + f)

Example:

- For the binary number:

1 10000000010 1000000000000000000000000000000000000000000000000000

- **Sign** s = 1 (negative)
- **Exponent** e = 1026 (decimal: 3)
- **Fraction** f = 1.5

The exponent in decimal is $1 \cdot 2^{10} + 1 \cdot 2^{1} - 1023 = 3$. The fraction is $1 \cdot \frac{1}{2^1} + 0 \cdot \frac{1}{2^2} + \ldots = 0.5$. Therefore $n = (-1)^1 \cdot 2^3 \cdot (1 + 0.5) = -12.0$ (base10). See the following figure for details.

Precision Considerations:

•The gap between two adjacent floating point numbers increases with the magnitude of the numbers.

•The function numpy.spacing(x) can be used to find the distance (gap) between numbers, which grows as x increases.

Special Cases and Limitations

Special Floating Point Cases:

- **Subnormal Numbers:** Occur when the exponent is zero, allowing representation of very small numbers.
- **Infinity and NaN (Not a Number):** Occur when the exponent is all 1s (2047). If the fraction is zero, the result is infinity; if not, the result is NaN.

Handling Overflow and Underflow:

- **Overflow:** Occurs when a result exceeds the largest representable number, handled by Python as inf.
- **Underflow:** Occurs when a result is smaller than the smallest representable subnormal number, handled by Python as 0.

Understanding Round-Off Errors in Numerical Computations

What are Round-Off Errors?

- Round-off error occurs due to the approximation of numbers when stored in a computer's memory.
- These errors happen because computers represent floating-point numbers in binary, which cannot always perfectly represent decimal numbers.
- **Key Concept:** The difference between the actual value of a number and its approximation in the system is called a round-off error.

Representation Error:

- Common cause of round-off errors.
- Example: pi (3.14159…) or 1./3. (0.33333…) can't be perfectly represented, leading to small but cumulative errors in calculations.
- Rounding Multiple Times: Successive rounding can exacerbate errors. For instance:
  Rounding 4.845 to two decimals: 4.85, then to one decimal: 4.9, leading to a larger cumulative error than rounding directly to one decimal: 4.8.

Examples of Round-Off Errors in Python

Floating Point Arithmetic Issues

- Operations in Python demonstrate how floating-point numbers accumulate round-off errors.
- Example:
  - 4.9 - 4.845 in Python yields 0.05500000000000604, not exactly 0.055.
  - 0.1 + 0.2 + 0.3 != 0.6 due to rounding approximations.

Accumulation of Round-off Error:

- Errors accumulate over repetitive calculations.
- Example: Adding and subtracting 1/3 multiple times:
  - If done once: 1 + 1/3 - 1/3 evaluates to 1
  - Repeated 100 times then evaluate: Result becomes 1.0000000000000002
  - As the number of repetitions increases, the error increases proportionally.

TXT Files in Python

- **Text Files Overview**:
  - Text files contain only plain text and are often used to store data.
  - Common file extension: .txt
  - Programs expect text files to follow a specific format.

- **Python** open() **Function**:
  - Used to work with text files in Python by returning a file object.
  - Syntax: f = open(filename, mode)
  - Common modes:
    - 'r': Read mode (default).
    - 'w': Write mode (creates a new file if not existing).
    - 'a': Append mode.
    - 'r+': Read and write mode.
    - 'w+': Write and read, discarding previous content.
    - 'a+': Append and read mode.

- **Best Practice**: Always close files after use with f.close() to avoid data loss.

- **Writing to a File**:
  - Example:

```python
f = open('test.txt', 'w')
for i in range(5):
    f.write(f"This is line {i}\n")
f.close()
```

- Writes lines to test.txt and saves them.

- **Appending to a File**:
  - Example:

```python
f = open('test.txt', 'a')
f.write("This is another line\n")
f.close()
```

- **Reading from a File**:
  - Reading entire content:

```python
f = open('test.txt', 'r')
content = f.read()
f.close()
```

- Reading line by line:

```python
f = open('test.txt', 'r')
contents = f.readlines()
f.close()
```

Working with Numbers and Arrays in TXT Files

- **Saving Numerical Data**:
  - Using numpy to handle arrays:

```python
import numpy as np
arr = np.array([[1.20, 2.20, 3.00], [4.14, 5.65, 6.42]])
np.savetxt('my_arr.txt', arr, fmt='%.2f', header='Col1 Col2 Col3')
```

  - Saves a 2D array to my_arr.txt with a custom header.

- **Loading Numerical Data**:
  - Reading arrays back into Python:

```python
my_arr = np.loadtxt('my_arr.txt')
```

- Automatically skips the header and loads the array back into memory.

CSV Files in Python

- **What are CSV Files?**
  - CSV stands for *Comma-Separated Values*, a simple and widely used file format for storing tabular data.
  - Each line in a CSV file represents a data record, and each record consists of one or more fields separated by commas.
  - CSV files are highly compatible with applications like *Microsoft Excel*, making them easy to visualize.

- **Python's CSV Handling:**
  - Python provides several tools to handle CSV files, including the csv module and third-party libraries like numpy and pandas.
  - In scientific applications, CSV files often store large tables of numerical data, which can be efficiently managed using Python.

Writing and Reading CSV Files Using Numpy

- **Writing a CSV File:**
  - Python’s numpy package can be used to write CSV files efficiently.
  - Example code to generate and save random data to a CSV file:

```python
import numpy as np
data = np.random.random((100, 5))
np.savetxt('test.csv', data, fmt='%.2f', delimiter=',', header='c1, c2, c3, c4, c5')
```

- The code creates a file with 100 rows and 5 columns of random numbers, saved in CSV format.

- **Reading a CSV File:**
  - The same numpy package allows reading CSV files into arrays for easy data manipulation:

```python
my_csv = np.loadtxt('test.csv', delimiter=',')
print(my_csv[:5, :])
```

  - This reads the CSV back into a numpy array and prints the first five rows.

Beyond Numpy for CSV Handling

- **Advanced CSV Handling with Pandas:**
  - While numpy works well for numerical data, pandas provides more advanced capabilities, especially for working with labeled data and DataFrames.
  - pandas supports easier manipulation of CSV data, including data cleaning, filtering, and summarization.

  - Example code to use pandas for CSV operations:

```python
import pandas as pd
df = pd.read_csv('test.csv')
print(df.head())
```

  - This provides a richer interface for working with CSV files, especially for tabular data analysis.

Pickle Files

Overview:

- **Pickle** is a Python module used for **serializing** and **de-serializing** Python object structures.
- Serialization involves converting objects in memory (e.g., dictionaries, lists) into a **byte stream** for storage on disk as a **binary file**.
- De-serialization is the reverse process, where the byte stream is converted back into its original object form.

When to Use:

- Useful for saving Python-specific data types, like dictionaries, lists, or tuples, for later use.
- Ideal for sharing data with other Python programs but has limitations when sharing across different programming languages.

Key Functions:

- pickle.dump(): Serializes an object and writes it to a file.
- pickle.load(): Reads a pickle file and de-serializes it back into the original Python object.

Pickling and Unpickling Process

How to Write a Pickle File:

- First, import the pickle module: import pickle.
- Example of serializing a dictionary:

```python
dict_a = {'A':0, 'B':1, 'C':2}
pickle.dump(dict_a, open('test.pkl', 'wb'))
```

- The file is saved in **binary** format ('wb' stands for write-binary mode).

How to Read a Pickle File:

- De-serialize the binary file back to a Python object:

```python
my_dict = pickle.load(open('./test.pkl', 'rb'))
```

- The mode 'rb' signifies read-binary mode.

Limitations and Compatibility

Compatibility Issues:

- **Not a universal format**: While TXT and CSV files can be used across languages (e.g., R, MATLAB, Java), Pickle is specific to Python.
- **Python Version Differences**: Pickle files created in Python 2 may not be compatible with Python 3. Use encoding='latin1' in the pickle.load() function for compatibility with Python 2 files.

Example:

- Reading a Python 2 Pickle file in Python 3:

```python
new_dict = pickle.load(open(filename, 'rb'), encoding='latin1')
```

Key Warning:

- Avoid using Pickle files if you need cross-language compatibility or long-term data preservation, as formats may change across Python versions.

JSON Files in Python

- **What is JSON?**
  - JSON stands for JavaScript Object Notation.
  - A lightweight, language-independent data format often used for data exchange.
  - JSON files typically end with the extension .json.

- **Key Characteristics**
  - Stores data in key-value pairs, similar to Python dictionaries.
  - Supports various data types: strings, numbers, lists, and nested objects.
  - Compact and faster than Pickle for serialization (Python object to byte stream).

- **Why Use JSON?**
  - Language agnostic: Can be read by many programming languages.
  - More efficient disk usage compared to Pickle.
  - JSON is ideal for cross-platform data exchange, making it widely adopted in APIs.

Working with JSON Files in Python

- **Writing to a JSON File**
  - Use Python's json library to easily handle JSON.
  - Example:

```python
import json
school = {
    "school": "UC Berkeley",
    "address": {
        "city": "Berkeley",
        "state": "California",
        "postal": "94720"
    },
    "list": ["student 1", "student 2", "student 3"]
}
json.dump(school, open('school.json', 'w'))
```

- **Reading from a JSON File**
  - JSON can be read using json.load():

```python
my_school = json.load(open('school.json', 'r'))
print(my_school)
```

  - The loaded object is a dictionary-like structure, allowing easy access to keys and values.

HDF5 Files in Python

- **What is HDF5?**
  - HDF5 (Hierarchical Data Format) is a powerful, open-source binary file format designed for handling large amounts of data. It allows fast reading/writing and supports parallel I/O.
  - HDF5 files store two main types of objects:
    - **Datasets**: Array-like collections of data (similar to NumPy arrays).
    - **Groups**: Folder-like structures that can contain datasets or other groups.
  - HDF5 supports attributes to describe properties of datasets and groups, creating a hierarchical organization similar to a filesystem.

- **Why use HDF5?**
  - No size limit for data storage.
  - Efficient storage and retrieval through built-in optimizations.
  - Commonly used in scientific computing for managing large datasets.

- **Python Support for HDF5:**
  - Libraries: **h5py** and **PyTables**.
  - **h5py** allows Python users to create, read, and manipulate HDF5 files with ease.

Practical Example – Storing Data in HDF5 Format

- **Scenario**: Monitoring accelerations and GPS locations in the Bay Area.
  - **Data**:
    - Two accelerometers (Berkeley and Oakland) with different sampling rates.
    - One GPS station in San Francisco sampling every 60 seconds.
- **Steps for Creating HDF5 File**:
  - Generate random data using NumPy for sensors in different locations.
  - Create an HDF5 file (station.hdf5), and organize the data in groups and subgroups (/acc/1/data, /gps/1/data).
  - Use **attributes** to store additional metadata (e.g., location, start time, sampling interval).

- **Sample Code**:

```python
import numpy as np
import h5py

# Data generation
acc_1 = np.random.random(1000)
location_1 = 'Berkeley'
start_time_1 = 1542000276
dt_1 = 0.04

hf = h5py.File('station.hdf5', 'w')
hf['/acc/1/data'] = acc_1
hf['/acc/1/data'].attrs['dt'] = dt_1
hf['/acc/1/data'].attrs['start_time'] = start_time_1
hf['/acc/1/data'].attrs['location'] = location_1
hf.close()
```

Reading HDF5 Files

- **How to Read Data from HDF5**:
  - Open the file in read mode ('r').
  - Use the keys() function to explore the structure of the file (e.g., see the groups acc and gps).
  - Access datasets directly using paths (e.g., hf_in[‘/acc/1/data']).

- **Example**:

```python
hf_in = h5py.File('station.hdf5', 'r')
acc_data = hf_in['/acc/1/data'][:10]  # Access first 10 entries
acc_attrs = hf_in['/acc/1/data'].attrs['location']  # Access metadata
print(acc_data, acc_attrs)  # Output data and location
```

- **Key Benefits**:
  - Structured organization of data, making it easy to navigate large datasets.
  - Metadata retrieval through attributes enhances data clarity.

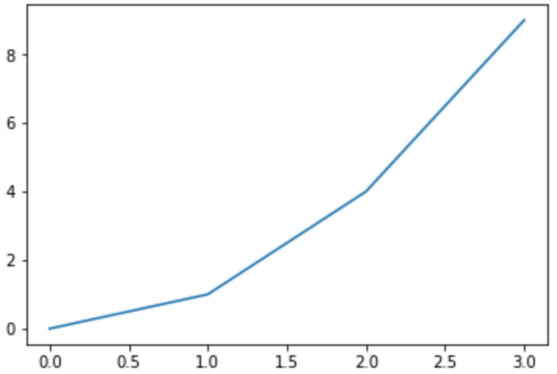
2D Plotting in Python

- **Overview**:

Python's matplotlib package is essential for creating visual plots. It's commonly used in scientific and engineering contexts for data visualization.

- **Basic Plotting Function**:
  - The function plot(x, y) creates a 2D plot from two lists or arrays, x and y, representing the coordinates.
  - Example:

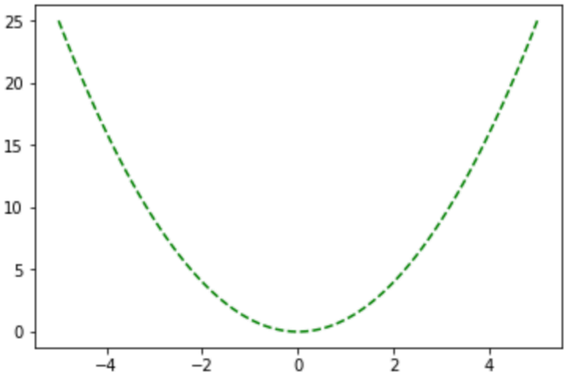
```python
x = [0, 1, 2, 3]
y = [0, 1, 4, 9]
plt.plot(x, y)
plt.show()
```



- **Interactive Features**:
  - In Jupyter Notebooks, %matplotlib notebook allows interactive features like panning and zooming.
  - %matplotlib inline disables these features for static images.

- **Customization**:
  - Line and marker styles can be customized, e.g., 'ro' plots red circles, while 'g--' creates a dashed green line.

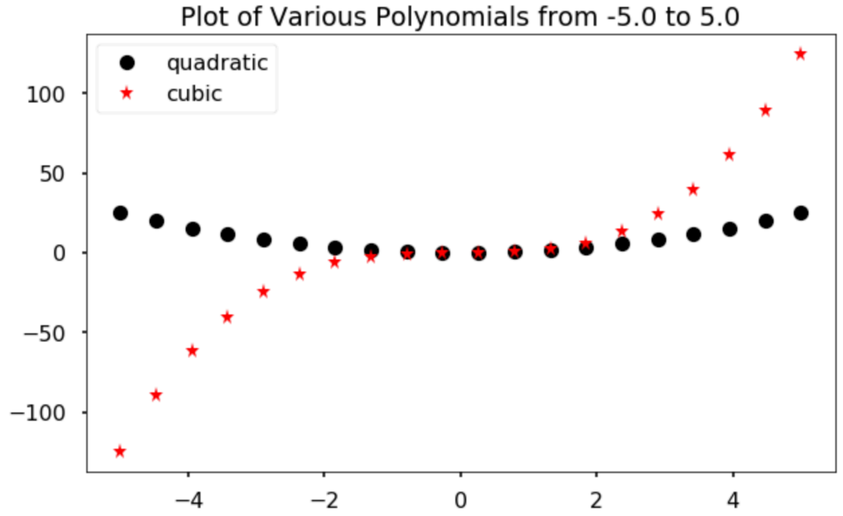
```python
x = np.linspace(-5,5,100)
plt.plot(x, x**2, 'g--')
plt.show()
```



Enhancing 2D Plots with Titles, Labels, and Legends

- **Adding Titles and Labels**:
  - Titles and axis labels provide context to the plot and can be customized for clarity.
  - Example:

```python
plt.title('Quadratic Function')
plt.xlabel('X axis')
plt.ylabel('Y axis')
```



- **Figure Size**:
  - Customizing the plot size can improve readability.

```python
plt.figure(figsize=(10, 6))
```

- **Legends and Styles**:
  - Multiple datasets can be plotted in one figure. Adding a label argument to plot() and using plt.legend() helps distinguish different datasets.
  - Styles, such as seaborn and ggplot, can be applied globally with plt.style.use('seaborn-poster').

Example of multiple plots with legends:

```python
x = np.linspace(-5,5,100)
plt.plot(x, x**2, 'ko', label='Quadratic')
plt.plot(x, x**3, 'r*', label='Cubic')
plt.legend(loc=2)
plt.show()
```

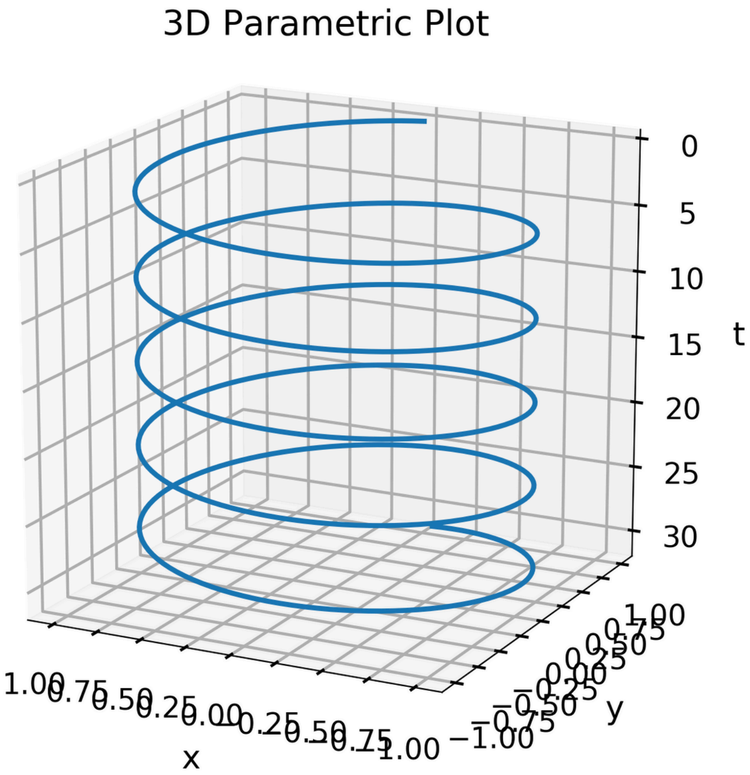
3D Plotting with Matplotlib

- **Matplotlib** is a powerful plotting library in Python. To plot 3D figures, the mplot3d toolkit must be imported and axes defined (ax = plt.axes(projection='3d') commonly).
- The 3D plot functionalities include:
  - Line plots using plot3D().
  - Scatter plots using scatter().
  - Surface plots using plot_surface().

Example 1: 3D Line Plot

- **Goal**: Create a 3D line plot using parametric data.
- **Key Code**:

```python
import numpy as np
from mpl_toolkits import mplot3d
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(10,10))
ax = plt.axes(projection='3d')
t = np.arange(0, 10*np.pi, np.pi/50)
x = np.sin(t)
y = np.cos(t)
ax.plot3D(x, y, t)
ax.set_title('3D Parametric Plot')
plt.show()
```



- Result: A 3D parametric plot of (x, y, t) , with grid and axis labels enabled.

Surface and Scatter Plotting in 3D

3D Surface Plot:

- **Purpose**: To visualize functions of two variables Z = f(X, Y) , where X and Y are generated from np.meshgrid().
- Steps:

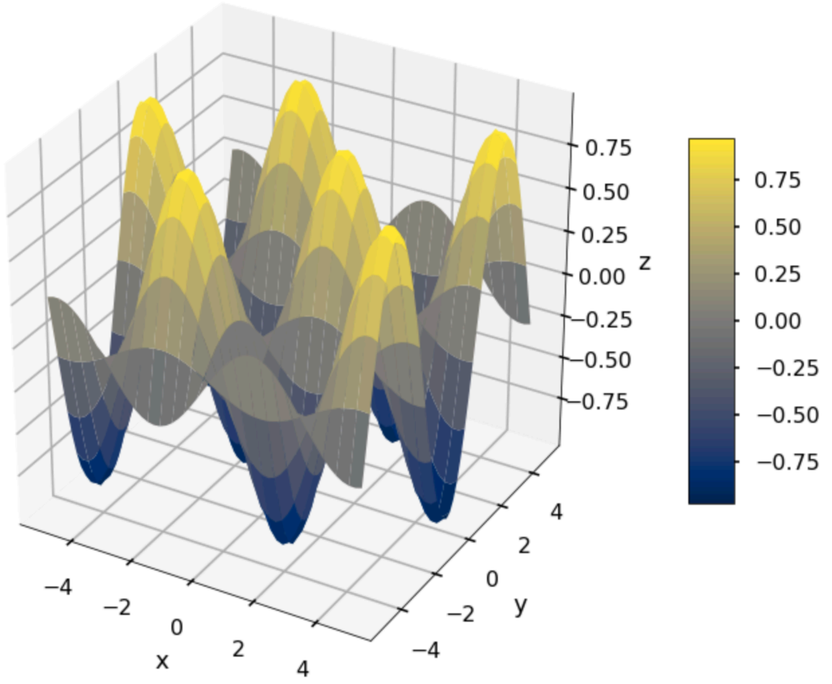
1.Create a meshgrid using lists/arrays for X and Y.

2.Compute Z values using the function Z = sin(X) * cos(Y) .

3.Plot the surface with plot_surface(), using a colormap (e.g., plt.cm.cividis).

- **Key Code:**

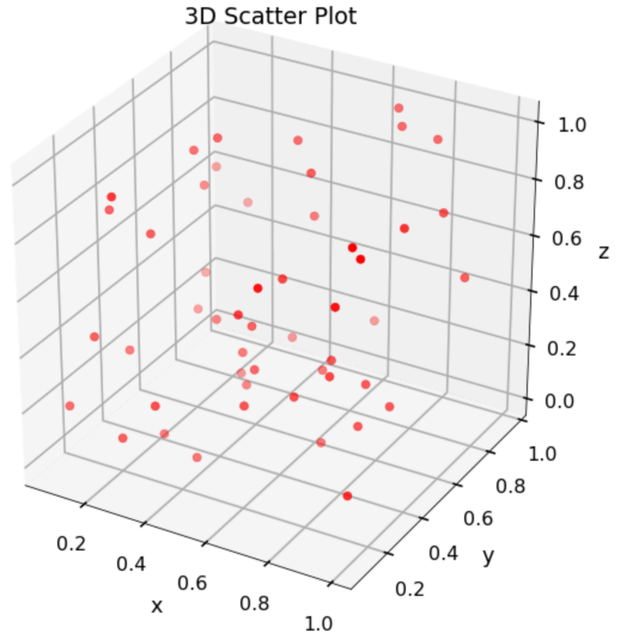
```python
x = np.arange(-5, 5.1, 0.2)
y = np.arange(-5, 5.1, 0.2)
X, Y = np.meshgrid(x, y)
Z = np.sin(X)*np.cos(Y)
ax.plot_surface(X, Y, Z, cmap=plt.cm.cividis)
plt.show()
```



3D Scatter Plot:

- **Goal**: Create a 3D scatter plot with randomly generated data.
- **Key Code**:

```python
x = np.random.random(50)
y = np.random.random(50)
z = np.random.random(50)
ax.scatter(x, y, z, c='r', s=50)
plt.show()
```



Working with Maps in Python

- **Purpose**: Engineers and scientists often need to interact with maps to visualize spatial data.

- **Python Packages for Maps**:
  - **Basemap**: Initially used with Matplotlib, deprecated after 2016.
  - **Cartopy**: Replaced Basemap, offers robust map projections and plotting functionalities.
  - **Folium**: Enables interactive web-based maps.

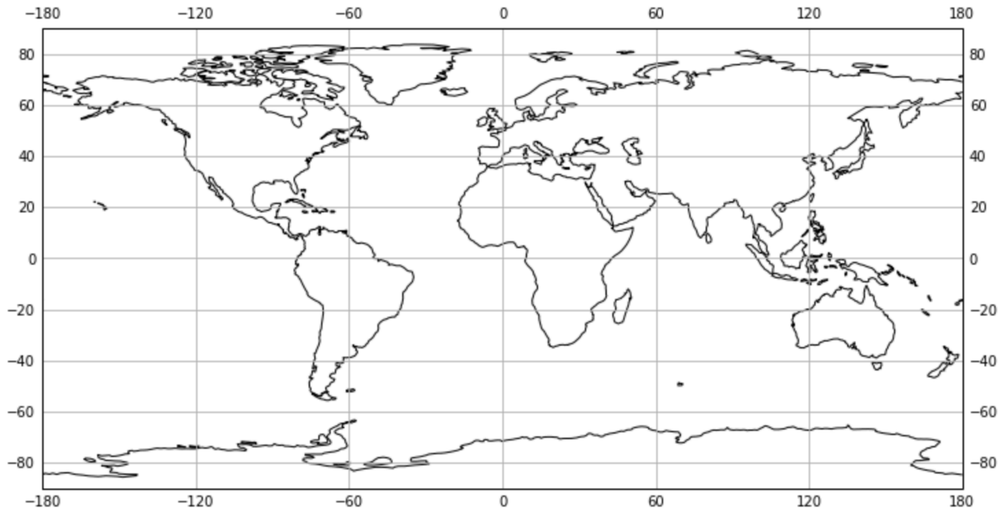
- **Basics of Mapping**:
  - A map is a **2D plot** with specific projections.
  - **Longitude**: Ranges from -180 to 180 degrees (East-West position).
  - **Latitude**: Ranges from -90 to 90 degrees (North-South position).

Plotting Maps with Cartopy

- **Map Projections**: Cartopy supports various projections, with **Plate Carrée** being a common choice.

- **Steps to Plot a Map**:
  - Use **Matplotlib** for creating plots.
  - Set map projection: ax = plt.axes(projection=ccrs.PlateCarree())
  - Add features like coastlines and gridlines.

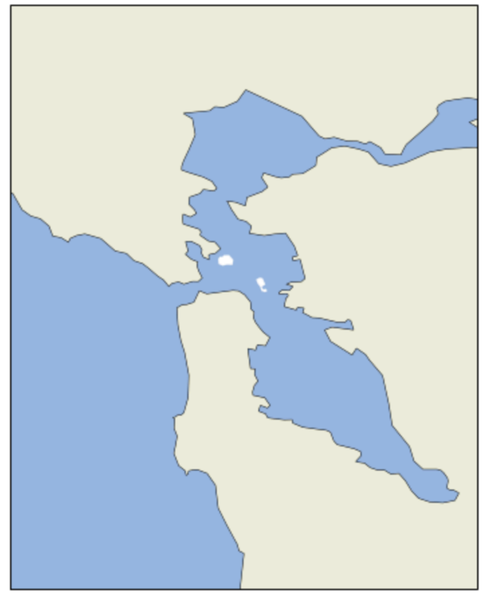
```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
ax = plt.axes(projection=ccrs.PlateCarree())
ax.coastlines()
ax.gridlines(draw_labels=True)
plt.show()
```



- **Enhancements**: Background maps can be improved using downsampled Natural Earth images.

Advanced Map Features in Cartopy

- **Adding Features**:
  - **ax.set_extent()**: Zoom into specific regions by specifying longitude and latitude limits.
  - Add **land, ocean, borders, lakes, rivers** using Cartopy's built-in features.



```python
import cartopy.feature as cfeature

ax.add_feature(cfeature.LAND)
ax.add_feature(cfeature.OCEAN)
ax.add_feature(cfeature.BORDERS)
ax.add_feature(cfeature.LAKES, alpha=0.5)
```

- **Example**: Plotting high-resolution San Francisco Bay area with 10m resolution coastlines and adding geographical features.

```python
ax.set_extent([-122.8, -122, 37.3, 38.3])
```

Animations in Python

- **Definition**:
  - An animation is a sequence of still frames displayed rapidly to create the illusion of motion.
  - Useful for better visual representation of dynamic systems compared to static plots.
- **Python Tools**:
  - The primary tool for creating animations in Python is the matplotlib.animation.Animation class.
  - This class offers a framework to build animation functionality.
- **Basic Steps for Creating an Animation**:

1. **Define Metadata**:
   - Define movie metadata such as title, artist, and comments using manimation.writers['ffmpeg'].
2. **Initialize Background**:
   - Set up static background elements like axis labels and curves.
3. **Update Frames**:
   - Inside a loop, update objects (like a moving point) for each frame.

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# Data generation
n = 1000
x = np.linspace(0, 6 * np.pi, n)
y = np.sin(x)

# Create the figure and axis
fig, ax = plt.subplots()
ax.set_xlim(0, 6 * np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel('x')
ax.set_ylabel('sin(x)')

# Plot the sine wave (background)
sine_line, = ax.plot(x, y, 'b')

# Create an empty red circle that will move
red_circle, = ax.plot([], [], 'ro', markersize=10)

# Initialize the animation
def init():
    red_circle.set_data([], [])
    return red_circle,

# Update function for animation
def update(frame):
    red_circle.set_data([x[frame]], [y[frame]])
    return red_circle,
```

```python
# Create the animation
ani = animation.FuncAnimation(
    fig,
    update,
    frames=n,
    init_func=init,
    interval=15,
    blit=True
)

plt.show()
```

Example: Sine Wave Animation

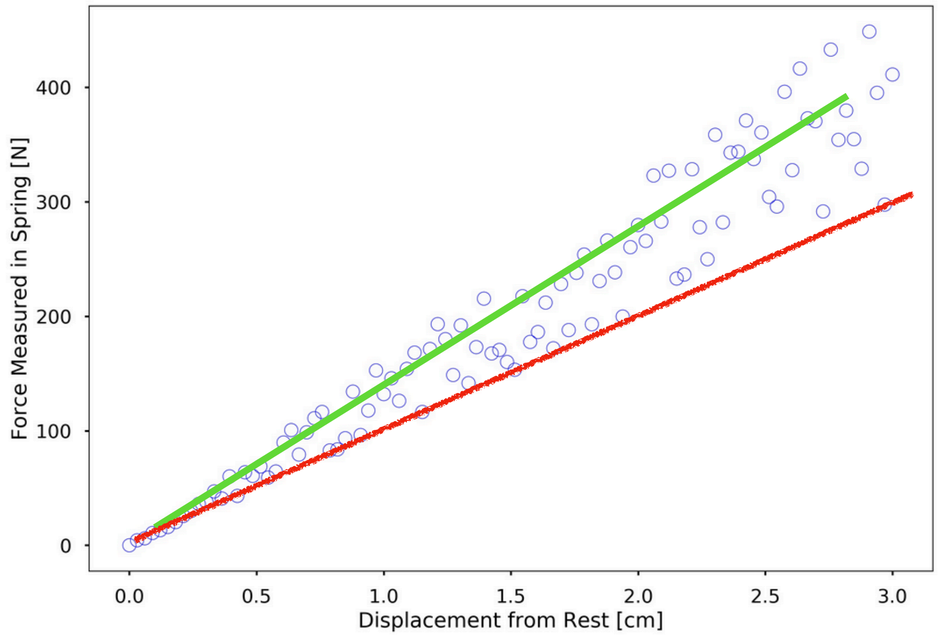
- **Objective**: Animate a red circle moving along a blue sine wave.
- **Key Components of Code**:
  - Import required libraries: numpy, matplotlib.pyplot, and matplotlib.animation.
  - Set up metadata for the movie (e.g., frames per second: fps=15).
  - **Sine Wave Setup**:
    - Use np.linspace() to generate 1000 data points for x, and np.sin(x) for the sine curve.
  - **Figure Initialization**:
    - Plot the sine wave and an empty placeholder for the red circle.
  - **Frame Updates**:
    - For each frame in a loop, update the position of the red circle along the sine wave.
- **Saving the Animation**:
  - Output the animation as an mp4 file with a resolution of 100 dpi using writer.saving().

python

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as manimation

# Data generation
n = 1000
x = np.linspace(0, 6*np.pi, n)
y = np.sin(x)

# Define movie metadata
FFMpegWriter = manimation.writers['ffmpeg']
metadata = dict(title='Sine Wave Animation', artist='Matplotlib', comment='A red
writer = FFMpegWriter(fps=15, metadata=metadata)

# Initialize the movie
fig, ax = plt.subplots()
ax.set_xlim(0, 6*np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel('x')
ax.set_ylabel('sin(x)')

# Plot the sine wave
sine_line, = ax.plot(x, y, 'b')
red_circle, = ax.plot([], [], 'ro', markersize=10)

# Update the frames for the movie
with writer.saving(fig, "sine_wave_animation.mp4", 100):
    for i in range(n):
        red_circle.set_data(x[i], y[i])  # Update the red circle's position
        writer.grab_frame()  # Capture the frame

print("Animation saved as sine_wave_animation.mp4")
```

Least Squares Regression

- **What is Least Squares Regression?**
  - A mathematical method used to find the best-fitting line (or model) through a set of data points by minimizing the sum of the squares of the differences between the observed data and the predicted values.
  - Commonly used in scientific and engineering problems to estimate unknown parameters from experimental data.
- **Motivation**
  - In practical situations, model parameters, like a spring stiffness in a set of spring experiments, are not directly known. Instead, we rely on experimental data that usually contains noise.
  - Least Squares Regression helps in determining the most likely model parameters, even when the data contains errors or noise.
- **Example**: Determining spring stiffness k from experimental data of force F and displacement x in a linear spring system: $F = kx$.



Methods of Deriving Least Squares Regression

- **Two Key Derivations:**

  - **Linear Algebra Approach**: Solves the regression problem by constructing a system of linear equations based on the data and minimizing the residual sum of squares.

  - **Multivariable Calculus Approach**: Minimizes the residual sum of squares by taking derivatives and setting them to zero to find the optimal parameters.

- **Linear Model Example**: For a linear relationship $y = mx + b$, the goal is to minimize the error E, where

$$E = \sum (y_{\text{observed}} - y_{\text{predicted}})^2$$

  - The solution yields the optimal slope m and intercept b.

Least Squares Regression in Python and Nonlinear Extensions

- **Implementing in Python**
  - Python libraries like **NumPy** and **SciPy** provide efficient functions to perform least squares regression.
  - Example: Using numpy.linalg.lstsq() to solve a linear system or scipy.optimize.curve_fit() for nonlinear models.

- **Nonlinear Regression**
  - Least Squares can also be extended to nonlinear functions, such as fitting data to exponential, logarithmic, or polynomial models.
  - The process involves linearizing the problem or using iterative optimization techniques to minimize the error for complex models.

- **Definition**: Least Squares Regression aims to find an estimation function y_hat(x) that best fits a set of data points (x_i, y_i). The estimation function is typically a linear combination of ‘basis functions’.
- **Estimation Function**: The general form of the estimation function is:

$$\hat{y}(x) = \sum_{i=1}^{n} \alpha_i f_i(x)$$

$$\begin{aligned}
\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\
\hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\
&\cdots \\
\hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).
\end{aligned}$$

(m x n matrix)

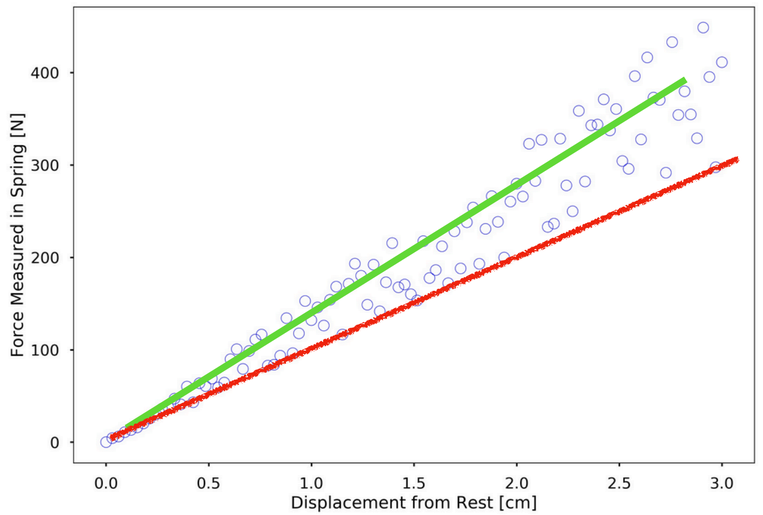
- alpha_i are the parameters to be determined.
- f_i(x) are the basis functions which must be linearly independent (cannot be linear combination of each other).
- Objective: To minimize the total squared error between the estimated and actual data points. The squared error E is defined as a sum over data points:

$$E = \sum_{i=1}^{m} (y_i - \hat{y}(x_i))^2$$

Where:

- y_i is the actual dependent variable.
- y_hat (x_i) is the estimated value from the model at position x_i.

- **Spring System Example**: To better understand the concept, let us consider a force-displacement relationship for a linear spring.



- The **force** F is related to the displacement x by Hooke's Law:

$$F(x) = kx$$

- k is the spring stiffness, which serves as the model parameter.

- Estimation Function: In this case, we know that a good estimation function is simply:

$$\hat{y}(x) = kx$$

- The only basis function here is f(x) = x, and the parameter to be determined is k.

- Goal: Minimize the squared error to find the optimal value of k that fits the force-displacement data.

Minimizing the Total Squared Error

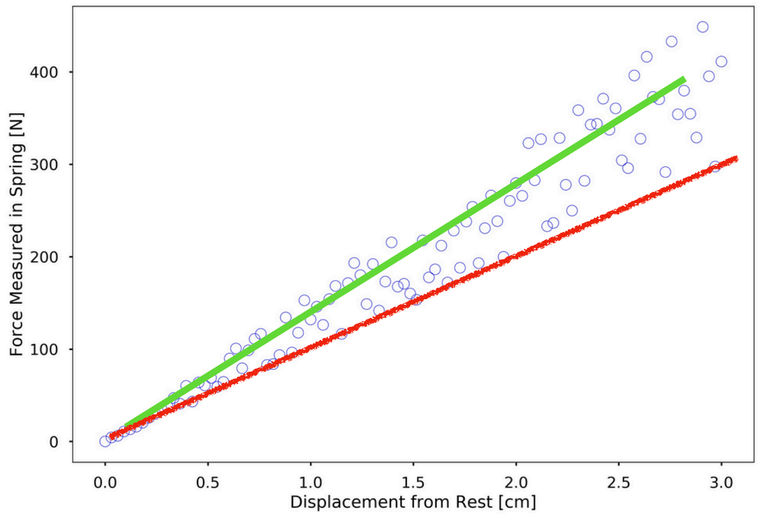
- **Error Definition**: The individual errors (residuals) are defined as:

$$e_i = y_i - \hat{y}(x_i)$$

The objective is to minimize the squared error E, which is essentially the (L2) norm of the residual m-dimensional vector e:

$$E = \|e\|_2^2$$

$$E = \sum_{i=1}^{m} (y_i - \hat{y}(x_i))^2$$



Least Squares Regression - Algebra approach

Objective:

To find the best-fit line for a set of data points by minimizing the total squared error between the observed and estimated values.

Key Concepts:

- **Data Points**: Represented as (x_i, y_i), where x_i is the independent variable and y_i is the dependent variable.
- Goal: Estimate the relationship between x_i and y_i using a linear model Y_hat = A.beta, where A is the matrix of predictor functions, and beta is the vector of alpha coefficients.

Total Squared Error: Minimize the error

$$E = \|Y - \hat{Y}\|^2$$

where Y_hat is the predicted value from the regression model.

$$\hat{Y}_i = \hat{y}(x_i)$$

β be a column vector such that $\beta_i = \alpha_i$

$$\hat{Y} = A\beta$$

Derivation Using Linear Algebra

Vector Notation:

- Let Y be a column vector of observed values and A be a matrix whose columns represent the predictor variables.
- The regression problem becomes solving A.beta = Y by minimizing the total squared error.

Key Equation:

- The total squared error is given by

$$E = \|Y - A\beta\|^2$$

To minimize E, we seek the beta that makes A.beta as close as possible to Y. This can be shown to result in the equation:

$$A^T(Y - A\beta) = 0$$

Recall: $\mathrm{dot}(v, w) = v^T w$

Solution:

- Solving for beta gives the normal equation:

$$\beta = (A^T A)^{-1} A^T Y$$

This is the least squares regression formula, where A^T A is invertible if A has linearly independent columns.

Least Squares Regression - Calculus approach

Recall the Key Concept:

- The goal of least squares regression is to find the set of parameters alpha_j that minimizes the total error E, represented as the sum of squared differences between the predicted values and actual data points.

Equation for Total Error:

$$E = \sum_{i=1}^{m} (y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i))^2$$

Where:

- m = number of data points
- n = number of basis functions
- alpha_j = unknown parameters
- f_j(x_i) = basis functions evaluated at data points x_i
- y_i = actual/target values

Concept of Minimization:

- The error function E is minimized by solving a system of linear equations derived by setting the partial derivatives of E with respect to a specific alpha_k to zero.

Derivation Using Multivariable Calculus

Steps in Derivation:

1. **Partial Derivatives:**

   • For each parameter alpha_k, take the partial derivative of the error function E with respect to alpha_k and set it to zero:

$$\frac{\partial E}{\partial \alpha_k} = 2 \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right) f_k(x_i) = 0$$

2. **Matrix Representation:**

   • Consideration all possible k, the system of equations can be expressed in matrix form:

$$A^T A \alpha = A^T Y$$

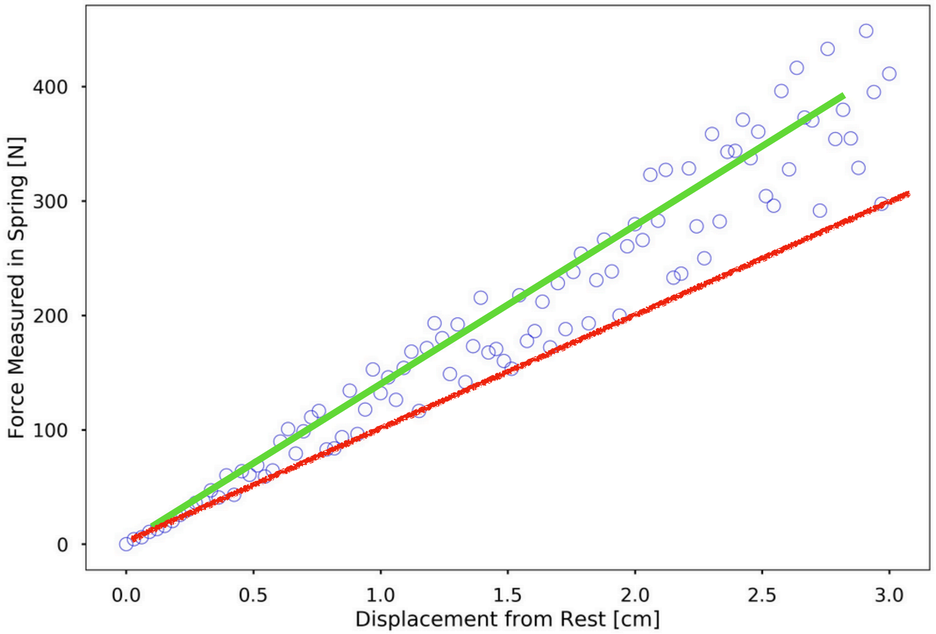
Where:

- A is the matrix of basis functions evaluated at the data points.
- Y is the column vector of output values.
- alpha is the column vector of parameters to be solved.

3. **Final Solution:**

   • Solving the matrix equation leads to the least squares solution:

$$\alpha = (A^T A)^{-1} A^T Y$$



Python Methods for Least Squares Regression

- **Key Python Libraries**:
  - Several libraries in Python can be used for least squares regression: Numpy and Scipy
    - **Numpy**: Provides functions such as numpy.linalg.lstsq() and numpy.linalg.pinv() for solving the least squares problem.
    - **Scipy**: Contains optimize.curve_fit() for more advanced fitting, including non-linear regression.
- **Example Workflow (Using Numpy)**:
  - **Generate data, Form the matrix A, Solve using least square**

```python
# generate x and y
x = np.linspace(0, 1, 101)
y = 1 + x + x * np.random.random(len(x))
```

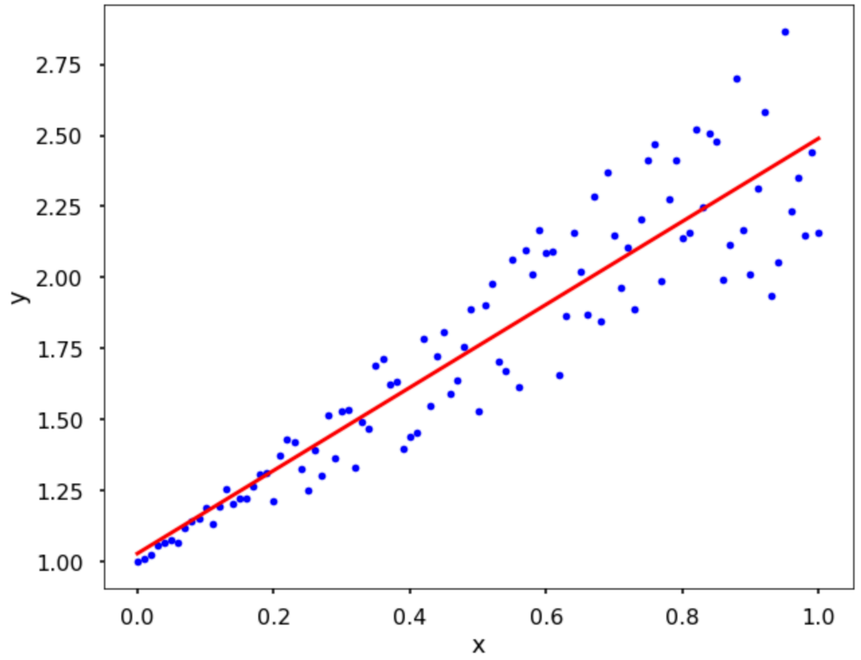
$$\hat{y} = \alpha_1 x + \alpha_2$$

```python
# assemble matrix A
A = np.vstack([x, np.ones(len(x))]).T

# turn y into a column vector
y = y[:, np.newaxis]
```

$$\beta = (A^T A)^{-1} A^T Y$$

```python
# Direct least square regression
alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T,A)),A.T)),y)
print(alpha)
```

```
[[1.459573  ]
 [1.02952189]]
```



  - **Plot the results**:

```python
plt.plot(x, y, 'b.', x, alpha[0]*x + alpha[1], 'r')
```

Use numpy.linalg.lstsq

```
alpha = np.linalg.lstsq(A, y, rcond=None)[0]
print(alpha)
```

```
[[1.459573  ]
 [1.02952189]]
```

Use optimize.curve_fit from scipy

using this function, we don't need to turn y into a column vector.

```
# generate x and y
x = np.linspace(0, 1, 101)
y = 1 + x + x * np.random.random(len(x))
```

```
def func(x, a, b):
    y = a*x + b
    return y

alpha = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(alpha)
```

```
[1.44331612 1.0396133 ]
```

Least Squares Regression for Nonlinear Functions

- **Definition**: Least squares regression estimates parameters by minimizing the sum of squared differences between observed and predicted data points.

- **Nonlinear Functions**: Some functions can't be modeled using traditional linear combinations of basis functions but can still benefit from least squares regression.

- **Methods for Nonlinear Functions**:
  1. **Logarithmic Transformations**: Convert nonlinear functions (e.g., exponential and power functions) to a linear form by applying logarithms.
  2. **Direct Nonlinear Fitting**: Using methods like curve_fit from scipy to fit a nonlinear model without transforming the function.

Logarithmic Tricks for Nonlinear Models

1. **Exponential Functions**:

- Given an exponential function, we can linearize it using logarithms:

$$y = \alpha e^{\beta x}$$

$$\log(y) = \log(\alpha) + \beta x$$

•Perform least squares on the linearized form, then recover parameters.

2. **Power Functions**:

- Similar approach for power functions

$$y = bx^m$$

$$\log(y) = m\log(x) + \log(b)$$

•After transforming, we apply least squares regression to solve for the parameters m and b.

Other Techniques for Nonlinear Regression

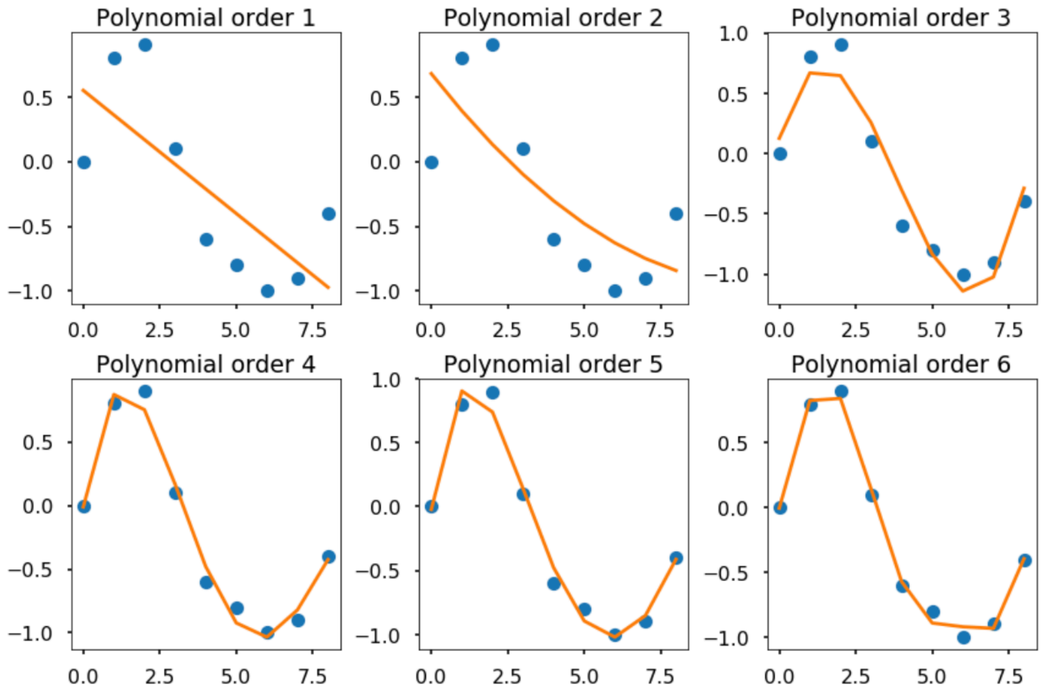
1. **Polynomial Regression**:

- Use least squares for polynomials of varying orders:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0$$

- Higher-order polynomials can be more flexible but risk overfitting.

```
x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1, -0.9, -0.4])

plt.figure(figsize = (12, 8))
for i in range(1, 7):

    # get the polynomial coefficients
    y_est = np.polyfit(x_d, y_d, i)
    plt.subplot(2,3,i)
    plt.plot(x_d, y_d, 'o')
    # evaluate the values for a polynomial
    plt.plot(x_d, np.polyval(y_est, x_d))
    plt.title(f'Polynomial order {i}')

plt.tight_layout()
plt.show()
```



Other Techniques for Nonlinear Regression

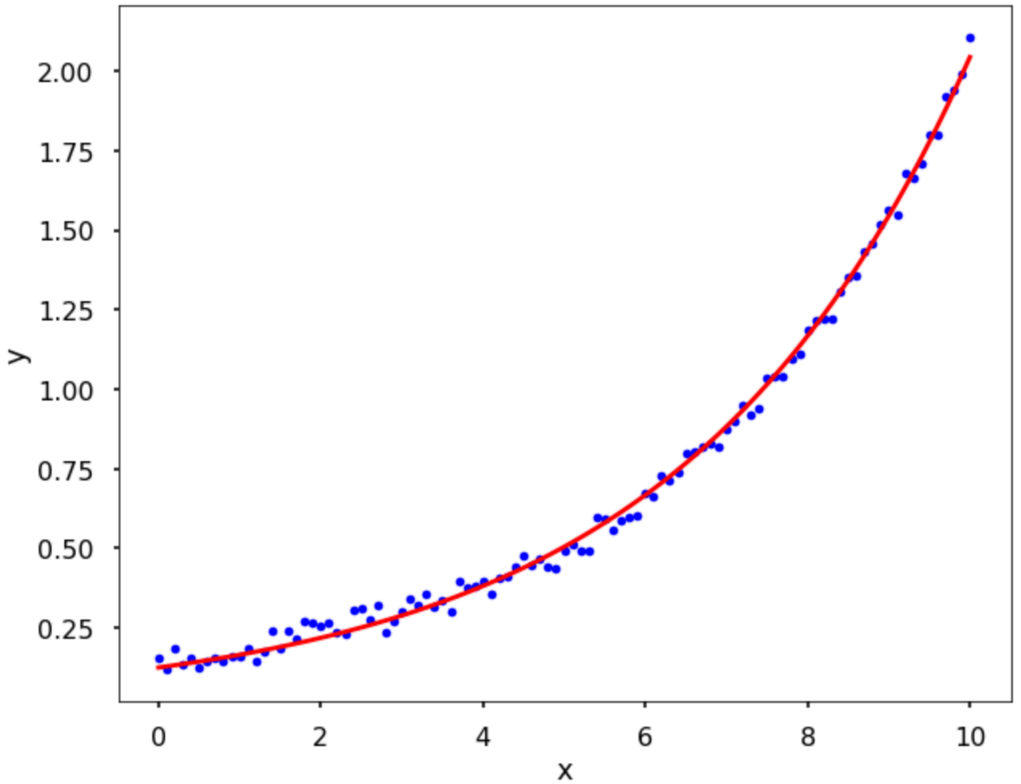
2. **Direct Nonlinear Fitting with** curve_fit:

- The curve_fit function from scipy directly estimates parameters for nonlinear functions without requiring transformations.
- Example: fitting an exponential function. $y = \alpha e^{\beta x}$

```
# let's define the function form
def func(x, a, b):
    y = a*np.exp(b*x)
    return y

alpha, beta = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(f'alpha={alpha}, beta={beta}')
```

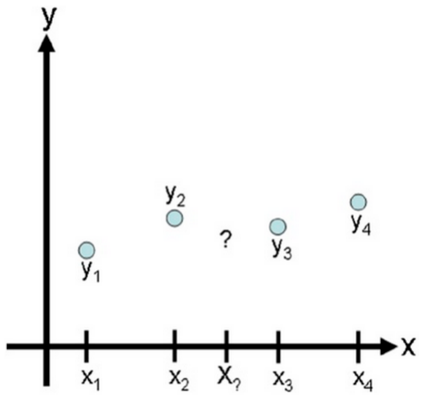
```
alpha=0.12420824748558632, beta=0.28005793982974525
```

```
# Let's have a look of the data
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha*np.exp(beta*x), 'r')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



Interpolation Problem Statement

Overview:



- **Interpolation Definition:** The process of estimating unknown values that fall within the range of known data points.
- **Problem Statement:**
  - Given a dataset of independent data values (x_i) and dependent data values (y_i), the goal is to find an estimation function f_hat(x) such that f_hat(x_i) = y_i for every point in the dataset.
  - This function should pass through all the data points, ensuring an accurate estimation for new x values not explicitly in the dataset.

Key Characteristics:

- Unlike regression, interpolation does not assume a model for the data.
- Suitable when data points are reliable and provide an accurate representation of the trend.
- Example: Estimating the motion of a car over time where data points are smooth, reflecting the continuous nature of the movement.

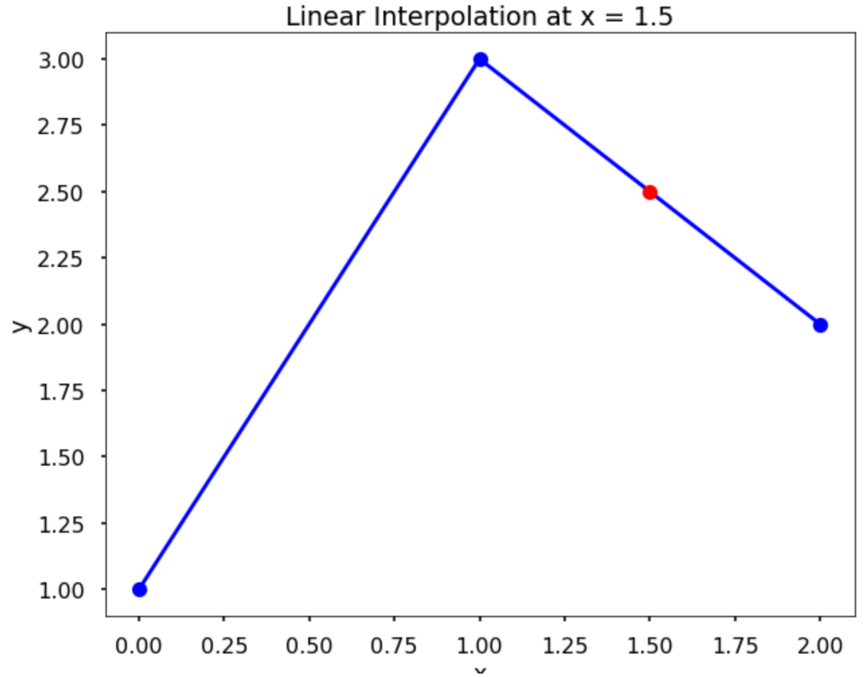
Linear Interpolation

- **Definition:** Assumes that the estimated point lies on the straight line joining the nearest data points on either side.
- **Mathematical Expression:**
  - For data points x_i < x < x_{i+1}, the linear interpolation formula is:

$$\hat{f}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{x_{i+1} - x_i}$$

  - Example Calculation:
  - Given x = [0, 1, 2] and y = [1, 3, 2], to find the interpolation at x = 1.5:

$$\hat{f}(1.5) = 3 + \frac{(2-3)(1.5-1)}{2-1} = 2.5$$



Practical Implementation:

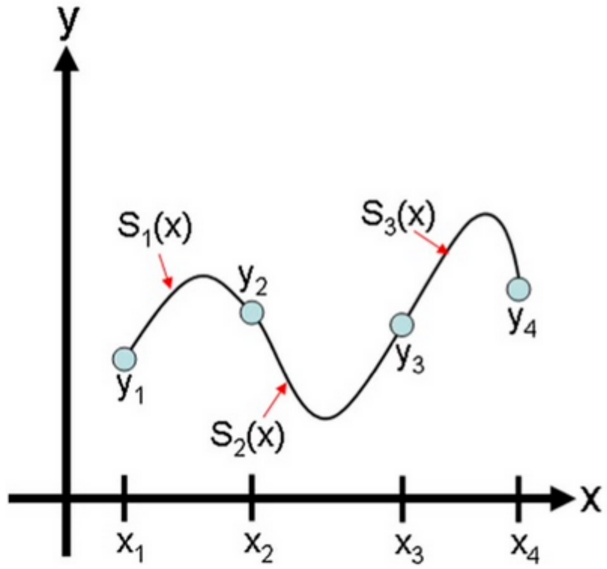
```
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

plt.style.use('seaborn-poster')
```

```
x = [0, 1, 2]
y = [1, 3, 2]

f = interp1d(x, y)
y_hat = f(1.5)
print(y_hat)
```

Cubic Spline Interpolation

- **Definition**: Cubic spline interpolation is a method of constructing a smooth curve through a set of points using piecewise cubic polynomials.

- **Objective**: The goal is to ensure that the curve not only passes through the given data points but also maintains a smooth transition between these points by matching first and second derivatives.



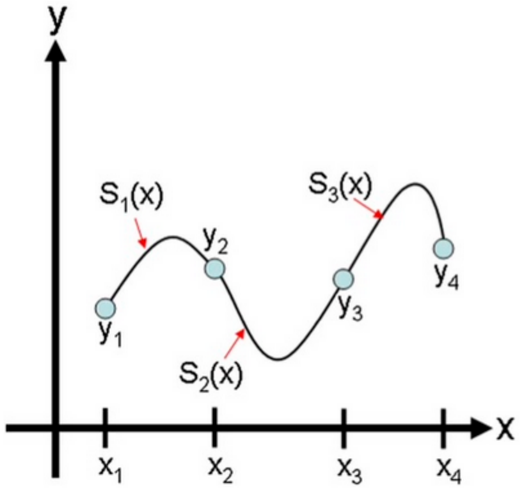
- **Mathematical Formulation**:
  - For each interval between points x_i and x_{i+1} , a cubic polynomial of the form:

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

is used.

  - There are n-1 cubic functions to find, requiring four coefficients per function, leading to a total of 4(n-1) unknowns.

Conditions and Constraints for Cubic Spline Interpolation



- **Equation Setup**:
  - **Interpolation condition**: The cubic polynomials must intersect the given data points.
  - **Continuity**:
    - The first derivative and second derivative of each spline must be continuous at the data points to ensure smooth transitions.

$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}), \quad i = 1, \dots, n-2,$$
$$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1}), \quad i = 1, \dots, n-2,$$

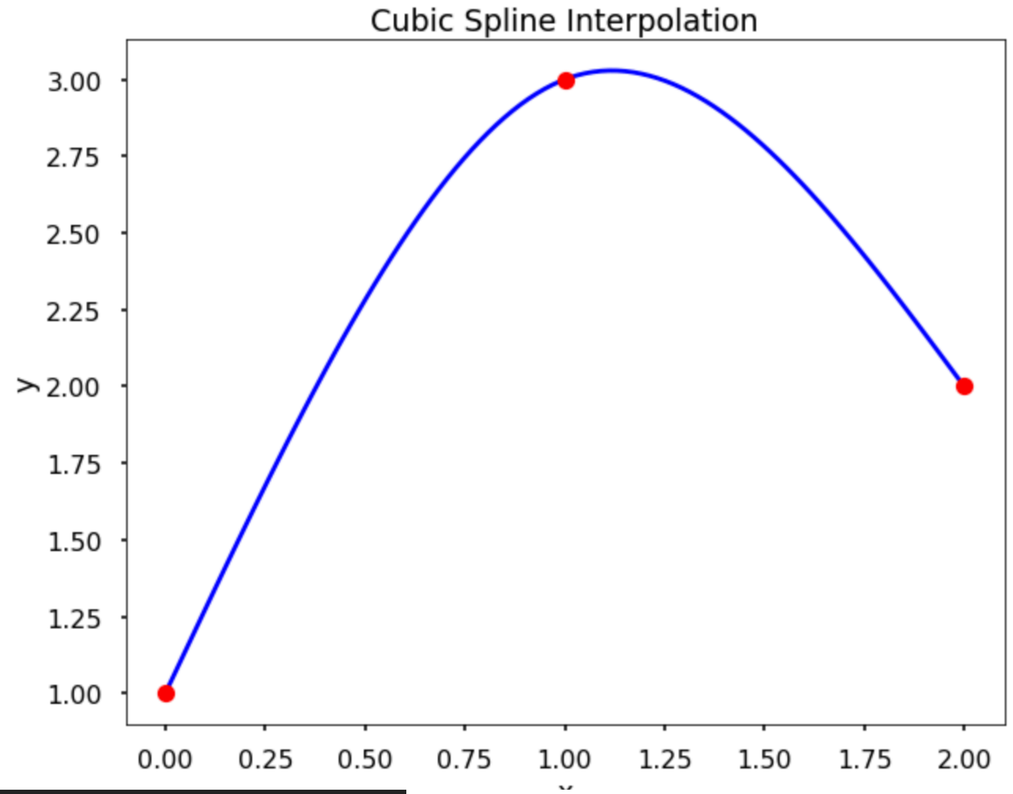
- **Boundary Conditions:**
  - Commonly used constraints include setting the second derivatives at the endpoints to zero, known as the “natural spline” condition.
  - Alternative boundary conditions can be implemented by specifying the type (using bc_type parameter in Scipy’s CubicSpline function).

$$S_1''(x_1) = 0$$
$$S_{n-1}''(x_n) = 0.$$

Implementation Using Python's Scipy Library

- **Python Implementation Example**:
  - Example data: x = [0, 1, 2] , y = [1, 3, 2] .
  - Code snippet:

```python
from scipy.interpolate import CubicSpline
import numpy as np
import matplotlib.pyplot as plt

x = [0, 1, 2]
y = [1, 3, 2]
f = CubicSpline(x, y, bc_type='natural')
x_new = np.linspace(0, 2, 100)
y_new = f(x_new)

plt.figure(figsize=(10,8))
plt.plot(x_new, y_new, 'b')
plt.plot(x, y, 'ro')
plt.title('Cubic Spline Interpolation')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



- **Explanation**: This code generates a smooth curve passing through the specified points, demonstrating the natural cubic spline interpolation.

Lagrange Polynomial Interpolation

- Lagrange Polynomial Interpolation involves finding a single polynomial that passes through all given data points.
- It is represented as L(x), where the polynomial is constructed to satisfy the condition L(x_i) = y_i for every point in the dataset.

Lagrange Basis Polynomials:

- The interpolation polynomial can be expressed as a linear combination of Lagrange basis polynomials,P_i(x), where:

$$P_i(x) = \prod_{j=1, j \neq i}^{n} \frac{(x - x_j)}{(x_i - x_j)}$$

- Each basis polynomial P_i(x) has the property:

- $P_i(x_j) = 1$ if $i = j$
- $P_i(x_j) = 0$ if $i \neq j$

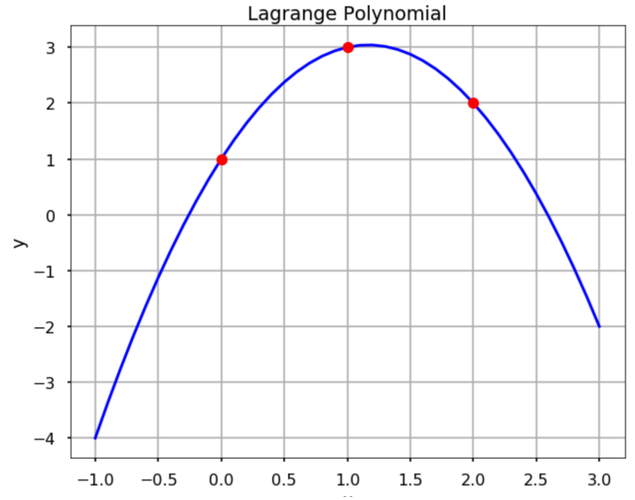
Key Properties:

- The Lagrange polynomial accurately passes through each of the specified data points.

$$L(x) = \sum_{i=1}^{n} y_i P_i(x)$$

- It should only be used for interpolation within the range of the given data points to avoid unrealistic growth outside this interval.

Example and Visualization of Lagrange Interpolation



Example Dataset:

- Consider the data points:x = [0, 1, 2] and y = [1, 3, 2].
- The Lagrange basis polynomials derived for this dataset are:

- $P_1(x) = -1.5x + 0.5x^2$
- $P_2(x) = 2x - x^2$
- $P_3(x) = -0.5x + 0.5x^2$

Visualization:

- The sum of these polynomials forms the complete Lagrange polynomial that goes through the points (0, 1), (1, 3), and (2, 2).
- When plotted, each basis polynomial P_i(x) shows distinct behavior, taking the value 1 at its corresponding data point and 0 at the others.

Using SciPy for Lagrange Interpolation:

- SciPy's lagrange function provides a straightforward way to calculate the Lagrange polynomial directly without manual computation.

```
from scipy.interpolate import lagrange
```

```
f = lagrange(x, y)
```

Newton’s Polynomial Interpolation

- Newton’s polynomial interpolation is a method to fit a polynomial through a set of data points, widely used in numerical analysis.

•Formula: The Newton polynomial of order n can be represented as:

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)...(x - x_{n-1})$$

•Key Feature: The coefficients (a_i) are determined using divided differences, allowing easy updates to the polynomial when new data points are added.

Divided Differences and Coefficient Calculation

- **Divided Differences:** A method to calculate the coefficients of the polynomial. It helps in determining each coefficient using previous calculations, which makes it efficient.

- **Iterative Process:** The divided differences can be represented as:

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

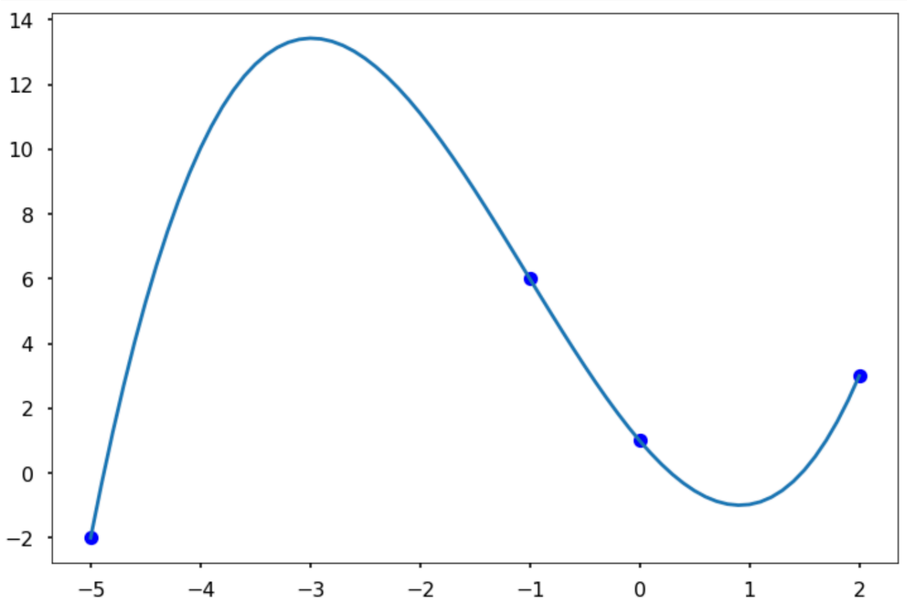
- **Divided Differences Table:** A table used to systematically calculate the coefficients needed for the polynomial.

Implementation Example in Python

- **Code Snippet:** Python implementation of Newton's interpolation using divided differences.

```
def divided_diff(x, y):
    '''
    function to calculate the divided
    differences table
    '''
    n = len(y)
    coef = np.zeros([n, n])
    # the first column is y
    coef[:,0] = y

    for j in range(1,n):
        for i in range(n-j):
            coef[i][j] = \
           (coef[i+1][j-1] - coef[i][j-1]) / (x[i+j]-x[i])

    return coef

def newton_poly(coef, x_data, x):
    '''
    evaluate the newton polynomial
    at x
    '''
    n = len(x_data) - 1
    p = coef[n]
    for k in range(1,n+1):
        p = coef[n-k] + (x -x_data[n-k])*p
    return p
```

```
x = np.array([-5, -1, 0, 2])
y = np.array([-2, 6, 1, 3])
# get the divided difference coef
a_s = divided_diff(x, y)[0, :]

# evaluate on new data points
x_new = np.arange(-5, 2.1, .1)
y_new = newton_poly(a_s, x, x_new)

plt.figure(figsize = (12, 8))
plt.plot(x, y, 'bo')
plt.plot(x_new, y_new)
```



Root Finding Problem Statement

Definition:

- A *root* or *zero* of a function f(x) is a value x_r such that f(x_r) = 0 .
- Some functions, like quadratic equations, have easily identifiable roots (e.g., f(x) = x^2 - 9 has roots at x = +/- 3).

Numerical Approximation:

- For more complex functions where exact solutions are difficult to find, numerical methods like fsolve from the SciPy library can be used.
- Example:

```python
from scipy import optimize
f = lambda x: np.cos(x) - x
r = optimize.fsolve(f, -2)
```

This approach finds approximate roots, with limitations indicated by factors such as reaching a maximum number of function evaluations.

Limitations:

- Numerical methods may fail if they reach the maximum iterations without converging to a solution.
- Understanding these methods' constraints is crucial for their proper application in engineering and scientific contexts.

Success

```python
import numpy as np
from scipy import optimize

f = lambda x: np.cos(x) - x
r = optimize.fsolve(f, -2)
print("r =", r)

# Verify the solution is a root
result = f(r)
print("result=", result)
```

```
r = [0.73908513]
result= [0.]
```

Failure

```python
f = lambda x: 1/x

r, infodict, ier, mesg = optimize.fsolve(f, -2, full_output=True)
print("r =", r)

result = f(r)
print("result=", result)

print(mesg)
```

```
r = [-3.52047359e+83]
result= [-2.84052692e-84]
The number of calls to function has reached maxfev = 400.
```

Concept of Tolerance in Numerical Methods

Definition of Tolerance:

- *Tolerance* refers to the acceptable level of error in a numerical solution.
- A solution is considered to have *converged* if its error is smaller than the defined tolerance.

Error Measurement:

- Two common methods to measure error when finding roots:

1. |f(x)| : Indicates how close the function value is to zero.
2. |x_{i+1} - x_i| : Measures the difference between successive guesses, highlighting convergence speed.

Importance in Applications:

- Different applications require different levels of precision; therefore, tolerance levels must be set according to specific needs.
- Careful selection of tolerance is vital to avoid false convergence or premature stopping of the numerical method.

Practical Example of Tolerance and Root Finding

Example Analysis:

- Consider the function $f(x) = 1/x$
- Setting the error metric as $|f(x)|$ and a small tolerance value might suggest convergence even when true roots do not exist.
- It demonstrates the importance of correctly interpreting tolerance levels and error metrics.

Use of Tolerance:

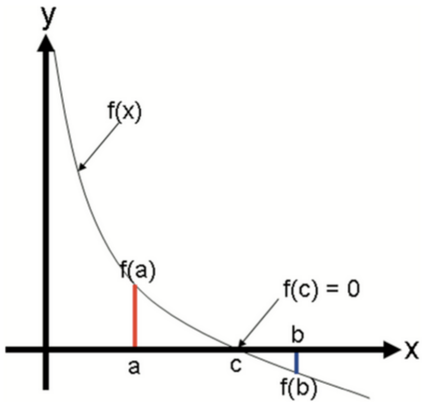
- An appropriate tolerance setting ensures that the numerical root-finding method performs optimally and provides a meaningful solution within acceptable error bounds.
- Always validate results by checking both the error magnitude and algorithm behavior.

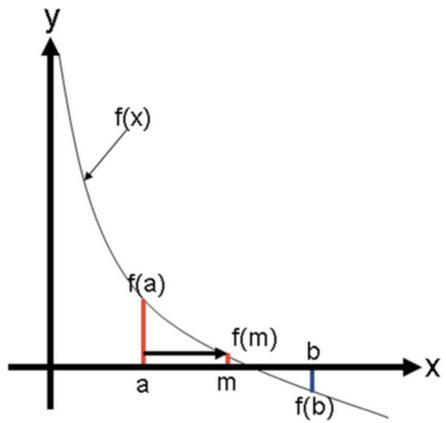
Key Takeaway:

- In numerical analysis, tolerance is not just a technical parameter but a crucial decision factor that influences the reliability and accuracy of computed results.

Bisection Method

- **Definition:**
  - The Bisection Method is a root-finding technique that applies the Intermediate Value Theorem.
  - It is used to find a root of a continuous function f(x) on an interval [a, b] where the function changes sign.
  - Key Concept:





  - If f(a) and f(b) have opposite signs ($f(a) \cdot f(b) < 0$), there exists at least one root c such that $f(c) = 0$ in the interval [a, b].
  - Process:
  - Calculate the midpoint $m = (a + b) / 2$
  - Evaluate f(m) . Depending on the sign of f(m) , update either the left bound a or the right bound b .
  - Repeat the process iteratively until the error or tolerance is sufficiently small.

Algorithm and Implementation

- **Algorithm Outline:**

1. Check Bounds: Ensure that f(a) and f(b) have opposite signs.
2. Calculate Midpoint: m = (a + b) / 2
3. Evaluate Midpoint:
   - If |f(m)| < tolerance, then m is the root.
   - If f(a) and f(m) have opposite signs, update the interval to [a, m].
   - If f(m) and f(b) have opposite signs, update the interval to [m, b].
4. Repeat: Continue until the desired accuracy is reached.

```python
import numpy as np

def my_bisection(f, a, b, tol):
    # approximates a root, R, of f bounded
    # by a and b to within tolerance
    # | f(m) | < tol with m the midpoint
    # between a and b Recursive implementation

    # check if a and b bound a root
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
         "The scalars a and b do not bound a root")

    # get midpoint
    m = (a + b)/2

    if np.abs(f(m)) < tol:
        # stopping condition, report m as root
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        # case where m is an improvement on a.
        # Make recursive call with a = m
        return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
        # case where m is an improvement on b.
        # Make recursive call with b = m
        return my_bisection(f, a, m, tol)
```

Example and Limitations

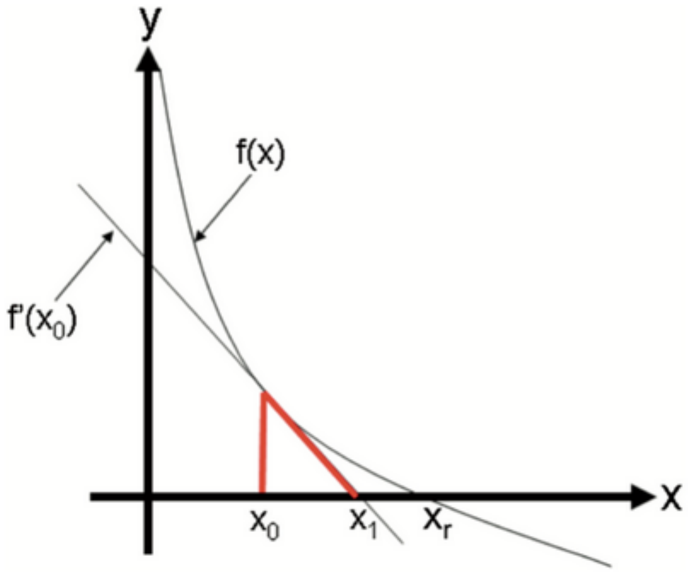
- **Example Calculation:** Function: f(x) = x^2 - 2
- **Initial Interval:** [0, 2]

- **Tolerances:**
- For tolerance 0.1, the method finds r1 = 1.4375 .
- For tolerance 0.01, it finds r01 = 1.4140625 , a closer approximation to the actual root (sqrt(2) = 1.414…….).

- **Limitations:**
- Requirement: The method requires the initial interval [a, b] to have a sign change.
- Convergence Speed: It converges linearly, which can be slower compared to other methods like Newton-Raphson.
- Exception Handling: If f(a) and f(b) do not bound a root, the method raises an error, indicating the chosen interval is incorrect.

Newton-Raphson Method

- The Newton-Raphson method is an iterative technique used to find the roots of a real-valued function.
- Assumes a smooth and continuous function f(x) with an unknown root x_r .
- Begins with an initial guess x_0 and improves upon this guess iteratively.



Linear Approximation

- The method relies on the linear approximation of the function around the initial guess x_0 .
- Uses the formula:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

- This equation represents the Newton step, used to compute an improved guess x_1 .

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

$$0 = f(x_0) + f'(x_0)(x_1 - x_0)$$

Process

1. Start with an initial guess x_0 .
2. Use the formula to find a better approximation of the root.
3. Repeat the process until the error is less than a specified tolerance level.

Implementation and Limitations of the Newton-Raphson Method

Key Limitations

- Initial Guess Sensitivity: The success of the method heavily depends on the choice of the initial guess x_0 .

- Derivative Issues: If the derivative $f'(x)$ is close to zero, the method can produce large and unstable steps.

- Root Convergence: May converge to a different root than expected or fail to converge if the initial guess is far from the actual root.

Comparison with Other Methods

- The Newton-Raphson method typically converges faster than the Bisection method when the initial guess is close to the root.

- However, due to its sensitivity, it is less reliable for functions with complex behavior.

```python
import numpy as np

f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
```

```python
def my_newton(f, df, x0, tol):
    # output is an estimation of the root of f
    # using the Newton Raphson method
    # recursive implementation
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df, x0 - f(x0)/df(x0), tol)
```

```python
estimate = my_newton(f, f_prime, 1.5, 1e-6)
print("estimate =", estimate)
print("sqrt(2) =", np.sqrt(2))
```

```
estimate = 1.4142135623746899
sqrt(2) = 1.4142135623730951
```

Using fsolve for Root Finding

- **Example Equation:**

The function used in the example:

$$f(x) = x^3 - 100x^2 - x + 100$$

Python Implementation:

```python
from scipy.optimize import fsolve

f = lambda x: x**3 - 100*x**2 - x + 100
roots = fsolve(f, [2, 80])
print(roots)
```

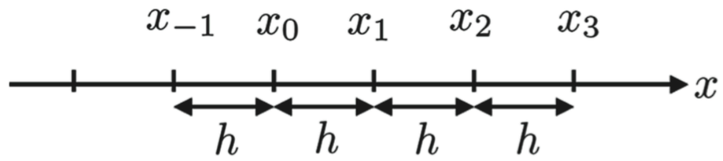
- **Output:** [1., 100.]

  - The initial guess of [2, 80] helps in converging to the two roots of the function: x = 1 and x = 100.

Note: fsolve admits many parameters, read about them and make sure you understand what is being done

Numerical Differentiation: Problem Overview

Concept: Numerical differentiation focuses on estimating the derivative of a function using a discrete set of points, often referred to as a *numerical grid*.



- **Numerical Grid**: A set of evenly spaced points over the domain of a function (independent variable).
  - The spacing between adjacent points on the grid is denoted by h, and each point is labeled as x_j, where j is the index.

- **Importance**:
  - Useful in cases where the function is only known at discrete points, such as data from sensors (e.g., temperature measurements).
  - Even though the function may be continuous in reality, the discrete points help approximate its derivative.

- **Application**: Numerical differentiation is critical in engineering and science for analyzing real-world data, where continuous functions are often approximated by discrete sets of observations.

Finite Difference Approximations

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

- **Finite Difference Method**: A numerical technique to approximate derivatives of functions.
- **Objective**: Estimate the slope of a function f(x) at a point by using values from neighboring points.

- **Key Formulations**

1. **Forward Difference**: Approximates the slope at x_j using f(x_j) and f(x_{j+1}) .

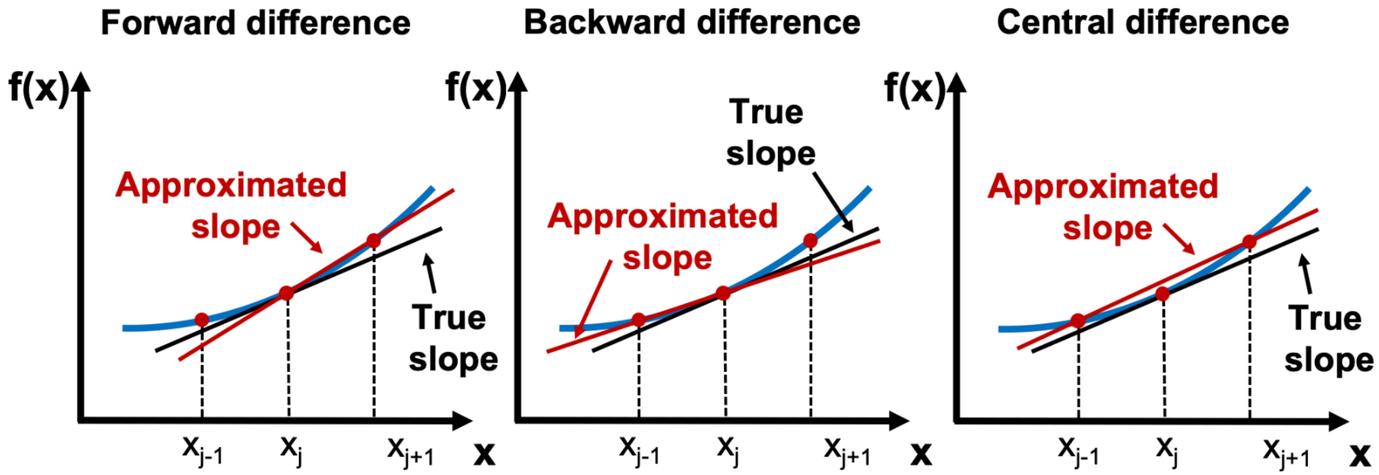
$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_j)}{x_{j+1} - x_j}$$

2. **Backward Difference**: Uses f(x_j) and f(x_{j-1}) .

$$f'(x_j) \approx \frac{f(x_j) - f(x_{j-1})}{x_j - x_{j-1}}$$

3. **Central Difference**: Uses both f(x_{j+1}) and f(x_{j-1}) .

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_{j-1})}{2h}$$



Accuracy of Finite Difference Methods

- **Order of Accuracy**:
  - **Forward and Backward Difference**: First-order accuracy, O(h) , meaning the error decreases linearly as h -> 0 .
  - **Central Difference**: Second-order accuracy, O(h^2) , meaning the error decreases quadratically as h -> 0 .

$$f(x_{j+1}) = \frac{f(x_j)(x_{j+1}-x_j)^0}{0!} + \frac{f'(x_j)(x_{j+1}-x_j)^1}{1!} + \frac{f''(x_j)(x_{j+1}-x_j)^2}{2!} + \frac{f'''(x_j)(x_{j+1}-x_j)^3}{3!} + \cdots$$

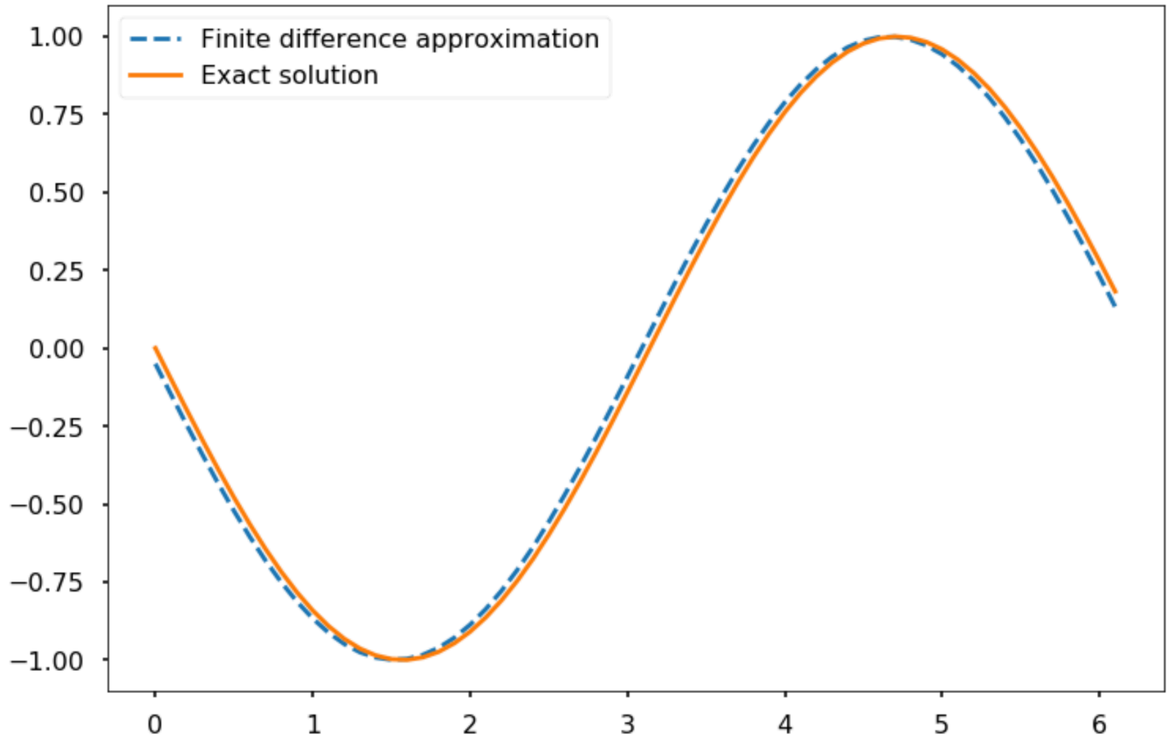
$$f'(x_j) = \frac{f(x_{j+1})-f(x_j)}{h} + \left(-\frac{f''(x_j)h}{2!} - \frac{f'''(x_j)h^2}{3!} - \cdots\right). \qquad f'(x_j) = \frac{f(x_{j+1})-f(x_j)}{h} + O(h).$$

- **Higher Order Terms**: Derived from Taylor series expansions, used to improve accuracy of approximations.
  - Taylor series expansion shows how truncation errors arise, and how higher-order terms improve precision.

Using Python for Finite Difference Approximation

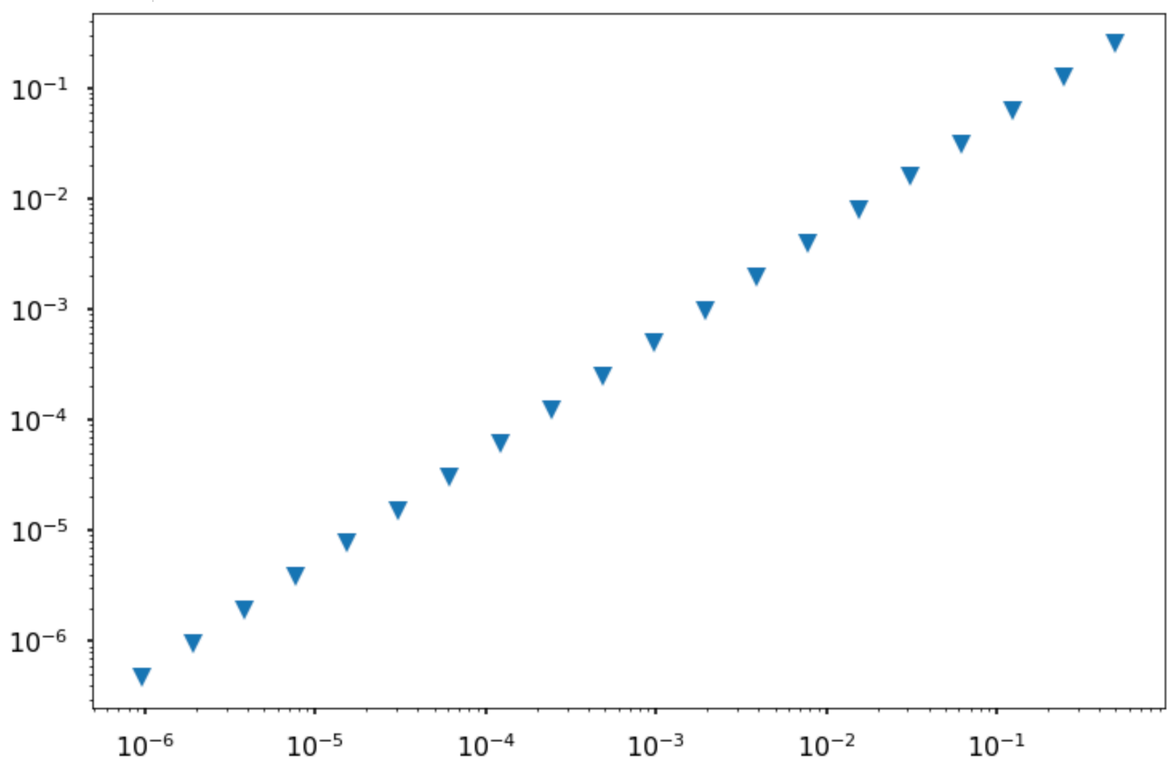
- **Python Code**: We can compute numerical derivatives using the numpy library.
  - Example function: f(x) = cos(x) , with known derivative f'(x) = -sin(x) .
  - The forward difference method is applied, and the resulting derivative is compared with the exact solution.
- **Python Code Snippet**:

```python
import numpy as np
import matplotlib.pyplot as plt

h = 0.1
x = np.arange(0, 2*np.pi, h)
y = np.cos(x)
forward_diff = np.diff(y) / h
x_diff = x[:-1]
exact_solution = -np.sin(x_diff)

plt.plot(x_diff, forward_diff, '--', label='Approximation')
plt.plot(x_diff, exact_solution, label='Exact')
plt.legend()
plt.show()
```



Error Analysis: Maximum error between numerical and exact solution is approximately 0.05 on previous figure, and decreases linearly if the step size h is reduced.

```
# define step size
h = 1
# define number of iterations to perform
iterations = 20
# list to store our step sizes
step_size = []
# list to store max error for each step size
max_error = []

for i in range(iterations):
    # halve the step size
    h /= 2
    # store this step size
    step_size.append(h)
    # compute new grid
    x = np.arange(0, 2 * np.pi, h)
    # compute function value at grid
    y = np.cos(x)
    # compute vector of forward differences
    forward_diff = np.diff(y)/h
    # compute corresponding grid
    x_diff = x[:-1]
    # compute exact solution
    exact_solution = -np.sin(x_diff)

    # Compute max error between
    # numerical derivative and exact solution
    max_error.append(\
            max(abs(exact_solution - forward_diff)))

# produce log-log plot of max error versus step size
plt.figure(figsize = (12, 8))
plt.loglog(step_size, max_error, 'v')
plt.show()
```



Approximating Higher Order Derivatives Using Taylor Series

Concept Overview:

Higher-order derivatives can be approximated using finite difference methods derived from Taylor series expansions. This technique is essential in numerical analysis when analytical solutions are difficult to obtain.

Taylor Series Approach:

1. **Taylor Series Expansion:**

   • Expand the function f(x) at x_j-1 and x_j+1 :

$$f(x_{j-1}) = f(x_j) - hf'(x_j) + \frac{h^2 f''(x_j)}{2} - \frac{h^3 f'''(x_j)}{6} + \cdots$$

$$f(x_{j+1}) = f(x_j) + hf'(x_j) + \frac{h^2 f''(x_j)}{2} + \frac{h^3 f'''(x_j)}{6} + \cdots .$$

2. **Combining Equations:**

   • By adding these expansions:

$$f(x_{j-1}) + f(x_{j+1}) = 2f(x_j) + h^2 f''(x_j) + \frac{h^4 f''''(x_j)}{24} + \cdots$$

3. **Second-Order Approximation:**

   • Solving for f''(x_j) gives a second-order approximation:

$$f''(x_j) \approx \frac{f(x_{j+1}) - 2f(x_j) + f(x_{j-1})}{h^2}$$

The error term is O(h^2) , implying the accuracy improves as h decreases.

Applications:

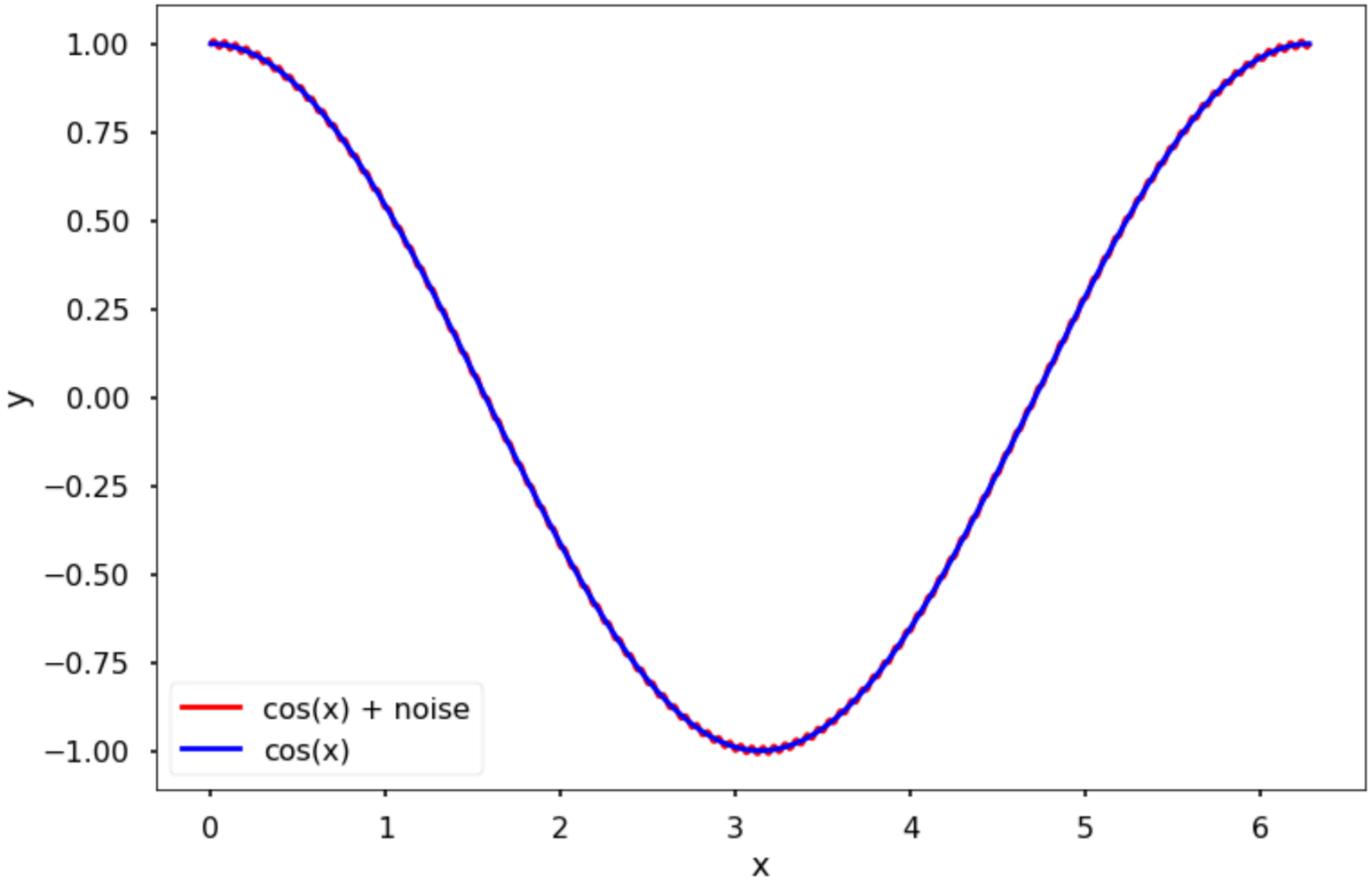
• Used in physics and engineering for problems involving wave equations, fluid dynamics, and structural analysis where higher-order derivatives are needed for simulations.

Numerical Differentiation with Noise

- **Definition:**
  - Numerical differentiation often involves approximating derivatives of a function given by discrete data points.
  - In real-world applications, these data points can be contaminated by noise due to measurement inaccuracies or external perturbations.
- **Impact of Noise:**
  - Noise can distort function values, leading to errors in the computed derivatives.
  - Example: Imagine trying to listen to a conversation in a crowded room. The speech signal represents the function, while background noise adds distortion to the signal.
- **Illustration:**
  - Consider the functions $f(x) = \cos(x)$ and $f_{\epsilon,\omega}(x) = \cos(x) + \epsilon \sin(\omega x)$, where epsilon is small (e.g., 0.01) and omega is large (e.g., 100).
  - The noise term $\epsilon \sin(\omega x)$ represents high-frequency noise superimposed on the original signal cos(x), making the derivative sensitive to noise, particularly when omega is large.

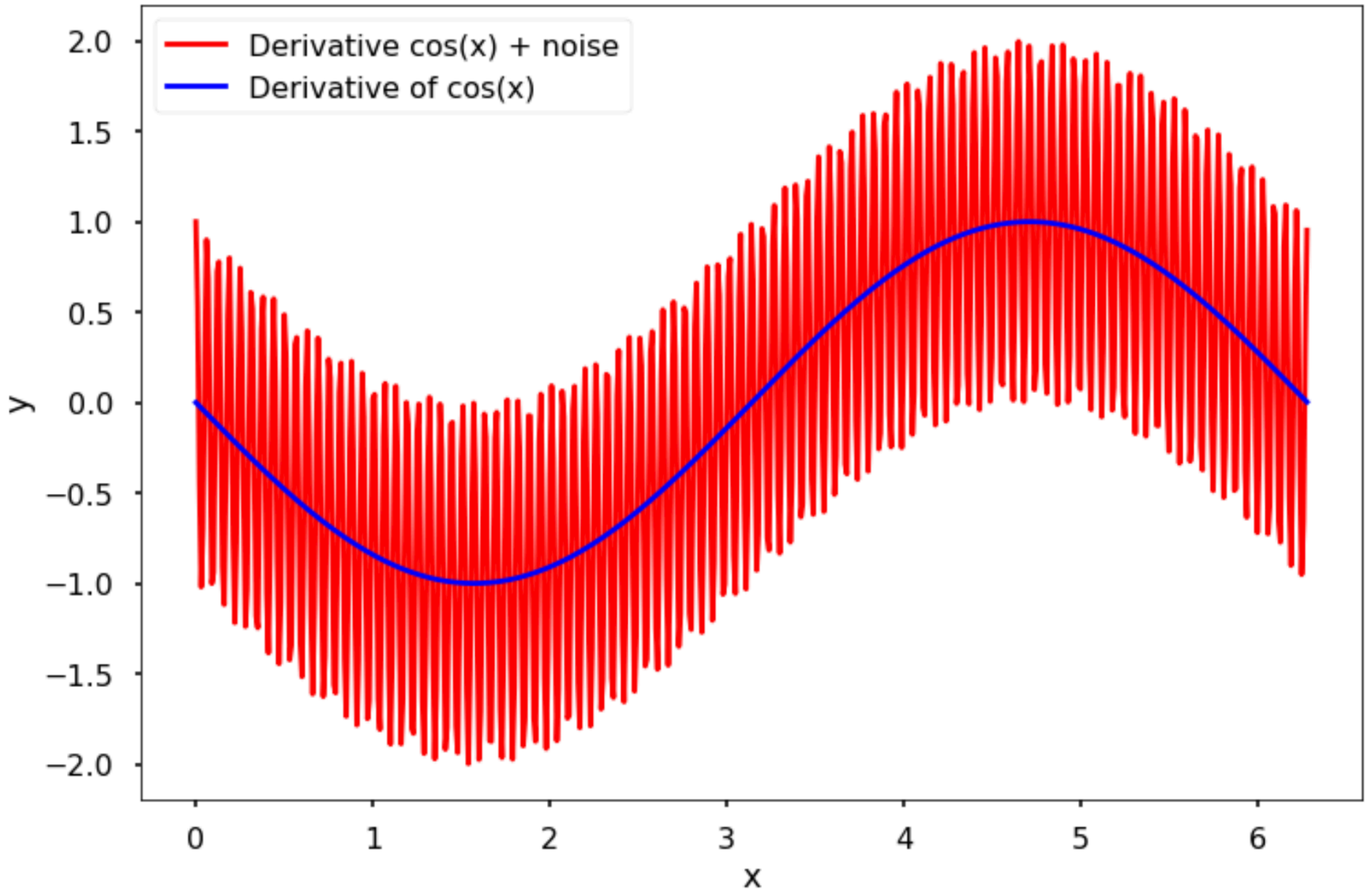
```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-poster')
%matplotlib inline
```

```
x = np.arange(0, 2*np.pi, 0.01)
# compute function
omega = 100
epsilon = 0.01

y = np.cos(x)
y_noise = y + epsilon*np.sin(omega*x)

# Plot solution
plt.figure(figsize = (12, 8))
plt.plot(x, y_noise, 'r-', \
         label = 'cos(x) + noise')
plt.plot(x, y, 'b-', \
         label = 'cos(x)')

plt.xlabel('x')
plt.ylabel('y')

plt.legend()
plt.show()
```



$$f'(x) = -\sin(x) \qquad f'_{\epsilon,\omega}(x) = -\sin(x) + \epsilon\omega\cos(\omega x).$$

```
x = np.arange(0, 2*np.pi, 0.01)
# compute function
y = -np.sin(x)
y_noise = y + epsilon*omega*np.cos(omega*x)

# Plot solution
plt.figure(figsize = (12, 8))
plt.plot(x, y_noise, 'r-', \
         label = 'Derivative cos(x) + noise')
plt.plot(x, y, 'b-', \
         label = 'Derivative of cos(x)')

plt.xlabel('x')
plt.ylabel('y')

plt.legend()
plt.show()
```



- The noise amplifies the high-frequency component, $\epsilon\omega\cos(\omega x)$, leading to large deviations from the true derivative as omega increases.

Handling Noise in Numerical Differentiation

- **Challenges:**
  - Noise introduces high-frequency oscillations in the derivative, making it difficult to distinguish between the true signal and noise.
  - High-frequency noise has a disproportionate impact on derivative calculations because differentiation amplifies the noise.

- **Conclusion:**
  - Numerical differentiation is highly sensitive to noise, especially for higher-order derivatives. Mitigating this sensitivity is crucial in engineering applications where precision is key.

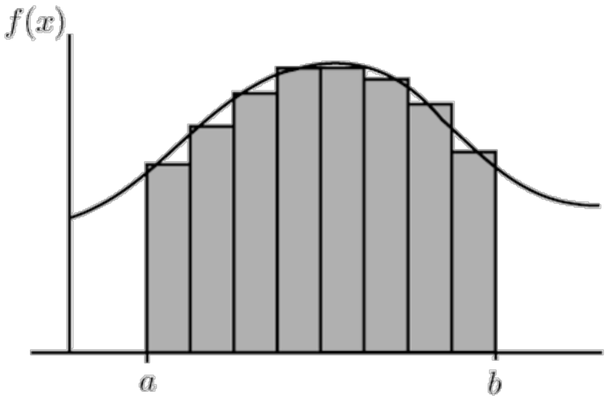
Riemann's Integral in Numerical Methods

- **Definition**: Riemann's Integral is a method of approximating integrals by summing areas of rectangles.

- **Approach**:
  - **Rectangle Width (h)**: The width of each rectangle is a uniform distance across subintervals.
  - **Rectangle Height**: Calculated based on the function's value at specific points (e.g., left or right endpoints of the interval).

  - **Formula**:

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} hf(x_i), \qquad \text{or} \qquad \int_a^b f(x)dx \approx \sum_{i=1}^{n} hf(x_i),$$



Choice of x_i : Left (x_i) or Right (x_{i+1}) endpoint.

- **Goal**: Achieve higher accuracy by reducing h (i.e., making subintervals finer).

Mathematical Foundation & Error Analysis

- **Taylor Series Expansion**:
  - Used to approximate the function f(x) over subintervals.
  - Taylor expansion:

$$f(x) = f(x_i) + f'(x_i)(x - x_i) + \cdots$$

$$\int_{x_i}^{x_{i+1}} f(x)dx = \int_{x_i}^{x_{i+1}} (f(x_i) + f'(x_i)(x - x_i) + \cdots)\, dx$$

- Integral over subinterval approximated as:

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + \frac{h^2}{2} f'(x_i) + O(h^3),$$

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + O(h^2).$$

- **Error Calculation**:
  - **Single Interval**: Approximation error is O(h^2) .
  - **Whole Interval**: Summed error across intervals yields O(h) , making finer subintervals crucial for accuracy.

Midpoint Rule in Numerical Integration

- **Definition**: The Midpoint Rule is an improved technique for approximating integrals by taking the function value at the midpoint of each subinterval.
- **Approach**:
  - **Midpoint Calculation**:
    - For each interval $[x_i, x_{i+1}]$, the midpoint is $y_i = (x_i + x_{I+1}/2$.
    - The function value $f(y_i)$ at the midpoint provides the rectangle's height.
  - **Formula**:

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} hf(y_i).$$

where h is the width of each subinterval.

- **Advantages**:
  - **Higher Accuracy**: The Midpoint Rule typically offers an error of $O(h^2)$, an improvement over the standard Riemann sum methods which are $O(h)$.
  - **Symmetry Benefit**: The symmetric position of the midpoint helps cancel out lower-order error terms in the Taylor expansion, leading to a more accurate result.
- **Limitations**:
  - **Data Dependency**: Requires function values at midpoints; if data points are fixed (not adjustable), midpoint values might not be computable.

Let's use the left Riemann Integral, right Riemann Integral, and Midpoint Rule to approximate $\int \sin(x)dx$ with 11 evenly spaced grid points over the whole interval. Compare to the exact value of 2.

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_riemannL = h * sum(f[:n-1])
err_riemannL = 2 - I_riemannL

I_riemannR = h * sum(f[1::])
err_riemannR = 2 - I_riemannR

I_mid = h * sum(np.sin((x[:n-1] \
        + x[1:])/2))
err_mid = 2 - I_mid

print(I_riemannL)
print(err_riemannL)

print(I_riemannR)
print(err_riemannR)

print(I_mid)
print(err_mid)
```

```
1.9835235375094546
0.01647646249054535
1.9835235375094546
0.01647646249054535
2.0082484079079745
-0.008248407907974542
```

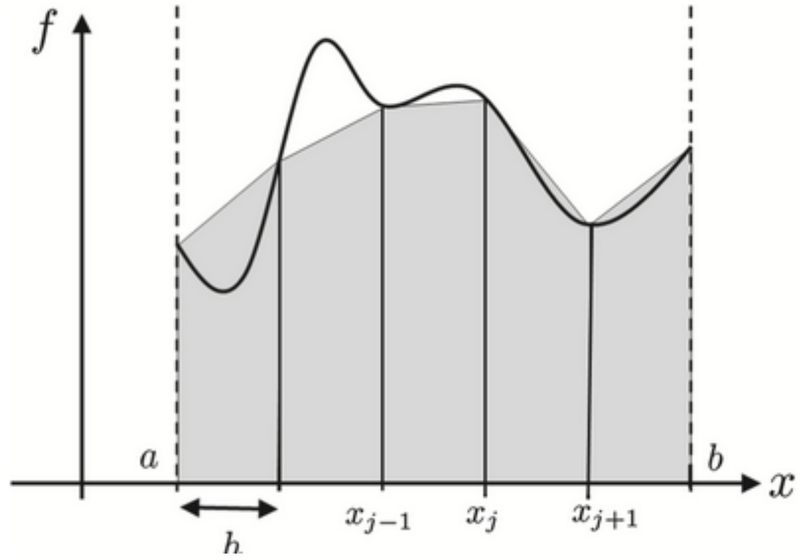
Trapezoid Rule for Numerical Integration

Concept Overview:

- The Trapezoid Rule approximates the integral of a function by dividing the area under the curve into trapezoidal segments.
- For each interval [x_i, x_{i+1}], the trapezoid is formed between the points (x_i, f(x_i)) and (x_{i+1}, f(x_{i+1})), making the area computation straightforward.
- Formula: The area of each trapezoid is calculated as:

$$\text{Area} = \frac{h}{2}(f(x_i) + f(x_{i+1}))$$

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} h\frac{f(x_i) + f(x_{i+1})}{2}.$$



where h is the width of each interval.

- Common in engineering and scientific applications where discrete data points approximate continuous functions.

Computational Efficiency and Practical Implementation

Efficient Computation:

- The Trapezoid Rule double-counts terms across intervals except at the boundaries, which can be computationally intensive. This can be optimized using the expression:

$$\int_a^b f(x)dx \approx \frac{h}{2}\left(f(x_0) + 2\left(\sum_{i=1}^{n-1} f(x_i)\right) + f(x_n)\right).$$

Trapezoid Rule to approximate $\int \sin(x)dx$ *with* 11 evenly spaced grid points over the whole interval. Compare to the exact value of 2.

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_trap = (h/2)*(f[0] + \
          2 * sum(f[1:n-1]) + f[n-1])
err_trap = 2 - I_trap

print(I_trap)
print(err_trap)
```

```
1.9835235375094546
0.01647646249054535
```

Error Analysis and Accuracy Considerations

Taylor Series Expansion for Error Estimation:

- The accuracy of the Trapezoid Rule can be evaluated by expanding f(x) as a Taylor series around the midpoint. This reveals that the error per interval is proportional to O(h^2).
- When applied over multiple intervals, the cumulative error becomes O(h) for the entire integration range.

Practical Implication:

- The method is more accurate for functions that are nearly linear over each subinterval.
- For functions with higher curvature, the accuracy decreases, and methods like Simpson’s Rule may provide better precision.

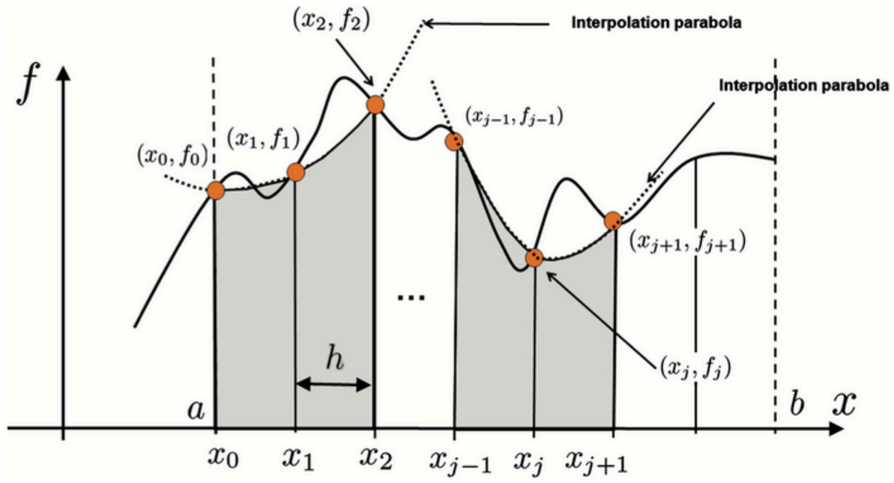
Simpson's Rule

- **Definition**: Simpson's Rule is a numerical method for approximating the integral of a function. It works by fitting a quadratic polynomial to segments of the function's curve and then calculating the area under this polynomial.
- **Concept**:
  - Uses **two consecutive subintervals** to approximate the integral over a range.
  - Applies a **parabolic approximation** through three key points (x_{i-1}, f(x_{i-1})), (x_i, f(x_i)), (x_{i+1}, f(x_{I+1})), using Lagrange polynomials.
  - Integrates this parabolic function exactly over each interval.
- **Formula**:

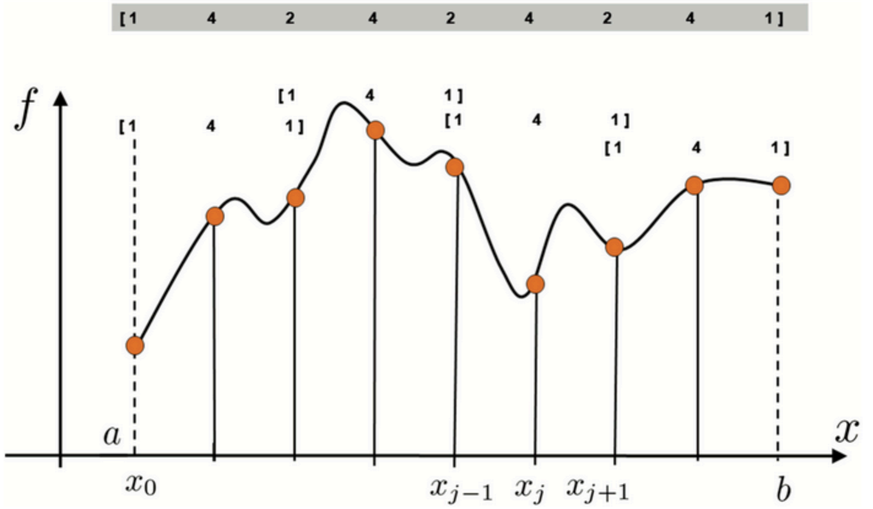
$$\int_a^b f(x)\,dx \approx \frac{h}{3}[f(x_0) + 4f(x_1) + 2f(x_2) + \ldots + 4f(x_{n-1}) + f(x_n)]$$

$$\int_a^b f(x)dx \approx \frac{h}{3}\left[f(x_0) + 4\left(\sum_{i=1,i\text{ odd}}^{n-1} f(x_i)\right) + 2\left(\sum_{i=2,i\text{ even}}^{n-2} f(x_i)\right) + f(x_n)\right]$$

where h is the width of each subinterval, requiring an **odd number of points**.



This regrouping is illustrated in the figure below:



SimpsonRule to approximate $\int \sin(x)dx$ with 11 evenly spaced grid points over the whole interval. Compare to the exact value of 2.

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_simp = (h/3) * (f[0] + 2*sum(f[:n-2:2]) \
            + 4*sum(f[1:n-1:2]) + f[n-1])
err_simp = 2 - I_simp

print(I_simp)
print(err_simp)
```

```
2.0001095173150043
-0.00010951731500430384
```

Advantages and Accuracy of Simpson’s Rule

- **Accuracy**:
  - Simpson’s Rule achieves **fourth-order accuracy**, making it more precise than the Trapezoid Rule.
  - It cancels certain error terms using the **Taylor Series expansion**, further enhancing accuracy over two intervals.

- **Considerations**:
  - Requires an **odd number of points** (even number of intervals).
  - Errors diminish rapidly with finer intervals but are highly effective for smooth functions with fewer divisions.

Numerical Integration in Python

- **Overview**: Numerical integration is essential for approximating definite integrals when analytic solutions are impractical. Python, with libraries like scipy.integrate, offers tools for implementing these methods efficiently.

- **Applications**: Common in engineering, physics, and computational science, numerical integration helps in scenarios involving irregular functions or data.

- **Key Methods**:
  - **Trapezoidal Rule**: Approximates the area under a curve using trapezoids.
  - **Cumulative Trapezoidal Integration**: Useful for analyzing incremental integration over intervals.
  - **Adaptive Quadrature Integration**: Higher accuracy method also providing an error estimate.

Let's use the *trapz* function to approximate $\int \sin(x)dx$ for 11 equally spaced points over the whole interval. Compare this value to the one computed in the earlier example using the Trapezoid Rule.

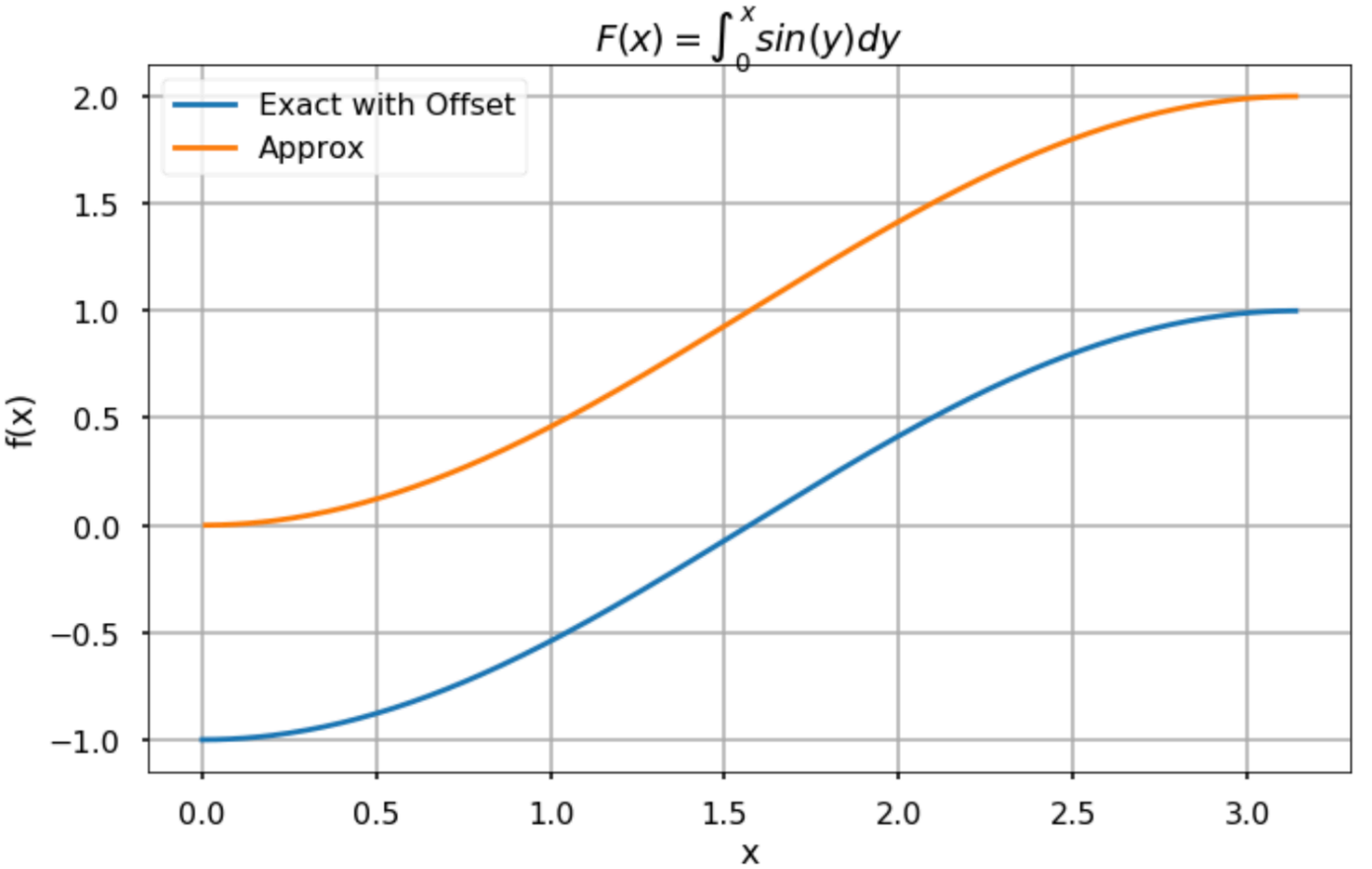
```
import numpy as np
from scipy.integrate import trapz

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_trapz = trapz(f,x)
I_trap = (h/2)*(f[0] + 2 * sum(f[1:n-1]) + f[n-1])

print(I_trapz)
print(I_trap)
```

```
1.9835235375094542
1.9835235375094546
```

Sometimes we want to know the approximated cumulative integral. That is, we want to know $F(X) = \int_{x_0}^{X} f(x)dx$. For this purpose, it is useful to use the *cumtrapz* function *cumsum*, which takes the same input arguments as *trapz*.

Let's use the *cumtrapz* function to approximate the cumulative integral of $f(x)$=sin(x) from 0 to π with a discretization step of 0.01. The exact solution of this integral is $F(x)$=-cos(x).

```
from scipy.integrate import cumtrapz
import matplotlib.pyplot as plt

%matplotlib inline
plt.style.use('seaborn-poster')

x = np.arange(0, np.pi, 0.01)
F_exact = -np.cos(x)
F_approx = cumtrapz(np.sin(x), x)

plt.figure(figsize = (10,6))
plt.plot(x, F_exact)
plt.plot(x[1::], F_approx)
plt.grid()
plt.tight_layout()
plt.title('$F(x) = \int_0^{x} sin(y) dy$')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['Exact with Offset', 'Approx'])
plt.show()
```



Let's use the *integrate.quad* function to compute $\int \sin(x)dx$. Compare the answer with the correct answer of 2.

```
from scipy.integrate import quad

I_quad, est_err_quad = \
          quad(np.sin, 0, np.pi)
print(I_quad)
err_quad = 2 - I_quad
print(est_err_quad, err_quad)
```

```
2.0
2.220446049250313e-14 0.0
```

TUTORIAL 1

Welcome to your first PHYB57 tutorial! We'll be meeting 3PM to 5PM every Monday in IC 230. I am your TA, Aishani Chaudhuri. You can reach me at aishani.chaudhuri@mail.utoronto.ca for any course-related questions.

SETTING UP JUPYTER NOTEBOOK

For this course, we'll mainly be using Jupyter notebooks (like this one)! You will be expected to submit assignments as Jupyter notebooks (file extension: .ipynb) with clear descriptions of the work, in the form of markdown cells or comments. So, let's get you set up!

1. Check that Python is installed

Run the following in your Terminal (macOS/Linux) or Command Prompt (Windows) to check that you have Python up and running.
`python3 --version`

2. Installing Jupyter

Run the following to install Jupyter.
`pip install jupyter`

If you're having trouble with pip, try the following.
`python3 get-pip.py` or `python3 -m ensurepip --upgrade` on macOS/Linux
`py get-pip.py` or `y -m ensurepip --upgrade` on Windows

3. Opening a Jupyter Notebook

Run the following to open Jupyter up in your browser.
`jupyter notebook`
Once this is open, navigate to the folder where you want to create files. Click 'New' in the top right corner and select 'Python 3 (ipykernel)' to open up a new Jupyter notebook.
To close Jupyter, it's not enough to simply close the tab! Return to the Terminal/Command Prompt and press `Ctrl + C`, type `y` and press `Enter`.

4. Alternative: Visual Studio Code

If you're more accustomed to using VSCode, there is a Jupyter extension that you can install that will allow you to create/open/edit Jupyter notebooks in VSCode.

5. Alternative: UofT's JupyterHub

If you can't get any of the above to work, UofT provides access to JupyterHub with your student account: https://datatools.utoronto.ca/.

VARIABLES

A variable is a way to store data that can be updated, referred to, or used at a later time. Variables can be one of many data types: integers, floats (decimals), strings (sequences of characters), booleans (true/false) etc. We assign values to variables using the `=` sign.

In [1]:
```
a = 2    # assigns the value 2 to the variable a
b = 1.6
c = "Hello World!"
d = a + 1   # uses variable a to assign a value to the variable d

print(a, b, c, d)    # output: 2 1.6 Hello World 3
```

```
2 1.6 Hello World! 3
```

We can check the type of a variable using the type() function.

In [2]:
```
print(type(a))    # type: int (integer)
print(type(b))    # type: float (float)
print(type(c))    # type: str (string)
```

```
<class 'int'>
<class 'float'>
<class 'str'>
```

When using variables, use descriptive names! It is more useful to have a variable named `dist` for distance than to have one called `x`. Also avoid using names that already have connotations in Python — common examples include `len`, `int`, `sum`, etc.

The values of a variable can be overwritten, but it will not affect anything that previously referred to that variable.

In [3]:
```
a = 4.5     # changes the value of a from 2 to 4.5

print(a, d)    # output: 4.5 3 (value of d does not change)
```

```
4.5 3
```

MATH OPERATIONS

We can do all sorts of simple and complicated math with Python.

In [4]:
```
p = a + b    # adds a and b
print(p)     # output: 6.1

# CHECK: which value of a is being used here, 2 or 4.5?

q = a - b    # subtracts b from a
print(q)     # output: 2.9

r = a * b    # multiplies a and b
print(r)     # output: 7.2

s = a / b    # divides a by b
print(s)     # output: 2.8125

t = a ** 2   # raises a to the 2nd power
print(t)     # output: 20.25
```

```
6.1
2.9
7.2
2.8125
20.25
```

In [5]:
```
# import Python's math module
import math

x = math.sqrt(16)        # takes the square root of the input
print(x)                 # output: 4.0

y = math.cos(2*math.pi)  # finds the cosine of the input (uses radians)
print(y)                 # output: 1.0
```

```
4.0
1.0
```

In [6]:
```
# import NumPy; has many functionalities useful for scientific computing
import numpy as np

x = np.sqrt(16)          # takes the square root of the input
print(x)                 # output: 4.0

y = np.cos(2*np.pi)      # finds the cosine of the input (uses radians)
print(y)                 # output: 1.0
```

```
4.0
1.0
```

EXERCISE: Assign the value of the square root of 3 to a variable, and then divide it by 2. Print the sine of this number.

In [7]:
```
n = np.sqrt(3)
n = n/2
print(np.sin(n))
```

```
0.7617599814162892
```

STRINGS

A string is a sequence of characters enclosed within quotes (single or double). A lot of operations can be performed on a string.

In [8]:
```
a = "Hello World!"

print(len(a))    # output: 12 (includes space and !)
```

```
12
```

A string can be indexed, with the index starting from 0. This is also useful for slicing strings. Negative indices, starting at -1, index from the end of the string.

To slice a string, specify the starting index and the index AFTER the stopping index.

In [9]:
```
print(a[4])      # output: o
print(a[-1])     # output: !

print(a[0:4])    # output: Hell
print(a[0:5])    # output: Hello
```

```
o
!
Hell
Hello
```

Non-strings can be converted to strings using the `str()` function. Strings can be concatenated (added together) using the `+` operator. You can also change the case of alphabet characters using the .lower() and .upper() functions.

In [10]:
```
b = "PHYB"
c = 57

# CHECK: what would happen if I tried b + c?

c = str(c)
d = b + c    # concatenates the strings "PHYB" and "57"
print(d)     # output: PHYB57

print(d.lower())    # output: phyb57
```

```
PHYB57
phyb57
```

A really useful way to write strings are formatted strings! This allows you to incorporate variables into strings of any data type without manually changing their type to `str`, and also directly evaluate expressions.

In [11]:
```
x = 4
y = 3
```

```
print(f"{x} raised to the power {y} equals {x ** y}.")    # output: 4 raised
```

```
4 raised to the power 3 equals 64.
```

EXERCISE: Assign the string "This week, I have a great many things to do!" to a variable. Then, cut out three pieces of the string: from the 14th character to the 26th character (inclusive), and from the 6th to the 9th character (inclusive), and the last character of the string. Concatenate the three pieces and print to reveal a hidden message!

In [12]:
```
message = "This week, I have a great many things to do!"
piece1 = message[13:26]
piece2 = message[5:9]
piece3 = message[-1]
print(piece1 + piece2 + piece3)
```

```
have a great week!
```

LOGICAL EXPRESSIONS AND OPERATORS

A logical expression is an expression that has a truth value attached to it — it is either True or False. The boolean data type has only two values: 0/False and 1/True.

In [13]:
```
print(1 < 2)    # output: True
print(1 > 2)    # output: False
```

```
True
False
```

We use comparison operators to construct logical expressions.

- `>` : Greater than
- `>=` : Greater than or equal to
- `<` : Less than
- `<=` : Less than or equal to
- `==` : Equal
- `!=` : Not equal

Note the difference between `=` and `==`. The first is used to assign values to variables, while the second is used to evaluate the equality of two values.

In [14]:
```
b = 3
a = b         # assigns the value of b to a
print(a)      # output: 3

c = (a == b)  # assigns the boolean value of the logical expression to c
print(c)      # output: True
```

```
3
True
```

In Python, a True expression has a value of 1, and a False expression has a value of 0.

In [15]:
```python
print((2 > 1) + (4 > 1) + (2 == 4))    # output: 2 (1 + 1 + 0)
```

```
2
```

Logical operators are used to create more complicated conditions involving two or more logical expressions.

- `and` : True if both expressions are true
- `or` : True if at least one expression is true
- `not` : Inverts the truth value of an expression

In [16]:
```python
P = True
Q = False

and_operator = P and Q
or_operator = P or Q
not_operator = not P
combining_operators = P and not Q

print(and_operator, or_operator, not_operator, combining_operators)    # out
```

```
False True False True
```

EXERCISE: Suppose P and Q are both False. Come up with 5 different logical expressions that evaluate to True.

Examples: not P, not Q, not (P and Q), not (P or Q), P or not Q, etc.

TUTORIAL 2

Today we're going to discuss data structures — lists, tuples, sets, and dictionaries. What is a data structure?

LISTS

Lists are defined using square brackets: `[item, item, item]`. The items in a list can be of any data type, and are ordered and changeable.

- ordered: `[1, 2, 3]` is NOT the same as `[3, 1, 2]`
- changeable: the items in the list can be modified

The position of an element in a list is determined by its index, starting at 0.

In [1]:
```
int_list = [1, 3, 2, 7]
str_list = ["a", "b", "c", "a"]    # items can be repeated
mixed_list = [True, -4.1, "hi!", int_list, [5, 15]]   # different data types

print(int_list[2])        # output: 2; prints the 3rd item in int_list
print(mixed_list[4][0])   # output: 5; prints the 1st item in the 5th item [
```

```
2
5
```

A lot of operations that we used on strings also work on lists! There are also some specialised functions such as `append()` and `remove()` that are particular to lists.

In [2]:
```
print(len(int_list))    # output: 4

concat_list = int_list + str_list
print(concat_list)      # output: [1, 3, 2, 7, "a", "b", "c", "a"]

str_list_slice = str_list[0:3]    # slices the list from index 0 to index 2
print(str_list_slice)             # output: ["a", "b", "c"]

str_list.remove("a")    # removes the first instance of "a" in the list
print(str_list)         # output: ["b", "c", "a"]

str_list.append("a")    # adds "a" to the end of the list
print(str_list)         # output: ["b", "c", "a", "a"]

str_list.pop(1)    # removes the element at index 1
print(str_list)    # output: ["b", "a", "a"]

str_list.insert(2, "c")    # inserts "c" at index 2
print(str_list)            # output: ["b", "a", "c", "a"]
```

```
str_list[3] = "k"    # sets the value of the item at index 3 to "k"
print(str_list)      # output: ["b", "a", "c", "k"]
```

```
4
[1, 3, 2, 7, 'a', 'b', 'c', 'a']
['a', 'b', 'c']
['b', 'c', 'a']
['b', 'c', 'a', 'a']
['b', 'a', 'a']
['b', 'a', 'c', 'a']
['b', 'a', 'c', 'k']
```

It is also easy to check membership with lists, and converting other sequence data types (like strings) to lists.

In [3]:
```
print(3 in int_list)    # output: True

print(list("hello"))    # output: ["h", "e", "l", "l", "o"]
```

```
True
['h', 'e', 'l', 'l', 'o']
```

EXERCISE: `fruits = ["apple", "banana", "cherry", "orange", "mango"]`

1. Print the last fruit.
2. Print the first three fruits.
3. Replace "banana" with "blueberry".
4. Add "grape" to the fourth spot in the list.
5. Remove "cherry".
6. Reverse the entire list.
7. Print the final list.

In [3]:
```
fruits = ["apple", "banana", "cherry", "orange", "mango"]
print(fruits[-1])
print(fruits[0:3])
fruits[1] = "blueberry"
fruits.insert(3, "grape")
fruits.remove("cherry")
print(fruits[::-1])
fruits.reverse()
print(fruits)
```

```
mango
['apple', 'banana', 'cherry']
['mango', 'orange', 'grape', 'blueberry', 'apple']
['mango', 'orange', 'grape', 'blueberry', 'apple']
```

TUPLES

Tuples are defined using round brackets: `(item, item, item)`. The items in a tuple can be of any data type, and are ordered and immutable.

- ordered: `(1, 2, 3)` is NOT the same as `(3, 1, 2)`
- immutable: the items of the tuple CANNOT be modified

The position of an element in a tuple is determined by its index, starting at 0.

Some list functions also work on tuples, including indexing, slicing, and `len()`. Tuples can also be concatenated to create new tuples. Tuples are especially useful when you want constant values in your code, or when you want to unpack results into multiple variables.

In [4]:
```
my_tuple = (2, 4, 8, 16, 32)

print(len(my_tuple))    # output: 5
print(my_tuple[1])      # output: 4
print(my_tuple[1:3])    # output: (4, 8)

a, b, c, d, e = my_tuple    # automatically assigns each variable a value fr
print(a, c, d)              # output: 2, 8, 16

new_item = (64,)          # the comma indicates that this is a tuple
my_longer_tuple = my_tuple + new_item
print(my_longer_tuple)    # output: (2, 4, 8, 16, 32, 64)
```

```
5
4
(4, 8)
2 8 16
(2, 4, 8, 16, 32, 64)
```

EXERCISE: `student = ("Alex", 19, "Computer Science", 87)`

1. Print their age.
2. Print their major.
3. Create a new tuple that contains the original information plus "Toronto" as a location.
4. Unpack the new tuple into five variables.

In [9]:
```
student = ("Alex", 19, "Computer Science", 87)
name, age, major, score = student
print(age)
print(student[1])
print(student[2])
location = ("Toronto",)
student_new = student + location
print(student_new)
```

```
name, age, major, score, city = student_new
print(name, age, major, score, city)
```

```
19
19
Computer Science
('Alex', 19, 'Computer Science', 87, 'Toronto')
Alex 19 Computer Science 87 Toronto
```

SETS

Sets are defined using curly brackets: `{item, item, item}`. The items in a set can only be of immutable data types (so, not lists!); they are unordered and changeable.

- unordered: `{1, 2, 3}` is the same as `{3, 1, 2}`; this means that items cannot be repeated or indexed
- changeable: the items in the set can be modified

A set can be created from a list, a tuple, or a string using the `set()` function. The mathematical operations you might be used to performing on sets (union, intersection, etc.) can also be performed in Python.

In [5]:
```
set_1 = set([1, 3, 5, 2, 5, 6, 2, 4, 2])
print(set_1)     # output: {1, 2, 3, 4, 5, 6}

set_2 = {2, 4, 6, 8, 12}
set_2.add(10)
set_2.remove(12)
print(set_2)     # output: {2, 4, 6, 8, 10}

union_set = set_1.union(set_2)    # returns set of items in either set witho
print(union_set)                  # output: {1, 2, 3, 4, 5, 6, 8, 10}

intersection_set = set_1.intersection(set_2)    # returns set of items commo
print(intersection_set)                         # output: {2, 4, 6}

diff_set = set_1.difference(set_2)    # returns set of items in set_1 but no
print(diff_set)                       # output: {1, 3, 5}

symmdiff_set = set_1.symmetric_difference(set_2)    # returns set of items i
print(symmdiff_set)                                 # output: {1, 3, 5, 8, 1

print(set_1.issubset(union_set))         # output: True
print(set_1.issuperset(symmdiff_set))    # output: False
```

```
{1, 2, 3, 4, 5, 6}
{2, 4, 6, 8, 10}
{1, 2, 3, 4, 5, 6, 8, 10}
{2, 4, 6}
{1, 3, 5}
{1, 3, 5, 8, 10}
True
False
```

EXERCISE: `list_of_colours = ["red", "blue", "green", "yellow", "blue", "red"]`

1. Create a set of unique colours.
2. Add "purple".
3. Remove "blue".
4. Check whether "pink" is in the set.
5. Create another set: `more_colours = {"yellow", "orange", "purple", "black"}`.
6. Find the union and intersection of the two sets.
7. Find the set of items only in `more_colours`.

In [17]:
```
list_of_colours = ["red", "blue", "green", "yellow", "blue", "red"]
colours = set(list_of_colours)
print(colours)
colours.add("purple")
colours.remove("blue")
print(colours)
print("pink" in colours)

more_colours = {"yellow", "orange", "purple", "black"}
union_set = colours.union(more_colours)
print(union_set)
intersection_set = colours.intersection(more_colours)
print(intersection_set)

for colour in more_colours:
    if colour not in colours:
        print(colour)

print(more_colours.difference(colours))
```

```
{'yellow', 'green', 'blue', 'red'}
{'yellow', 'green', 'purple', 'red'}
False
{'yellow', 'green', 'black', 'orange', 'purple', 'red'}
{'yellow', 'purple'}
black
orange
{'orange', 'black'}
```

DICTIONARIES

Dictionaries are collections of key:value pairs, defined using curly brackets: `{key:value, key:value, key:value}`. Keys can be of any datatype except for

lists, while values can be of any data type. The items in a dictionary are unordered and changeable.

- unordered: the items in a dictionary are accessed by keys instead of a numerical index; every key is unique
- changeable: the items in the dictionary can be modified

Dictionaries are really useful for storing data when a unique identifier (key) maps to a specific value.

In [18]:
```
my_dictionary = dict([("name", "Aishani"), ("course", "PHYB57")])    # conve
my_dictionary["age"] = 21    # adds a key:value pair to the dictionary

print(my_dictionary)         # output: {"name": "Aishani", "course": "PHYB57

print(my_dictionary.keys())      # output: dict_keys(["name", "course", "age
print(my_dictionary.values())    # output: dict_values(["Aishani", "PHYB57",

print(my_dictionary["course"])    # output: PHYB57

print("fave_colour" in my_dictionary)    # output: False; checks if key "fav
```

```
{'name': 'Aishani', 'course': 'PHYB57', 'age': 21}
dict_keys(['name', 'course', 'age'])
dict_values(['Aishani', 'PHYB57', 21])
PHYB57
False
```

EXERCISE: You run a corner store. Currently, you have only two items in stock: sodas (7) and chips (3).

1. Create an inventory dictionary.
2. A customer buys a soda and a packet of chips. Update your inventory.
3. You receive a shipment of 15 chocolate bars and 20 cookie boxes. Update your inventory.
4. Over the course of the day, you sell 4 more sodas, all your remaining chips, 9 chocolate bars and 2 cookie boxes. Update and print your final inventory.

In [21]:
```
inventory = {"sodas":7, "chips":3}
inventory["sodas"] -= 1
inventory["chips"] = inventory["chips"] - 1
print(inventory)
inventory["chocolate"] = 15
inventory["cookies"] = 20
print(inventory)
inventory["sodas"] -= 4
inventory["chips"] = 0
inventory["chocolate"] -= 9
inventory["cookies"] -= 2
print(inventory)
```

```
{'sodas': 6, 'chips': 2}
{'sodas': 6, 'chips': 2, 'chocolate': 15, 'cookies': 20}
{'sodas': 2, 'chips': 0, 'chocolate': 6, 'cookies': 18}
```

In [26]:
```python
import numpy as np

np_array = np.array([1, 2, 3, 4])
print(type(np_array))

np_array += 2
print(np_array)

np_tuple_array = np.array((1, 2, 3, 4))
print(np_tuple_array)
```

```
<class 'numpy.ndarray'>
[3 4 5 6]
[1 2 3 4]
```

In [ ]:
```python

```

TUTORIAL 3

Welcome to Tutorial 3! Today, we will build upon the variables, data types, and data structures, and explore conditional statements, loops, and functions.

IF/ELSE STATEMENTS

By default, Python executes code sequentially from top to bottom. Conditional statements allow our program to make decisions and execute specific blocks of code only when certain conditions are met.

In Python, code blocks under a condition are defined using a colon `:` and indentation (standard: 4 spaces).

- `if` : evaluates a condition and runs the indented block if the condition is `True`
- `elif` (short for "else if"): checked only if all preceding `if` and `elif` conditions were `False`
- `else` : runs if none of the preceding conditions were met

In [1]:
```
temp = -5

if temp <= 0:
    state = "solid"
elif temp < 100:
    state = "liquid"
else:
    state = "gas"

print(f"At {temp}°C, water is a {state}.")    # output: At -5°C, water is a

# CHECK: what would happen if we used three separate 'if' statements instead
```

```
At -5°C, water is a solid.
```

We can also combine multiple conditions using logical operators (`and` , `or` , `not`), or nest `if` statements inside one another.

In [2]:
```
x = 14

# combining conditions with 'and'
if x > 0 and x % 2 == 0:
    print(f"{x} is positive and even.")    # output: 14 is positive and even

# nested if statement
if x > 0:
    if x % 2 == 0:
        print("positive even number")     # output: positive even number
```

```
    else:
        print("positive odd number")
else:
    print("non-positive number")
```

```
14 is positive and even.
positive even number
```

EXERCISE: Evaluating Grades

A student achieves a final grade of 83 in a course. Their attendance for that course is 92%.

1. Write `if/elif/else` statements to assign a letter `grade` to a student based on their numerical `score`:
   - Score above (and including) 85: "A"
   - Score above (and including) 75 and below 85: "B"
   - Score above (and including) 50 and below 75: "C"
   - Otherwise: "F"
2. Using a nested `if` statement, check if the student also qualifies for "Honours Standing" (letter grade A and attendance above 90%).
3. Print the final grade and status.

FOR LOOPS

A `for` loop is used to iterate over a sequence (such as a list, string, tuple, dictionary, or range of numbers).

To loop over a list/string/tuple/etc.:

- `for variable in data_structure`: in Python, the loop variable automatically takes on the value of each element in the sequence, one by one
- `for idx, variable in enumerate(seq)`: the loop variable returned is a tuple of the element's index and value

These are a few ways to generate sequences of numbers:

- `range(stop_val)`: generates numbers from `0` up to (but not including) `stop_val`
- `range(start_val, stop_val, step_val)`: custom starting point `start_val`, stopping point `stop_val`, and step size `step_val` (default = 1)

Some other useful loop commands:

- `break`: exits the loop immediately and executes the rest of the code
- `continue`: skips the rest of the current iteration and jumps to the next one

In [3]:
```python
# looping over a list
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    print(f"{num} squared: {num**2}")

# accumulating values over a range
total = 0
for i in range(1, 6):   # iterates over 1, 2, 3, 4, 5 (stops before 6)
    total += i          # shorthand for total = total + i

print("Sum from 1 to 5:", total)    # output: Sum from 1 to 5: 15
```

```
1 squared: 1
2 squared: 4
3 squared: 9
4 squared: 16
5 squared: 25
Sum from 1 to 5: 15
```

In [4]:
```python
measurements = [5, 12, -1, 18, 999, 24]  # -1 is missing data, 999 is an err

for idx, val in enumerate(measurements):
    if val == -1:
        print(f"Skipping negative reading at index {idx}")
        continue   # skips to next measurement
    if val == 999:
        print(f"Critical error at index {idx}! Stopping loop.")
        break      # exits loop entirely
    print(f"Index {idx}: processed value {val}")

# CHECK: why was value 24 never printed?
```

```
Index 0: processed value 5
Index 1: processed value 12
Skipping negative reading at index 2
Index 3: processed value 18
Critical error at index 4! Stopping loop.
```

EXERCISE: `times = [10.2, 14.5, 9.8, 12.0, 15.6, 8.4, 11.1]`

1. Using a `for` loop, compute the total run time and the average of all run times.
2. Using `enumerate()`, print the index and value of every run that took longer than `12.0` seconds.
3. Using a `for` loop with `continue`, count how many runs took at least `10.0` seconds (skipping any run `< 10.0`).

WHILE LOOPS

A `while` loop repeatedly executes a block of code as long as a specified condition remains `True`.

Generally:

- use `for` loops when you know the sequence or the number of iterations in advance
- use `while` loops when looping depends on an evolving condition (e.g. numerical convergence, user input, reaching a threshold, etc.)

For while loops, always make sure the condition will eventually become `False` (or encounter a `break`), otherwise your code will be stuck in an infinite loop.

In [5]:
```python
# simulating radioactive decay: half-life of 5 years
mass = 80.0       # initial mass in grams
half_life = 5.0   # years per half-life
years = 0.0

while mass >= 10.0:
    mass = mass / 2.0
    years += half_life
    print(f"After {years} years: {mass:.1f} g remaining")

print(f"Decayed below 10 g in {years} years.")

# CHECK: what would happen if we commented out the line 'years += half_life'
```

```
After 5.0 years: 40.0 g remaining
After 10.0 years: 20.0 g remaining
After 15.0 years: 10.0 g remaining
After 20.0 years: 5.0 g remaining
Decayed below 10 g in 20.0 years.
```

In [6]:
```python
# finding the first positive integer whose cube exceeds 500 using while True
n = 1
while True:
    if n ** 3 > 500:
        print(f"Found: {n}^3 = {n**3} > 500")    # output: Found: 8^3 = 512
        break
    n += 1
```

```
Found: 8^3 = 512 > 500
```

EXERCISE: The Collatz Sequence

The Collatz sequence takes any positive integer `n` and follows two rules:

- If `n` is even, divide it by 2: `n = n / 2`
- If `n` is odd, multiply it by 3 and add 1: `n = 3 * n + 1`

Starting with `n = 26`:

1. Use a `while` loop to repeatedly apply the rules until `n` equals `1`.
2. Count the number of steps taken to reach `1`.
3. Print the value of `n` at each step and the total step count.

LIST/DICTIONARY COMPREHENSION

For loops and conditional statements can be used to efficiently create lists/dictionaries/etc.

- `lst` = [`output_val` for `input_val` if `condition`]
- `dict` = { `key:value` for `input_val` if `condition` }

In [7]:
```python
inventory = {
    "apple": 0.90,
    "bread": 1.00,
    "avocado": 2.50,
    "banana": 0.60,
    "almond milk": 3.40,
}

# create a list of items above $2
costly_items_1 = []
for key in inventory:
    if inventory[key] > 2:
        costly_items_1.append(key)
print(costly_items_1)    # output: ["avocado", "almond milk"]

costly_items_2 = [key for key in inventory if inventory[key] > 2]
print(costly_items_2)    # output: ["avocado", "almond milk"]

# create a new 20% discounted dictionary
discounted_prices = {key:round(inventory[key] * 0.8, 2) for key in inventory
print(discounted_prices)    # output: {"apple": 0.72, "bread": 0.8, "avocadc
```

```
['avocado', 'almond milk']
['avocado', 'almond milk']
{'apple': 0.72, 'bread': 0.8, 'avocado': 2.0, 'banana': 0.48, 'almond milk':
2.72}
```

FUNCTIONS

A function is a reusable block of code that takes inputs (called parameters or arguments), performs actions, and returns an output.

A function is defined using the `def` keyword: `def function_name(param1, param2):`

- `return` sends a value back to the caller so it can be saved in a variable or used in subsequent calculations

- `return` vs. `print()`: `print()` only displays text on the screen; `return` gives the resulting data back to your program

A function can have default parameters, which have set values unless specifically modified when the function is called (e.g., `g=9.81`).

It is good practice to use docstrings when writing functions. Docstrings are multi-line comments written using triple quotes (`"""..."""`) immediately under the `def` line to explain what the function does, what inputs it expects, and what outputs it returns.

In [8]:
```
import math

# function to calculate kinetic energy
def kinetic_energy(mass, vel):
    """
    Calculates kinetic energy: KE = 0.5 * m * v^2

    INPUTS:
    mass (float): mass in kg
    vel (float): velocity in m/s

    OUTPUTS:
    ke (float): kinetic energy in J
    """
    ke = 0.5 * mass * (vel ** 2)
    return ke

# function to calculate range of a projectile, using a default constant for
def projectile_range(v0, angle_deg, g=9.81):
    """
    Calculates horizontal range of a projectile.

    INPUTS:
    v0 (float): initial velocity in m/s
    angle_deg (float): angle of projectile launch in degrees

    OUTPUTS:
    r (float): range of projectile in km, rounded to 2 decimal places
    """
    angle_rad = math.radians(2 * angle_deg)
    r = (v0 ** 2) * math.sin(angle_rad) / g
    return round(r, 2)

# calling kinetic_energy and storing the returned value
energy = kinetic_energy(2.0, 3.0)
print("Kinetic Energy:", energy, "J")    # output: Kinetic Energy: 9.0 J

# calling projectile_range with default g
range_earth = projectile_range(20.0, 45.0)

# calling projectile_range with Mars gravity (g=3.71)
range_mars = projectile_range(20.0, 45.0, g=3.71)
```

```
print(f"Range on Earth: {range_earth} m")    # output: Range on Earth: 40.77
print(f"Range on Mars: {range_mars} m")      # output: Range on Mars: 107.82
```

```
Kinetic Energy: 9.0 J
Range on Earth: 40.77 m
Range on Mars: 107.82 m
```

Functions can be defined inside other function; these are called nested functions.

Why use nested functions?

1. helper code that is only meaningful inside the outer function
2. the inner function has access to variables in the outer function's scope
3. an outer function can configure and return a specialized inner function

In [9]:

```
# nested helper function
def solve_quadratic(a, b, c):
    """
    Finds the real roots of ax^2 + bx + c = 0 using a nested discriminant he

    INPUTS:
    a (float): leading coefficient of function
    b (float): middle coefficient of function:
    c (float): constant of function

    OUTPUTS:
    (root1, root2): tuple of real roots
    """
    def get_discriminant():
        # accesses a, b, and c directly from the enclosing scope
        return b**2 - 4*a*c

    d = get_discriminant()
    if d < 0:
        return "No real roots"
    root1 = (-b + d**0.5) / (2*a)
    root2 = (-b - d**0.5) / (2*a)
    return (root1, root2)

print(solve_quadratic(1, -5, 6))    # output: (3.0, 2.0)
print(solve_quadratic(1, 2, 5))     # output: No real roots

# specialised inner function
def make_power_function(power):
    """
    Creates a customized function that raises inputs to the specified power.

    INPUTS:
    power (float): specified power of custom function

    OUTPUTS:
    power_func (function): function that raises input to specified power
    """
    def power_func(x):
        return x ** power  # remembers 'power' even after make_power_functio
```

```
    return power_func

square = make_power_function(2)
cube = make_power_function(3)

print(square(5))    # output: 25
print(cube(3))      # output: 27

# CHECK: can you call 'get_discriminant()' directly from outside 'solve_quad
```

```
(3.0, 2.0)
No real roots
25
27
```

A lambda function is a small, often unnamed function defined in a single line using the `lambda` keyword: `lambda arguments: expression`.

It can take any number of arguments, separated by commas, but it can contain **only one expression**, which is automatically evaluated and returned (no `return` keyword is used).

It is best used for short, throwaway operations, especially as key functions in `sorted()`, `min()`, `max()`, or with `filter()` and `map()`. If logic requires loops or multiple lines, always define a regular function using `def` instead.

In [10]:
```
# regular function vs. lambda function
# def add(x, y):
#     return x + y
add = lambda x, y: x + y
print(add(4, 7))    # output: 11

# converting Celsius to Fahrenheit
c_to_f = lambda c: (c * 9/5) + 32
print(f"20°C is {c_to_f(20)}°F")    # output: 20°C is 68.0°F

# sorting complex data (list of tuples) using lambda keys
students = [("Alice", 88), ("Bob", 92), ("Charlie", 78), ("Diana", 95)]

by_name = sorted(students, key=lambda item: item[0])
print("Sorted by name:", by_name)

by_score = sorted(students, key=lambda item: item[1])
print("Sorted by score:", by_score)
```

```
11
20°C is 68.0°F
Sorted by name: [('Alice', 88), ('Bob', 92), ('Charlie', 78), ('Diana', 95)]
Sorted by score: [('Charlie', 78), ('Alice', 88), ('Bob', 92), ('Diana', 9
5)]
```

EXERCISE: Lab Particle Detector Pipeline

A physics detector records particle events as tuples: `(particle_id, energy_GeV, is_valid)`.

```
raw_events = [
    ("p1", 12.4, True),
    ("p2", 8.1, False),   # corrupted sensor reading
    ("p3", 25.6, True),
    ("p4", 4.3, True),
    ("p5", 19.8, False),  # corrupted sensor reading
    ("p6", 15.0, True)
]
```

Tasks:

1. Write a function `process_detector_data(events, threshold=10.0)` that:
   - Defines an inner (nested) helper function `estimate_momentum(e)` that calculates momentum $p = \sqrt{E^2 - m^2}$ (assume rest mass $m = 0.5$ GeV).
   - Uses list comprehension to create a `list` of only valid events.
   - Uses a `for` loop to iterate through `valid_events`
   - Categorises valid events into `"high-energy"` if the energy is above the threshold, or `"low-energy"` otherwise.
   - Stores and returns a list of dictionaries with keys: `"id"`, `"energy"`, `"momentum"`, and `"category"` for each event.
2. Call `process_detector_data(raw_events)`.
3. Using `sorted()` with a `lambda` function as the sorting key, sort the processed list in descending order of momentum.
4. Print the final sorted list of events.

TUTORIAL 4

Welcome to Tutorial 4! Today, we're going to discuss algorithmic complexity and Big-O notation.

WHAT IS ALGORITHMIC COMPLEXITY?

Measuring how "fast" a program runs by using a stopwatch in seconds isn't really useful; instead, in computer science we measure algorithmic complexity by analyzing how the number of basic operations grows as a function of the input size (n).

- input size (n): 6he amount of data being processed (e.g., length of a list, number of particles, number of data points)
- basic operations: operations that take roughly constant time regardless of input size (e.g., addition, assignment, comparison)
- time complexity: how the execution time grows as n goes to infinity

BIG-O NOTATION

Big-O notation ($\mathcal{O}$) describes the upper bound (worst-case scenario) of an algorithm's growth rate as the input size n becomes arbitrarily large.

Common Orders of Growth (from fastest to slowest):

- $\mathcal{O}(1)$: constant time — runtime does not depend on input size
- $\mathcal{O}(\log n)$: logarithmic time — problem size is divided by a factor at each step
- $\mathcal{O}(n)$: linear time — runtime scales proportionally with n
- $\mathcal{O}(n \log n)$: linearithmic time — standard for efficient sorting (e.g. Merge Sort, Timsort in Python)
- $\mathcal{O}(n^2)$: quadratic time — typical of nested loops over the data
- $\mathcal{O}(2^n)$: exponential time — operations double with each added element

CONSTANT TIME: $\mathcal{O}(1)$

An algorithm runs in $\mathcal{O}(1)$ time if the number of operations is fixed and independent of the input size n.

In [11]:
```
# O(1) Operations
data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

```
# accessing a list element by index takes O(1) time
first = data[0]
last = data[-1]

# basic arithmetic takes O(1) time
calc = (first + last) * 2

# dictionary key lookup takes O(1) on average
scores = {"Alex": 87, "Sam": 92, "Jordan": 78}
alex_score = scores["Alex"]

print("First element:", first)
print("Last element:", last)
print("Calculation result:", calc)
print("Dict lookup ('Alex'):", alex_score)
```

```
First element: 10
Last element: 100
Calculation result: 220
Dict lookup ('Alex'): 87
```

LINEAR TIME: $\mathcal{O}(n)$

An algorithm is $\mathcal{O}(n)$ if its work grows directly in proportion to the input size n. If n doubles, the number of operations doubles.

Typical examples include a single `for` loop over a sequence, linear search, and built-in functions like `sum()`, `min()`, and `max()`.

In [12]:
```
def find_max(numbers):
    """
    Finds the maximum value in a list of size n.
    Complexity: O(n)
    """
    current_max = numbers[0]
    ops = 0

    for val in numbers:          # loop runs exactly n times
        ops += 1
        if val > current_max:
            current_max = val

    return current_max, ops

sample_data = [4, 12, 7, 25, 19, 3, 31, 8]
max_val, ops_count = find_max(sample_data)

print("Input length (n):", len(sample_data))          # output: 8
print("Maximum found:", max_val)                      # output: 31
print("Loop operations performed:", ops_count)        # output: 8
```

```
Input length (n): 8
Maximum found: 31
Loop operations performed: 8
```

QUADRATIC TIME: $\mathcal{O}(n^2)$

An algorithm is $\mathcal{O}(n^2)$ when the number of operations scales with the square of the input size. This commonly occurs with nested loops where each loop runs n times.

A foundational physics example is the N-body problem: calculating the pairwise gravitational or electrostatic forces between every pair of n particles. For n particles, there are $\frac{n(n-1)}{2} \approx \frac{1}{2}n^2$ unique pairs, which scales as $\mathcal{O}(n^2)$.

In [13]:
```python
def count_pairwise_interactions(particles):
    """
    Simulates computing interactions between all unique pairs of particles.
    Complexity: O(n^2)
    """
    n = len(particles)
    interactions = 0

    for i in range(n):                # outer loop runs n times
        for j in range(i + 1, n):     # inner loop runs (n - i - 1) times
            interactions += 1

    return interactions

for n in [10, 20, 40]:
    particles = list(range(n))
    count = count_pairwise_interactions(particles)
    print(f"n = {n:2d} particles -> {count:4d} pairwise force calculations")
```

```
n = 10 particles ->   45 pairwise force calculations
n = 20 particles ->  190 pairwise force calculations
n = 40 particles ->  780 pairwise force calculations
```

LOGARITHMIC TIME: $\mathcal{O}(\log n)$

An algorithm runs in $\mathcal{O}(\log n)$ time if each step reduces the remaining problem size by a constant fraction (usually half). Unless stated otherwise, $\log$ in computer science means $\log_2$.

Examples include binary search on a sorted list, and the bisection method for finding roots of equations in numerical methods:

- If $n = 16$: $\log_2(16) = 4$ steps
- If $n = 1,024$: $\log_2(1024) = 10$ steps
- If $n = 1,000,000$: $\log_2(10^6) \approx 20$ steps!

In [14]:
```python
def binary_search(sorted_list, target):
    """
    Finds the target in a sorted list by repeatedly halving the search windc
```

```
    Complexity: O(log n)
    """
    low = 0
    high = len(sorted_list) - 1
    steps = 0

    while low <= high:
        steps += 1
        mid = (low + high) // 2

        if sorted_list[mid] == target:
            return mid, steps
        elif sorted_list[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1, steps

data_1000 = list(range(1000))
index, steps_taken = binary_search(data_1000, target=742)

print("Sorted array size:", len(data_1000))
print(f"Target 742 found at index {index} in {steps_taken} steps.")
```

```
Sorted array size: 1000
Target 742 found at index 742 in 10 steps.
```

LINEARITHMIC TIME: $\mathcal{O}(n \log n)$

An algorithm runs in $\mathcal{O}(n \log n)$ time when it performs $\mathcal{O}(\log n)$ levels of work, with approximately $\mathcal{O}(n)$ work performed at each level.

A common example is merge sort:

1. The list is repeatedly divided in half, creating $\mathcal{O}(\log n)$ levels.
2. At each level, all n elements are processed while the smaller lists are merged.
3. Therefore, the total complexity is:

$$\mathcal{O}(n) \times \mathcal{O}(\log n) = \mathcal{O}(n \log n)$$

For example:

- If $n = 16$: $16 \times \log_2(16) = 16 \times 4 = 64$ operations
- If $n = 1,024$: $1,024 \times \log_2(1,024) = 1,024 \times 10 = 10,240$ operations
- If $n = 1,000,000$: $10^6 \times \log_2(10^6) \approx 20,000,000$ operations!

This is faster than $\mathcal{O}(n^2)$ for large inputs, which is why algorithms such as merge sort are useful for sorting large datasets.

In [17]:
```
def merge_sort(data):
    """
```

```
    Sorts a list using the merge sort algorithm.
    Complexity: O(n log n)
    """
    if len(data) <= 1:
        return data

    # divide the list in half
    mid = len(data) // 2
    left = data[:mid]
    right = data[mid:]

    # recursively sort both halves
    left = merge_sort(left)
    right = merge_sort(right)

    # merge the two sorted halves
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # add any remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result


data_1000 = list(range(1000, 0, -1))

sorted_data = merge_sort(data_1000)

print("Array size:", len(data_1000))
print("First 10 elements:", sorted_data[:10])
print("Last 10 elements:", sorted_data[-10:])
```

```
Array size: 1000
First 10 elements: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Last 10 elements: [991, 992, 993, 994, 995, 996, 997, 998, 999, 1000]
```

EMPIRICAL RUN TIME COMPARISON

We can use Python's `time` module to measure actual execution times and see how $\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$ behave in practice.

In [16]:
```
import time
```

```python
sizes = [500, 1000, 2000]

print(f"{'n':>6} | {'Linear O(n) [s]':>16} | {'Quadratic O(n^2) [s]':>20}")

for n in sizes:
    test_list = list(range(n))

    # linear O(n): sum of elements
    t0 = time.perf_counter()
    total = sum(test_list)
    t_linear = time.perf_counter() - t0

    # quadratic O(n^2): nested loop over n
    t0 = time.perf_counter()
    count = 0
    for i in test_list:
        for j in test_list:
            count += 1
    t_quadratic = time.perf_counter() - t0

    print(f"{n:6d} | {t_linear:16.6f} | {t_quadratic:20.6f}")
```

```
     n |  Linear O(n) [s] | Quadratic O(n^2) [s]
   500 |         0.000004 |             0.013219
  1000 |         0.000004 |             0.033316
  2000 |         0.000005 |             0.101183
```

TUTORIAL 5

Welcome to Tutorial 5! Today, we will explore how to work with text files in Python (opening, writing, appending, and reading) and introduce data visualization using Matplotlib for 2D and 3D scientific plotting.

OPENING AND WRITING TO TEXT FILES

Text files (`.txt`) contain plain text and are among the simplest and most universal formats for saving simulation outputs, experiment logs, and raw configuration data.

In Python, the built-in `open()` function is used to interact with files: `f = open(filename, mode)`

Common access modes include:

- `'r'` : Read mode (default). Opens an existing file for reading. Raises `FileNotFoundError` if the file does not exist.
- `'w'` : Write mode. Creates a new file if it does not exist, or overwrites the file if it already exists.
- `'a'` : Append mode. Writes new data to the end of the file without erasing existing content.
- `'r+'` : Read and write mode (the file must already exist; does not overwrite it on open).
- `'w+'` : Write and read mode (creates a new file or completely overwrites existing content).
- `'a+'` : Append and read mode (creates a new file if needed, appends to the end).

Remember to always close files after use with `f.close()` to ensure all buffered data is written to disk and system resources are freed. Alternatively, you can use the `with` statement, which automatically closes the file even if errors occur.

In [1]:
```
# writing to a text file in 'w' mode
f = open('sensor_log.txt', 'w')    # will save the .txt file to the director
for i in range(5):
    f.write(f"Sample {i}: reading = {i * 2.5:.2f} mV\n")
f.close()
print("Successfully wrote initial 5 samples to sensor_log.txt.")

# appending new data in 'a' mode
f = open('sensor_log.txt', 'a')
f.write("Sample 5: reading = 12.50 mV (appended)\n")
f.close()
print("Successfully appended sample 5 to sensor_log.txt.")
```

```
# using 'with'
with open('run_summary.txt', 'w') as f_out:
    f_out.write("Status: RUN COMPLETE\n")
    f_out.write("Exit Code: 0\n")
print("Wrote run_summary.txt using 'with' statement.")
```

```
Successfully wrote initial 5 samples to sensor_log.txt.
Successfully appended sample 5 to sensor_log.txt.
Wrote run_summary.txt using 'with' statement.
```

READING FROM TEXT FILES

Python provides several methods to read data from an opened file:

- `f.read()` : Reads the entire content of the file into a single string.
- `f.readline()` : Reads a single line up to the next newline character `\n` .
- `f.readlines()` : Reads all remaining lines into a list of strings, where each item is one line.
- Direct iteration (`for line in f:`): Iterates through lines one by one without loading the entire file into memory at once (recommended for large files).

When reading lines, strings will retain their trailing newline (`\n`). Use `.strip()` to remove unwanted leading and trailing whitespace.

In [2]:
```
# reading the entire file content at once with .read()
f = open('sensor_log.txt', 'r')
content = f.read()
f.close()
print("Entire Content (read)")
print(content)

# reading into a list of lines with .readlines()
f = open('sensor_log.txt', 'r')
lines = f.readlines()
f.close()
print(f"Lines List (readlines) --- ({len(lines)} lines total)")
print(f"Line 0: {lines[0]}")

# iterating line-by-line using 'with' and stripping newlines
print("\nLine-by-line iteration with .strip()")
with open('sensor_log.txt', 'r') as f_in:
    for idx, line in enumerate(f_in):
        clean_line = line.strip()
        print(f"Line {idx}: {clean_line}")
```

```
Entire Content (read)
Sample 0: reading = 0.00 mV
Sample 1: reading = 2.50 mV
Sample 2: reading = 5.00 mV
Sample 3: reading = 7.50 mV
Sample 4: reading = 10.00 mV
Sample 5: reading = 12.50 mV (appended)

Lines List (readlines) --- (6 lines total)
Line 0: Sample 0: reading = 0.00 mV


Line-by-line iteration with .strip()
Line 0: Sample 0: reading = 0.00 mV
Line 1: Sample 1: reading = 2.50 mV
Line 2: Sample 2: reading = 5.00 mV
Line 3: Sample 3: reading = 7.50 mV
Line 4: Sample 4: reading = 10.00 mV
Line 5: Sample 5: reading = 12.50 mV (appended)
```

NUMERICAL DATA IN TEXT FILES (NUMPY SAVETXT & LOADTXT)

In scientific computing, data is frequently stored as tables of numbers. In such cases, NumPy's inbuilt functions are useful:

- `np.savetxt(filename, data)` : Saves 1D or 2D numerical arrays directly to disk in plain text.
- `np.loadtxt(filename)` : Reads columns of numbers directly into NumPy arrays, automatically ignores lines starting with comments (`#`).

There are a lot of other functionalities for formatting that you can look into.

In [3]:
```python
import numpy as np

# create a 2D array of simulated trajectory data: [time (s), position (m), v
time_vals = np.linspace(0, 4, 5)
pos_vals = 0.5 * 9.81 * (time_vals ** 2)
vel_vals = 9.81 * time_vals

data_matrix = np.column_stack((time_vals, pos_vals, vel_vals))

# save numerical array to a text file with a header and 2 decimal places
np.savetxt('trajectory.txt', data_matrix, fmt='%.2f', header='Time(s) Positi
print("Saved 2D array to trajectory.txt with header.")

# load the array back from disk
loaded_data = np.loadtxt('trajectory.txt')
print("\nLoaded array back into memory:")
print(loaded_data)

# unpack columns directly into individual variables
```

```
t, x, v = np.loadtxt('trajectory.txt', unpack=True)
print(f"\nUnpacked time values: {t}")
print(f"Unpacked position values: {x}")
```

```
Saved 2D array to trajectory.txt with header.

Loaded array back into memory:
[[ 0.    0.    0.  ]
 [ 1.    4.91  9.81]
 [ 2.   19.62 19.62]
 [ 3.   44.15 29.43]
 [ 4.   78.48 39.24]]

Unpacked time values: [0. 1. 2. 3. 4.]
Unpacked position values: [ 0.    4.91 19.62 44.15 78.48]
```

2D PLOTTING IN PYTHON WITH MATPLOTLIB

Python's `matplotlib` library is the foundational standard for scientific and engineering data visualization. Its sub-module `matplotlib.pyplot` provides a MATLAB-like plotting interface.

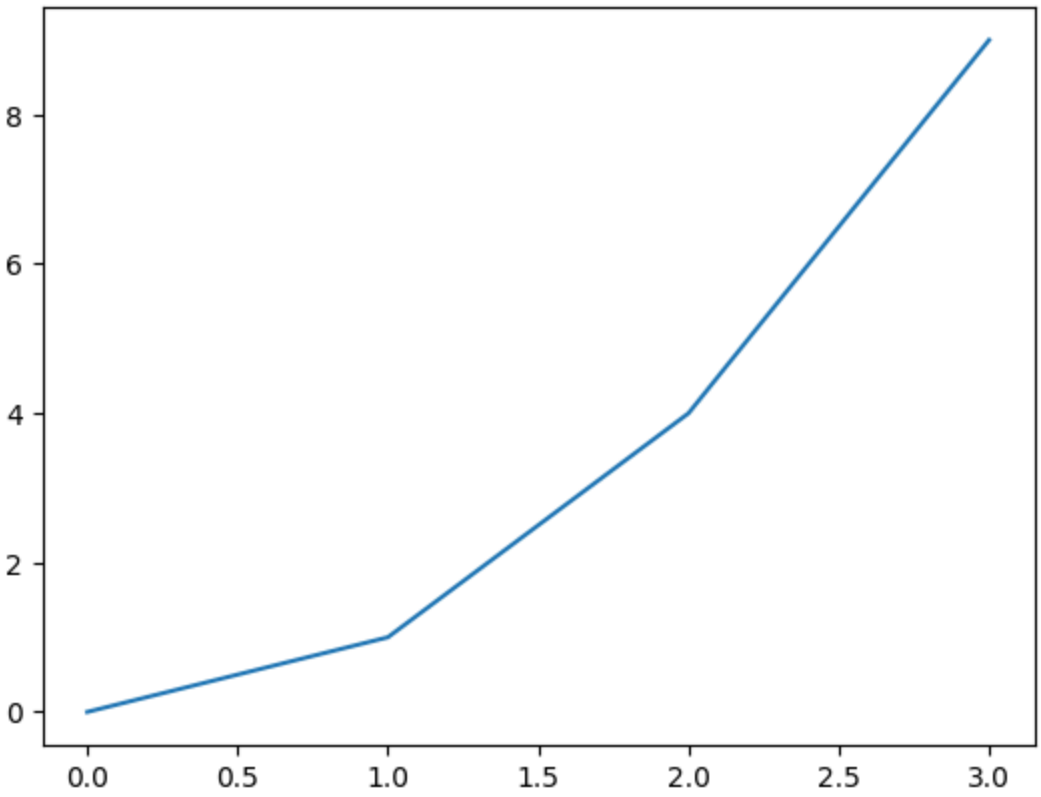
- Import convention: `import matplotlib.pyplot as plt`
- Basic command: `plt.plot(x, y)` takes coordinate sequences (lists or NumPy arrays) and draws a line or marker plot.
- Displaying the plot: `plt.show()` renders and displays the figure.

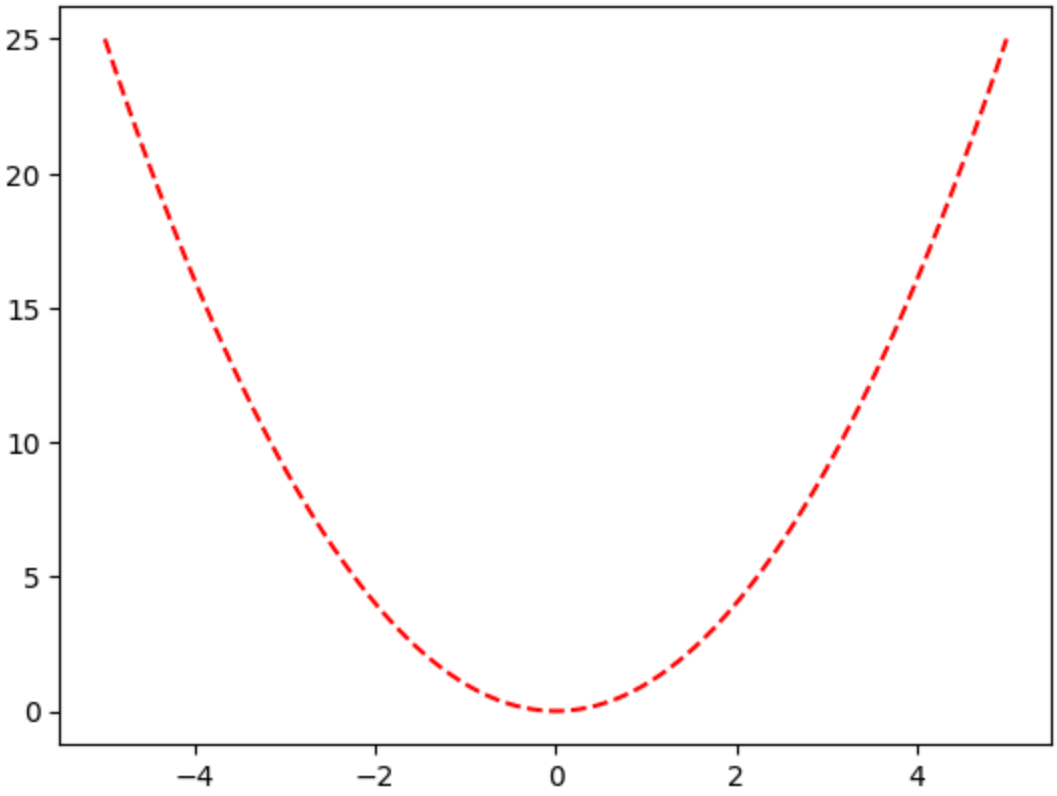
Customizing Line and Marker Styles

The format string argument in `plt.plot(x, y, '[color][marker][line]')` offers a fast shorthand:

- Colors: `'r'` (red), `'g'` (green), `'b'` (blue), `'k'` (black), etc.
- Markers: `'o'` (circle), `'*'` (star), `'x'` (cross), etc.
- Line styles: `'-'` (solid), `'--'` (dashed), `':'` (dotted), `'-.'` (dash-dot)

In [4]:
```
import matplotlib.pyplot as plt

# basic 2D line plot from coordinate lists
x = [0, 1, 2, 3]
y = [0, 1, 4, 9]

plt.plot(x, y)
plt.show()

# customized line and marker styles with numpy arrays
x_vals = np.linspace(-5, 5, 100)
plt.plot(x_vals, x_vals ** 2, color='r', linestyle='--')
plt.show()
```





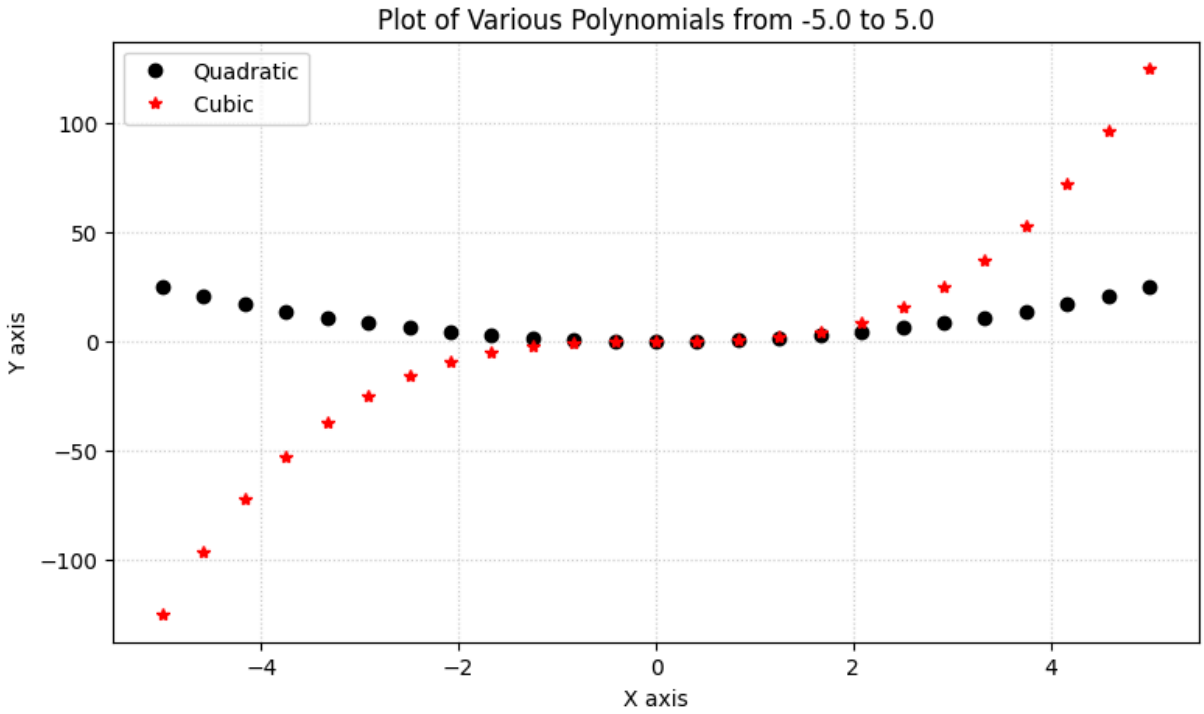
ENHANCING 2D PLOTS WITH TITLES, LABELS, AND LEGENDS

In scientific visualization, plots must always have clearly defined context: axes labels with units, informative titles, and legends to distinguish between multiple curves.

Key plot enhancement commands:

- `plt.figure(figsize=(width, height))` : Customizes canvas dimensions in inches for improved readability.
- `plt.title('Plot Title')` : Sets a descriptive title above the plot.
- `plt.xlabel('X Axis Label (units)')` / `plt.ylabel('Y Axis Label (units)')` : Labels the axes.
- `label='...'` inside `plt.plot()` : Assigns a label to a dataset for the legend.
- `plt.legend(loc='upper right')` : Displays the legend box (`loc` options include `'best'` , `'upper left'` , `2` , etc.).
- `plt.grid(True)` : Toggles grid lines for easier numeric inspection.
- `plt.savefig('figure.png', dpi=300)` : Exports the figure as an image file on disk.

In [5]:
```
# generate coordinate data for multiple polynomials
x = np.linspace(-5, 5, 25)

# customize figure dimensions
plt.figure(figsize=(9, 5))

# plot multiple curves with distinct markers, colors, and labels
plt.plot(x, x**2, 'ko', label='Quadratic')
plt.plot(x, x**3, 'r*', label='Cubic ')

# add titles, labels, grid, and legend
plt.title('Plot of Various Polynomials from -5.0 to 5.0')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc="upper left")
plt.show()
```



EXERCISE: Damped Oscillator Comparison

A mechanical spring-mass damper satisfies the damped harmonic motion equation:

$$y(t) = Ae^{-\gamma t}\cos(\omega t)$$

We wish to compare an underdamped oscillator with an overdamped oscillator over $t \in [0, 10]$ seconds.

Given:

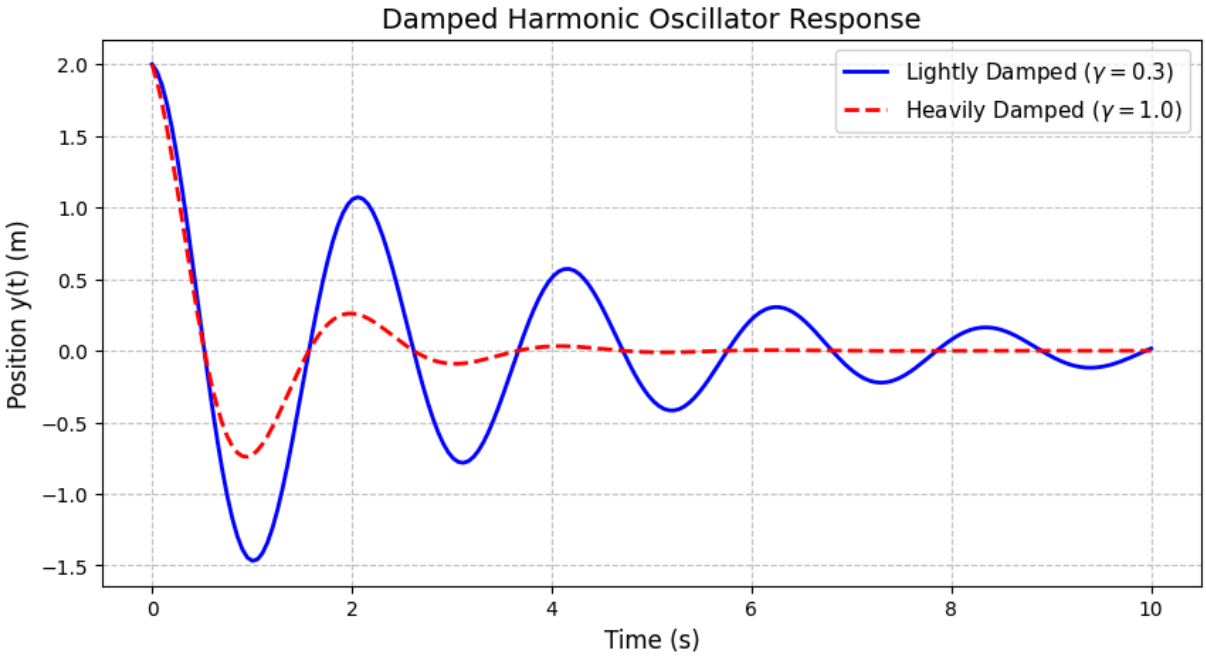
- Amplitude $A = 2.0\ \text{m}$
- Angular frequency $\omega = 3.0\ \text{rad/s}$
- Damping factor 1 (lightly damped): $\gamma_1 = 0.3\ \text{s}^{-1}$
- Damping factor 2 (heavily damped): $\gamma_2 = 1.0\ \text{s}^{-1}$

Tasks:

1. Use `np.linspace()` to generate 200 evenly spaced time points between 0 and 10 seconds.
2. Compute the positions $y_1(t)$ and $y_2(t)$.
3. Save t, $y_1(t)$, and $y_2(t)$ to a `.txt` file.
4. Create a figure with dimensions `figsize=(10, 5)`.
5. Plot $y_1(t)$ as a blue solid line (`'b-'`, linewidth 2) labeled `"Lightly Damped (\gamma = 0.3)"`.
6. Plot $y_2(t)$ as a red dashed line (`'r--'`, linewidth 2) labeled `"Heavily Damped (\gamma = 1.0)"`.

7. Add:
    - Title: `"Damped Harmonic Oscillator Response"`
    - X-axis label: `"Time (s)"`
    - Y-axis label: `"Position y(t) (m)"`
    - A legend positioned in the upper right
    - A reference grid
8. Display the plot with `plt.show()`.

In [6]:
```python
import numpy as np
import matplotlib.pyplot as plt

# time array
t = np.linspace(0, 10, 200)

# parameters
A = 2.0
omega = 3.0
gamma_1 = 0.3
gamma_2 = 1.0

# compute trajectories
y1 = A * np.exp(-gamma_1 * t) * np.cos(omega * t)
y2 = A * np.exp(-gamma_2 * t) * np.cos(omega * t)

# save data
data_matrix = np.column_stack((t, y1, y2))
np.savetxt('oscillators.txt', data_matrix, fmt='%.2f', header='Time(s) y_1(m

# create figure
plt.figure(figsize=(10, 5))

# plot curves
plt.plot(t, y1, 'b-', linewidth=2, label=r'Lightly Damped ($\gamma = 0.3$)')
plt.plot(t, y2, 'r--', linewidth=2, label=r'Heavily Damped ($\gamma = 1.0$)'

# labels, legend, grid, title
plt.title('Damped Harmonic Oscillator Response', fontsize=14)
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Position y(t) (m)', fontsize=12)
plt.legend(loc='upper right', fontsize=11)
plt.grid(True, linestyle='--', alpha=0.7)

# display figure
plt.show()
```


Damped Harmonic Oscillator Response
Lightly Damped ($\gamma = 0.3$)
Heavily Damped ($\gamma = 1.0$)
Position y(t) (m)
Time (s)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0
2
4
6
8
10

TUTORIAL 6

Welcome to Tutorial 6! Today, we're discussing least squares regression.

WHAT IS LEAST SQUARES REGRESSION?

In scientific and engineering experiments, model parameters are rarely known *a priori*. Instead, we must determine them from empirical measurements. However, real-world data inevitably contains sensor inaccuracies, background noise, and measurement errors. Least squares regression is a mathematical method used to find the best-fitting curve or model through a set of data points by minimizing the sum of the squares of the differences (residuals) between observed data and model predictions.

Physical Motivation: Hooke's Law Example

Consider an experiment to determine the stiffness k of a linear spring. According to Hooke's Law:

$$F(x) = kx$$

where:

- F is the restoring force measured in the spring (in N).
- x is the displacement from rest (in cm or m).
- k is the spring stiffness (the model parameter to be estimated).

In a laboratory, we apply known displacements x_i and measure corresponding forces F_i. Because of measurement noise, no single straight line passes through every data point. Least squares regression allows us to find the most probable stiffness k that best describes the system.

The Estimation Function and Basis Functions

In general, least squares regression seeks an estimation function $\hat{y}(x)$ that best fits a dataset of m points $(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$.

The estimation function is typically expressed as a linear combination of n basis functions $f_j(x)$:

$$\hat{y}(x) = \sum_{j=1}^{n} \alpha_j f_j(x) = \alpha_1 f_1(x) + \alpha_2 f_2(x) + \cdots + \alpha_n f_n(x)$$

Key properties:

- α_j are the unknown parameters (coefficients) to be determined.
- $f_j(x)$ are the basis functions, which must be linearly independent (i.e., none can be written as a linear combination of the others). Examples include $f(x) = 1$, $f(x) = x$, $f(x) = x^2$, or $f(x) = \sin(x)$.
- Notice that "linear" in linear regression means linear in the parameters α_j, even if the basis functions $f_j(x)$ themselves are nonlinear in x!

Evaluating this estimation function at all m data points yields an $m \times n$ system of equations:

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1)\\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2)\\ &\vdots\\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m)\end{aligned}$$

Minimizing the Total Squared Error

For each data point, the individual error (or **residual**) is defined as:

$$e_i = y_i - \hat{y}(x_i)$$

The objective is to minimize the total squared error E, which is the square of the Euclidean (L_2) norm of the residual vector $\mathbf{e} = [e_1, e_2, \ldots, e_m]^T$:

$$E = \|\mathbf{e}\|_2^2 = \sum_{i=1}^{m} e_i^2 = \sum_{i=1}^{m} \left(y_i - \hat{y}(x_i)\right)^2$$

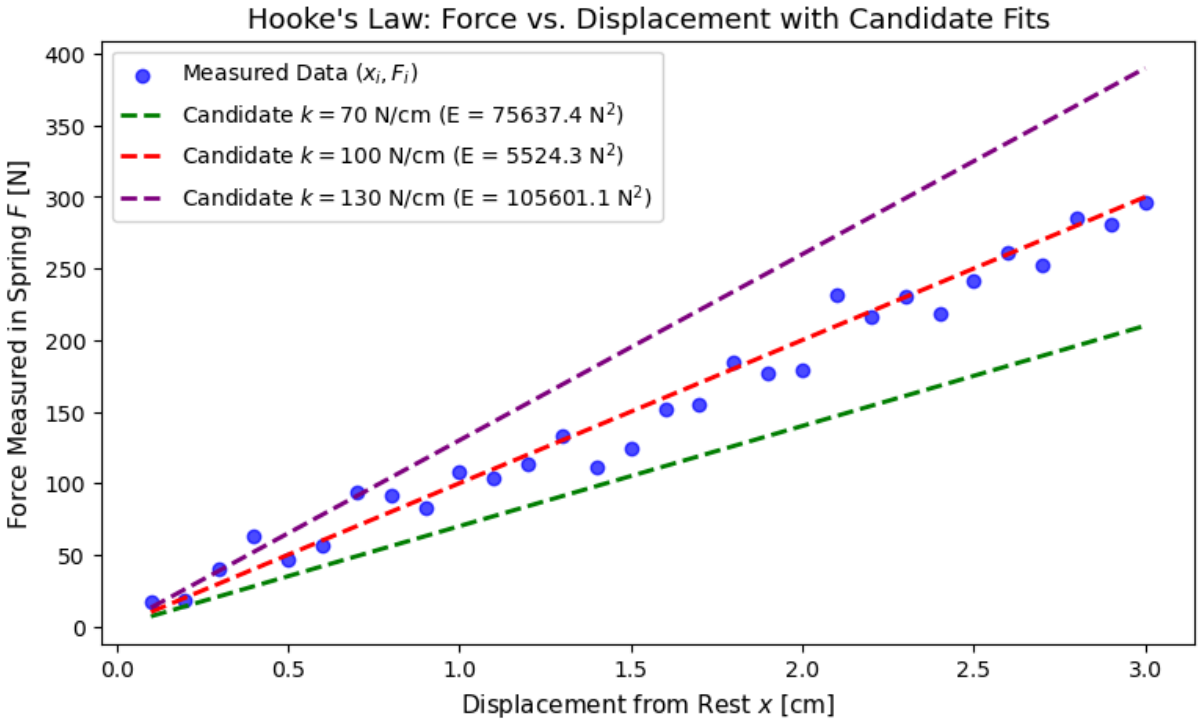
where:

- m is the total number of data points.
- y_i is the actual observed dependent variable.
- $\hat{y}(x_i)$ is the predicted value from the model at position x_i.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

# simulate noisy experimental spring measurements: F = k * x + noise
np.random.seed(42)
x_disp = np.linspace(0.1, 3.0, 30)          # displacement from rest in cm
k_true = 100.0                               # true stiffness = 100 N/cm
noise = np.random.normal(0, 15.0, len(x_disp))
F_meas = k_true * x_disp + noise             # measured force in N

# evaluate total squared error E for different candidate stiffness values
candidate_k = [70.0, 100.0, 130.0]
plt.figure(figsize=(9, 5))
plt.scatter(x_disp, F_meas, color='blue', alpha=0.7, label='Measured Data $(
```

```
colors = ['green', 'red', 'purple']
for k_cand, col in zip(candidate_k, colors):
    F_pred = k_cand * x_disp
    total_error = np.sum((F_meas - F_pred) ** 2)
    plt.plot(x_disp, F_pred, color=col, linestyle='--', linewidth=2,
             label=f'Candidate $k = {k_cand:.0f}$ N/cm (E = {total_error:.1f
    print(f"Stiffness k = {k_cand:5.1f} N/cm -> Total Squared Error E = {tot

plt.title("Hooke's Law: Force vs. Displacement with Candidate Fits", fontsiz
plt.xlabel("Displacement from Rest $x$ [cm]", fontsize=11)
plt.ylabel("Force Measured in Spring $F$ [N]", fontsize=11)
plt.legend(loc='upper left', fontsize=10)
plt.show()
```

```
Stiffness k =  70.0 N/cm -> Total Squared Error E =  75637.45 N^2
Stiffness k = 100.0 N/cm -> Total Squared Error E =   5524.27 N^2
Stiffness k = 130.0 N/cm -> Total Squared Error E = 105601.10 N^2
```



The Linear Algebra Set-Up

The linear algebra approach formulates the least squares problem as finding the optimal vector β that makes $A\beta$ as close as possible to the observation vector Y.

Let:

- Y be the $m \times 1$ column vector of observed values:

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

- A be the $m \times n$ design matrix whose columns represent the basis functions evaluated at each data point:

$$A = \begin{bmatrix} f_1(x_1) & f_2(x_1) & \dots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \dots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \dots & f_n(x_m) \end{bmatrix}$$

- β be the $n \times 1$ column vector of parameters to be estimated:

$$\beta = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix}$$

For a simple linear relationship $y = mx + b = \alpha_1 x + \alpha_2$, the basis functions are $f_1(x) = x$ and $f_2(x) = 1$. The matrix A becomes:

$$A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{bmatrix}, \quad \beta = \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix}$$

The model predictions across all data points are compactly written as:

$$\hat{Y} = A\beta$$

The total squared error in vector norm notation is:

$$E = \|Y - \hat{Y}\|_2^2 = \|Y - A\beta\|_2^2$$

The vector $A\beta$ is a linear combination of the columns of A. Therefore, $A\beta$ must lie within the column space (or range) of A, denoted $\mathrm{col}(A)$.

In an overdetermined system ($m > n$), experimental noise means Y rarely lies exactly in $\mathrm{col}(A)$. The point $\hat{Y} = A\beta$ in $\mathrm{col}(A)$ that minimizes the Euclidean distance $\|Y - A\beta\|$ is the orthogonal projection of Y onto $\mathrm{col}(A)$.

Consequently, the residual error vector $\mathbf{e} = Y - A\beta$ must be perpendicular (orthogonal) to every column of A:

$$A^T(Y - A\beta) = \mathbf{0}$$

Recall that the dot product of two vectors is given by $\mathrm{dot}(\mathbf{v}, \mathbf{w}) = \mathbf{v}^T\mathbf{w}$. Setting the dot product with all columns to zero:

$$A^TY - A^TA\beta = \mathbf{0}$$

$$A^T A\beta = A^T Y$$

This fundamental system of linear equations is known as the Normal Equations.

Provided that the matrix A has linearly independent columns (i.e., full column rank n), the $n \times n$ matrix $A^T A$ is guaranteed to be invertible. Multiplying by $(A^T A)^{-1}$ yields:

$$\beta = (A^T A)^{-1} A^T Y$$

In [2]:
```python
# assemble the design matrix A and target vector Y
# model: y = alpha_1 * x + alpha_2  (slope and intercept)
A = np.vstack([x_disp, np.ones(len(x_disp))]).T   # (m, n) = (30, 2)
Y = F_meas[:, np.newaxis]                         # (m, 1) = (30, 1)

# compute normal matrix A^T * A and vector A^T * Y
ATA = np.dot(A.T, A)          # equivalent to A.T @ A
ATY = np.dot(A.T, Y)          # equivalent to A.T @ Y

# direct least squares solution via normal equations: beta = (A^T A)^(-1) A^
beta = np.dot(np.linalg.inv(ATA), ATY)
alpha_1, alpha_2 = beta[0, 0], beta[1, 0]    # indexed like so because it's

print(f"\nEstimated Slope (alpha_1 [k]):      {alpha_1:.4f} N/cm")
print(f"Estimated Intercept (alpha_2 [F0]): {alpha_2:.4f} N")
```

```
Estimated Slope (alpha_1 [k]):      94.7290 N/cm
Estimated Intercept (alpha_2 [F0]): 5.3478 N
```

The Multivariable Calculus Approach

The multivariable calculus approach arrives at the exact same solution by treating the total squared error E as an analytical function of n scalar variables $(\alpha_1, \alpha_2, \ldots, \alpha_n)$ and finding the point where the gradient is zero.

The error function is:

$$E(\alpha_1, \alpha_2, \ldots, \alpha_n) = \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right)^2$$

where:

- m = number of data points.
- n = number of basis functions.
- α_j = unknown model parameters.
- $f_j(x_i)$ = basis functions evaluated at data point x_i.
- y_i = actual/target values.

To find the minimum of E, we calculate the partial derivative with respect to each specific parameter α_k (for $k = 1, 2, \ldots, n$) and set it equal to zero:

$$\frac{\partial E}{\partial \alpha_k} = 0$$

After some computation, we get:

$$\sum_{j=1}^{n} \alpha_j \left(\sum_{i=1}^{m} f_j(x_i) f_k(x_i) \right) = \sum_{i=1}^{m} f_k(x_i) y_i \quad \text{for each } k = 1, 2, \ldots, n$$

Consider all n equations simultaneously:

- The term $\sum_{i=1}^{m} f_j(x_i) f_k(x_i)$ is precisely the dot product of the k-th column of A with the j-th column of A, representing the (k, j)-th entry of the matrix $A^T A$.
- The term $\sum_{i=1}^{m} f_k(x_i) y_i$ is the dot product of the k-th column of A with vector Y, representing the k-th entry of $A^T Y$.

Hence, the system of n scalar equations can be written in matrix notation as:

$$A^T A \boldsymbol{\alpha} = A^T Y$$

$$\boldsymbol{\alpha} = (A^T A)^{-1} A^T Y$$

Both the linear algebra and multivariable calculus approaches yield the identical least squares solution!

Classical Linear Model ($y = mx + b$)

For a simple line with two parameters ($m = \alpha_1$ and $b = \alpha_2$), setting $\frac{\partial E}{\partial m} = 0$ and $\frac{\partial E}{\partial b} = 0$ gives us:

$$m = \frac{m \sum x_i y_i - \sum x_i \sum y_i}{m \sum x_i^2 - (\sum x_i)^2}, \qquad b = \frac{\sum y_i - m \sum x_i}{m} = \bar{y} - m\bar{x}$$

In [3]:
```
# compute slope m and intercept b using the formulas above
m_points = len(x_disp)
sum_x  = np.sum(x_disp)
sum_y  = np.sum(F_meas)
sum_xy = np.sum(x_disp * F_meas)
sum_x2 = np.sum(x_disp ** 2)

slope_calc = (m_points * sum_xy - sum_x * sum_y) / (m_points * sum_x2 - sum_
intercept_calc = (sum_y - slope_calc * sum_x) / m_points

print(f"Slope m [k]:      {slope_calc:.4f} N/cm")
print(f"Intercept b [F0]: {intercept_calc:.4f} N")
```

```
# matches what we got with the linear algebra approach!
```

```
Slope m [k]:      94.7290 N/cm
Intercept b [F0]: 5.3478 N
```

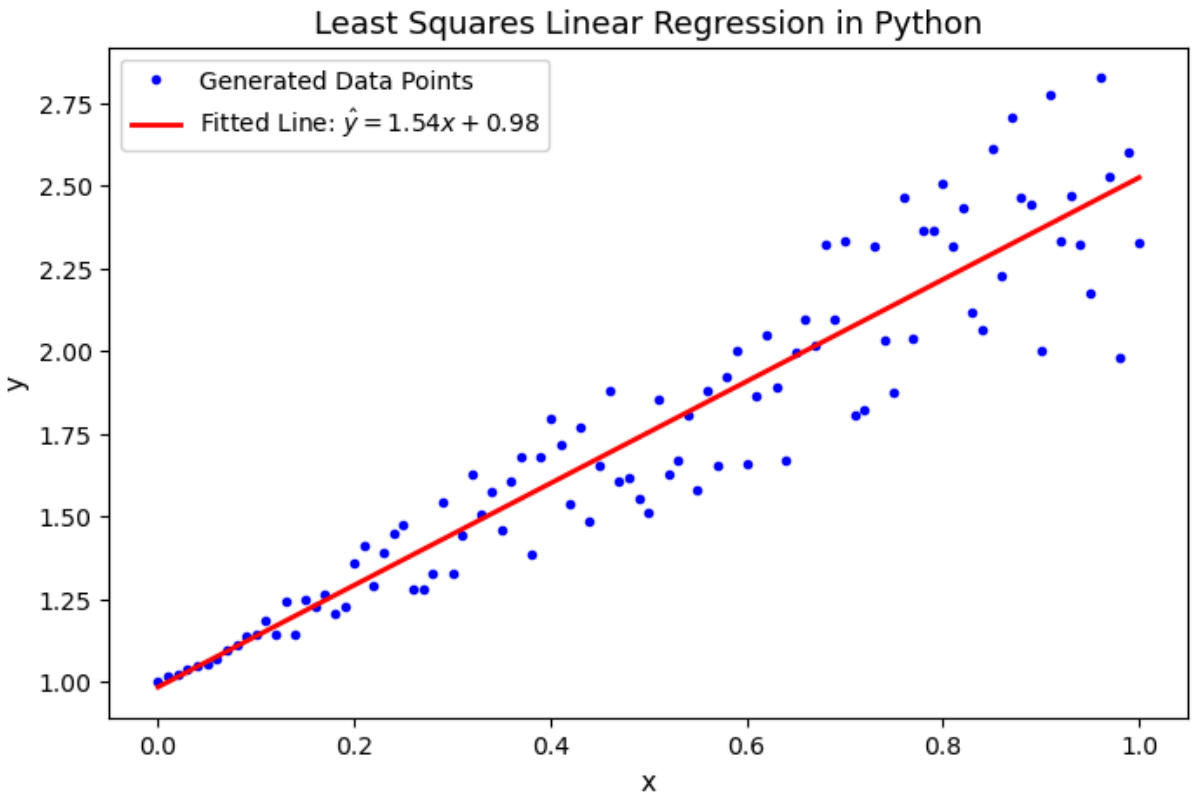
PYTHON METHODS FOR LEAST SQUARES REGRESSION

In [4]:

```
from scipy import optimize

# generate data: y = 1 + x + (x * noise)
x = np.linspace(0, 1, 101)
np.random.seed(1)
y = 1 + x + x * np.random.random(len(x))

# assemble matrix A and turn y into a column vector
A = np.vstack([x, np.ones(len(x))]).T
y_col = y[:, np.newaxis]

# Method 1: direct least squares regression using normal equations
alpha_direct = np.dot((np.dot(np.linalg.inv(np.dot(A.T, A)), A.T)), y_col)
print("Method 1 (Direct Normal Equation):")
print(alpha_direct)

# Method 2: Use numpy.linalg.lstsq
alpha_lstsq = np.linalg.lstsq(A, y_col)[0]
print("\nMethod 2 (numpy.linalg.lstsq):")
print(alpha_lstsq)

# Method 3: Use optimize.curve_fit from scipy (no need to turn y into a colu
def linear_func(x, a, b):
    return a * x + b

alpha_scipy = optimize.curve_fit(linear_func, xdata=x, ydata=y)[0]
print("\nMethod 3 (scipy.optimize.curve_fit):")
print(alpha_scipy)

# results
plt.figure(figsize=(8, 5))
plt.plot(x, y, 'b.', label='Generated Data Points')
plt.plot(x, alpha_direct[0] * x + alpha_direct[1], 'r-', linewidth=2,
         label=f'Fitted Line: $\hat{{y}} = {alpha_direct[0,0]:.2f}x + {alpha
plt.xlabel('x', fontsize=11)
plt.ylabel('y', fontsize=11)
plt.title('Least Squares Linear Regression in Python', fontsize=13)
plt.legend(loc='upper left', fontsize=10)
plt.show()
```

```
Method 1 (Direct Normal Equation):
[[1.54120263]
 [0.98370554]]

Method 2 (numpy.linalg.lstsq):
[[1.54120263]
 [0.98370554]]

Method 3 (scipy.optimize.curve_fit):
[1.54120263 0.98370554]
```

```
<>:34: SyntaxWarning: invalid escape sequence '\h'
<>:34: SyntaxWarning: invalid escape sequence '\h'
/tmp/ipykernel_123/223895980.py:34: SyntaxWarning: invalid escape sequence
'\h'
  label=f'Fitted Line: $\hat{{y}} = {alpha_direct[0,0]:.2f}x + {alpha_direct
[1,0]:.2f}$')
```



NONLINEAR REGRESSION: LOGARITHMIC TRANSFORMATIONS

Some physical relationships cannot be modeled as linear combinations of basis functions in their native form, but we can sometimes use logarithmic transformations to convert the problem into an equivalent linear form.

1. Exponential Functions

Given an exponential model:

$$y = \alpha e^{\beta x}$$

where $\alpha > 0$ is a scale factor and β is the rate constant.

Taking the natural logarithm ($\ln$) of both sides:

$$\ln(y) = \ln\left(\alpha e^{\beta x}\right) = \ln(\alpha) + \beta x$$

Let $\tilde{y} = \ln(y)$ and $c = \ln(\alpha)$:

$$\tilde{y} = \beta x + c$$

This is a straight line! We perform standard linear least squares on $(x_i, \tilde{y}_i)$, determine slope β and intercept c, then recover the original parameters:

$$\beta = \text{slope}, \qquad \alpha = e^{c}$$

2. Power Functions

Given a power-law model:

$$y = bx^{m}$$

where $b > 0$ and $x > 0$.

Taking the natural logarithm of both sides:

$$\ln(y) = \ln(bx^{m}) = m\ln(x) + \ln(b)$$

Let $\tilde{y} = \ln(y)$, $\tilde{x} = \ln(x)$, and $c = \ln(b)$:

$$\tilde{y} = m\tilde{x} + c$$

After transforming both x and y, we apply linear least squares to solve for slope m and intercept c, then recover:

$$m = \text{slope}, \qquad b = e^{c}$$

POLYNOMIAL REGRESSION AND OVERFITTING

A polynomial model of degree n is defined as:

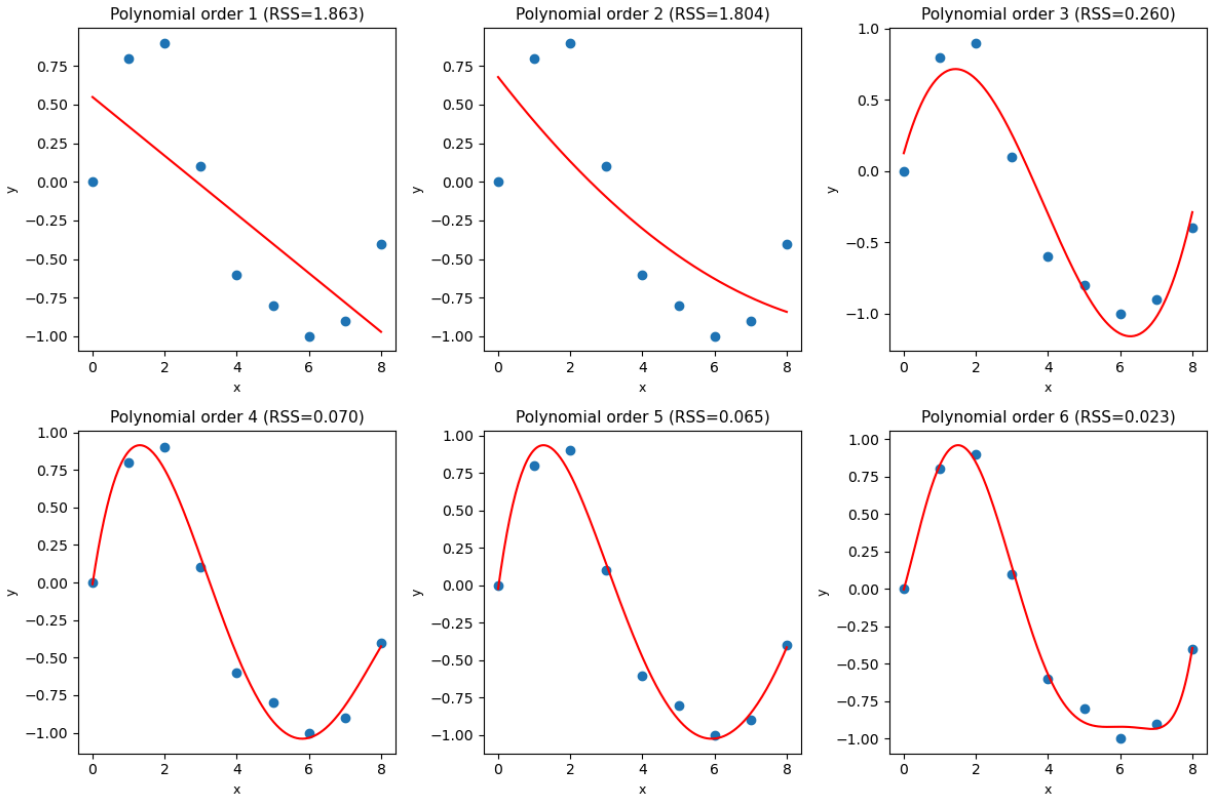
$$f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_2 x^2 + a_1 x + a_0$$

In Python, NumPy provides a dedicated function to fit polynomials: `np.polyfit(x, y, deg)`. This computes least-squares polynomial coefficients of degree `deg`.

However, we need to be careful when fitting polynomials to data.

- Underfitting: Low-order polynomials (e.g., degree 1 for curved data) fail to capture the real physical trend.
- Overfitting: Higher-order polynomials provide excess flexibility. While they may pass through nearly every training data point (achieving zero training error), they oscillate erratically between points and generalize terribly to new data.

In [5]:
```
# Dataset from lecture slides (PDF 2, Slide 5)
x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1, -0.9, -0.4])

plt.figure(figsize=(12, 8))
x_dense = np.linspace(0, 8, 200)

for i in range(1, 7):
    # get the polynomial coefficients
    y_est = np.polyfit(x_d, y_d, i)

    # evaluate goodness of fit
    y_fitted = np.polyval(y_est, x_d)   # evaluates the polynomial at given
    rss = np.sum((y_d - y_fitted) ** 2)

    plt.subplot(2, 3, i)
    plt.plot(x_d, y_d, 'o', markersize=6, label='Data')
    # evaluate the values for a smooth polynomial curve
    plt.plot(x_dense, np.polyval(y_est, x_dense), 'r-', linewidth=1.5, label
    plt.title(f'Polynomial order {i} (RSS={rss:.3f})', fontsize=11)
    plt.xlabel('x', fontsize=9)
    plt.ylabel('y', fontsize=9)

plt.tight_layout()
plt.show()

# summary table of polynomial fits
print("Polynomial Fit Summary:")
for order in range(1, 7):
    coeffs = np.polyfit(x_d, y_d, order)
    residuals = y_d - np.polyval(coeffs, x_d)
    rss = np.sum(residuals ** 2)
    r2 = 1.0 - rss / np.sum((y_d - np.mean(y_d)) ** 2)
    print(f"Order {order}: RSS = {rss:7.4f}, R^2 = {r2:.4f}")
```




```
Polynomial Fit Summary:
Order 1: RSS =  1.8629, R^2 = 0.5376
Order 2: RSS =  1.8038, R^2 = 0.5523
Order 3: RSS =  0.2595, R^2 = 0.9356
Order 4: RSS =  0.0696, R^2 = 0.9827
Order 5: RSS =  0.0654, R^2 = 0.9838
Order 6: RSS =  0.0227, R^2 = 0.9944
```

DIRECT NONLINEAR FITTING WITH SCIPY CURVE_FIT

Many scientific functions cannot be linearised (e.g., sums of exponentials $y = a_1 e^{-k_1 x} + a_2 e^{-k_2 x}$, sinusoidal functions with unknown frequency, etc.).

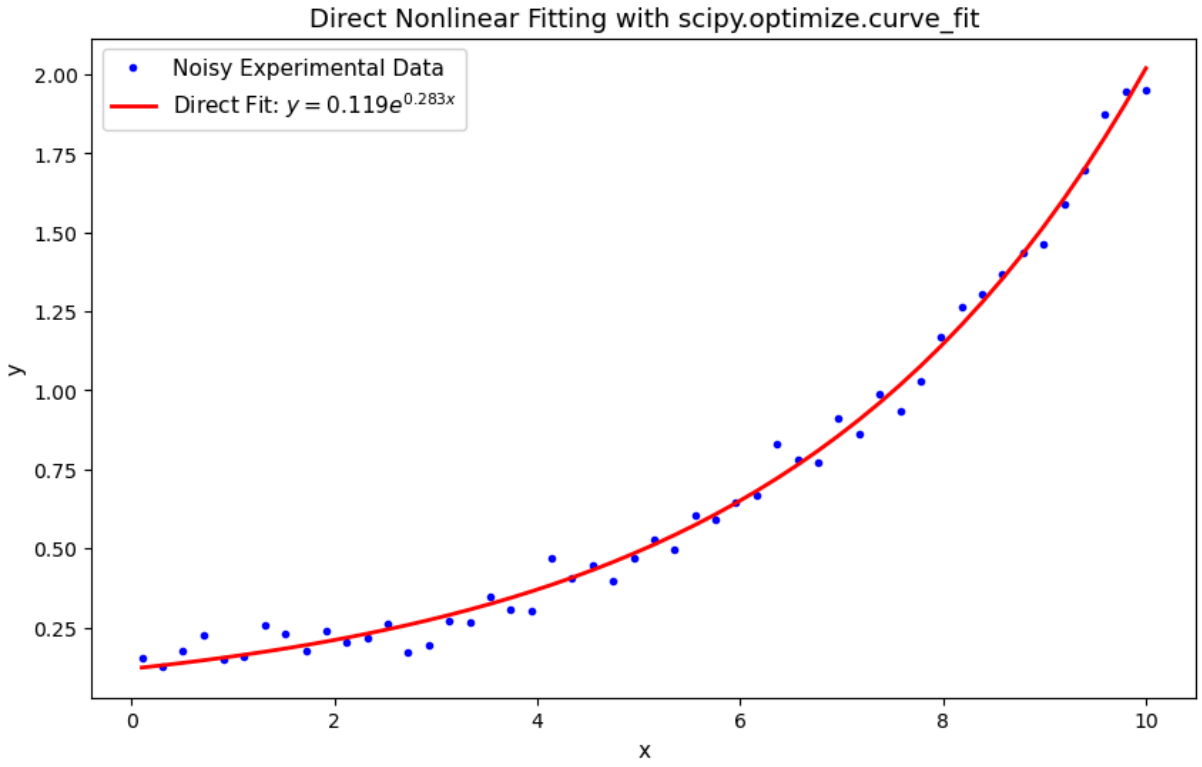
In these cases, we perform direct nonlinear regression using `scipy.optimize.curve_fit`. This uses an iterative method, and so it is a good idea to provide an initial guess (if you don't provide one, SciPy assumes an initial guess of 1 for each parameter). The function returns:

1. `popt`: An array of optimal parameter values.
2. `pcov`: The estimated covariance matrix of `popt`. The diagonals $\sqrt{\mathrm{pcov}[j,j]}$ represent the standard errors (uncertainties) of the estimated parameters.

In [6]:
```python
# generate exponential data with noise
x = np.linspace(0.1, 10, 50)
np.random.seed(42)
alpha_true, beta_true = 0.124, 0.28
y = alpha_true * np.exp(beta_true * x) + np.random.normal(0, 0.05, len(x))
```

```python
# define the function form
def func(x, a, b):
    y = a * np.exp(b * x)
    return y

# direct nonlinear fitting with curve_fit
popt, pcov = optimize.curve_fit(func, xdata=x, ydata=y, p0=[0.1, 0.2])
alpha_fit, beta_fit = popt[0], popt[1]
alpha_err, beta_err = np.sqrt(pcov[0, 0]), np.sqrt(pcov[1, 1])

print(f"Fitted parameters: alpha = {alpha_fit:.6f} +/- {alpha_err:.6f}")
print(f"                   beta  = {beta_fit:.6f} +/- {beta_err:.6f}")

# plot the data and fit
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'b.', label='Noisy Experimental Data')
plt.plot(x, func(x, alpha_fit, beta_fit), 'r-', linewidth=2,
         label=f'Direct Fit: $y = {alpha_fit:.3f} e^{{{beta_fit:.3f}x}}$')
plt.xlabel('x', fontsize=11)
plt.ylabel('y', fontsize=11)
plt.title('Direct Nonlinear Fitting with scipy.optimize.curve_fit', fontsize
plt.legend(loc='upper left', fontsize=11)
plt.show()
```

```
Fitted parameters: alpha = 0.119231 +/- 0.004634
                   beta  = 0.282969 +/- 0.004556
```



TUTORIAL 7

Welcome to Tutorial 7! Today, we will cover interpolation, root finding, numerical differentiation, and numerical integration.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

INTERPOLATION

Interpolation is the process of estimating unknown values of a function that fall within the range of known discrete data points.

Given a dataset of n independent coordinates x_i and corresponding dependent values y_i:

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$$

The goal is to find an estimation function $\hat{f}(x)$ such that:

$$\hat{f}(x_i) = y_i \quad \text{for every point in the dataset}$$

Interpolation is different from regression because it ensures the function passes through every data point. Regression assumes an underlying parameterized model (such as a straight line $y = mx + b$) and minimizes the overall discrepancy (residuals), meaning the fitted line rarely passes through all data points.

Interpolation is suitable when data points are highly reliable, low-noise measurements representing a continuous physical phenomenon (e.g. tracking vehicle trajectory over time).

LINEAR INTERPOLATION

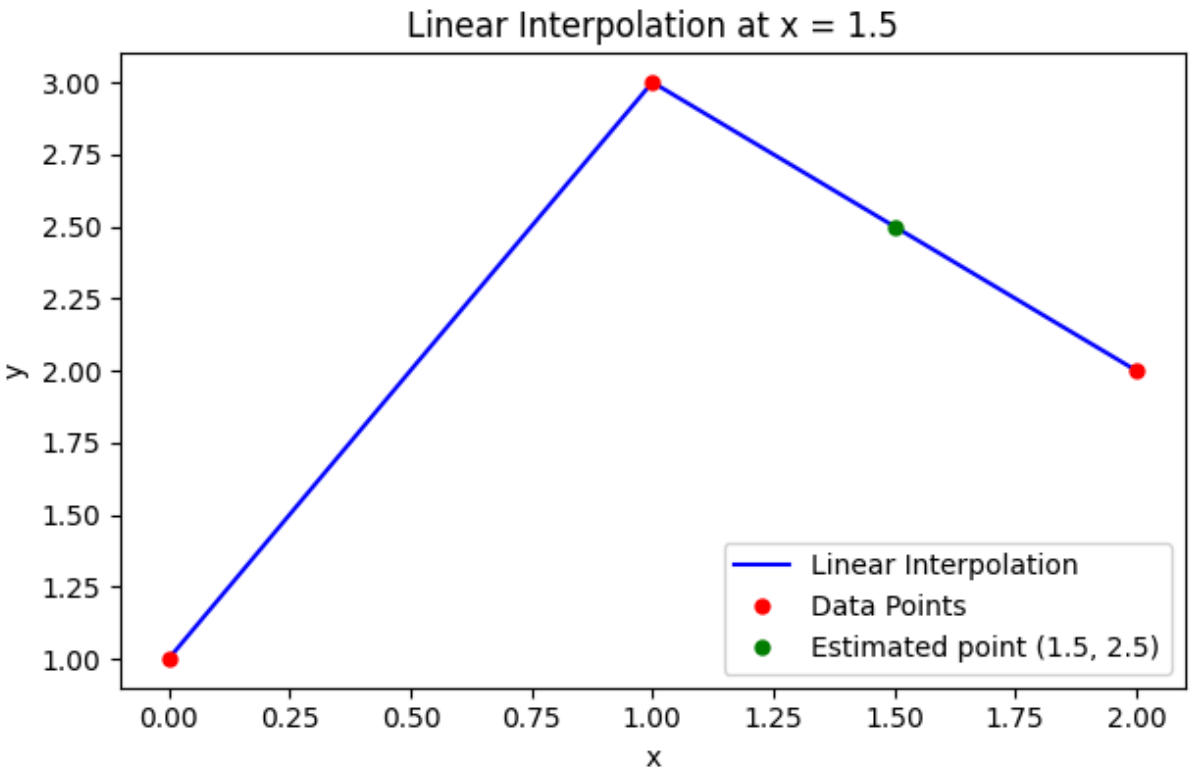
The simplest approach is linear interpolation, which assumes that any intermediate point lies on the straight line segment connecting the two nearest known data points on either side.

For any x lying in the interval $x_i \leq x \leq x_{i+1}$, the equation of the line connecting the two points is:

$$\hat{f}(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

In [2]:
```python
from scipy.interpolate import interp1d

# known discrete data points
x_data = np.array([0, 1, 2])
y_data = np.array([1, 3, 2])

# construct 1D linear interpolator
f_linear = interp1d(x_data, y_data, kind='linear')   # returns a function

# evaluate at target point x = 1.5
x_target = 1.5
y_target = float(f_linear(x_target))
print(f"Linear Interpolation at x = {x_target}: y_hat = {y_target:.4f}")

# plotting the linear interpolation
x_dense = np.linspace(0, 2, 100)
plt.figure(figsize=(7, 4.2))
plt.plot(x_dense, f_linear(x_dense), 'b-', label='Linear Interpolation')
plt.plot(x_data, y_data, 'ro', markersize=5, label='Data Points')
plt.plot(x_target, y_target, 'go', markersize=5, label=f'Estimated point ({x
plt.title(f'Linear Interpolation at x = {x_target}', fontsize=12)
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

```
Linear Interpolation at x = 1.5: y_hat = 2.5000
```



CUBIC SPLINE INTERPOLATION

A major limitation of linear interpolation is that the resulting curve has sharp, non-differentiable corners at each data point (C^0 continuity only). In physics and engineering, mechanical motion requires continuous velocities and continuous accelerations.

Cubic spline interpolation constructs a smooth curve through points using piecewise cubic polynomials:

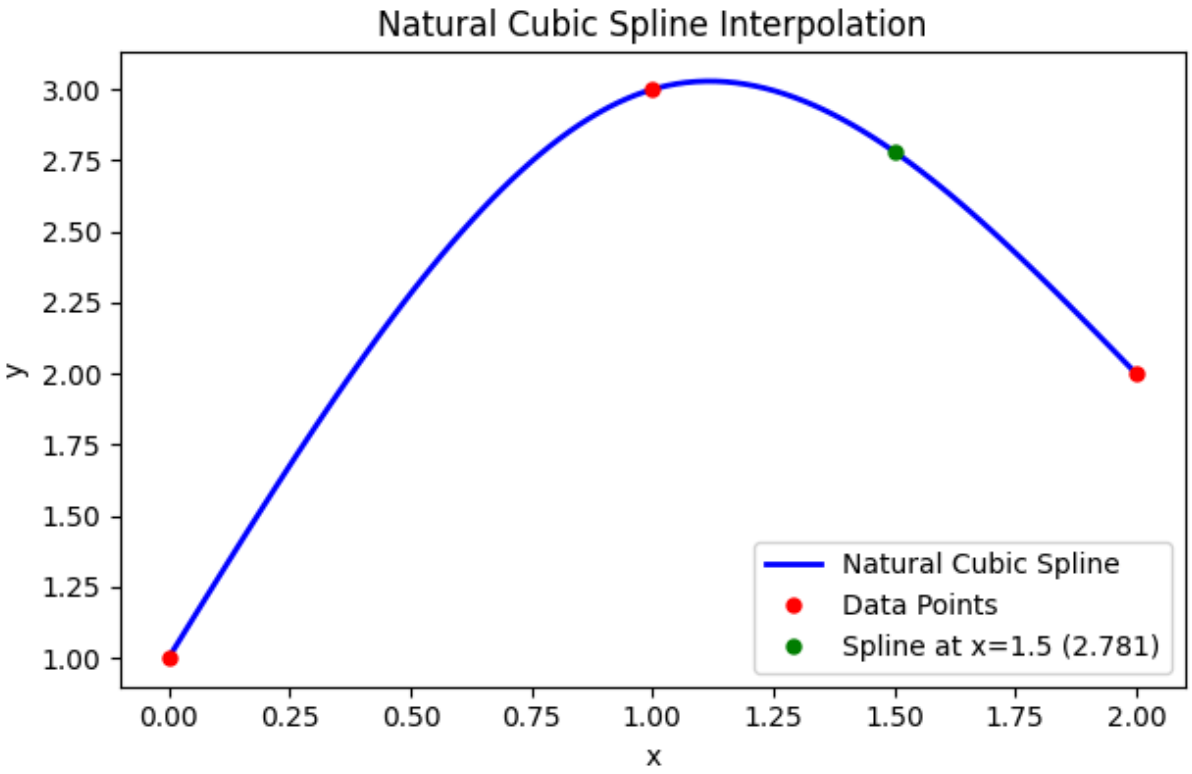
$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i \quad \text{for } x \in [x_i, x_{i+1}]$$

For n data points, there are $n-1$ intervals, requiring $4(n-1)$ unknown coefficients. These are determined uniquely by enforcing:

1. each spline passes through the given data points,
2. the first and second derivatives are continuous at every data point,
3. boundary conditions (default is to set second derivatives are outer endpoints to 0, but can be modified for SciPy's `CubicSpline` function).

In [3]:
```
from scipy.interpolate import CubicSpline

# same dataset as before
x_data = np.array([0, 1, 2])
y_data = np.array([1, 3, 2])

# fit natural cubic spline (bc_type='natural' enforces S''(x) = 0 at endpoir
f_spline = CubicSpline(x_data, y_data, bc_type='natural')

# evaluate at dense grid and target x = 1.5
x_dense = np.linspace(0, 2, 100)
y_dense = f_spline(x_dense)
y_target_spline = float(f_spline(x_target))

print(f"Cubic Spline at x = {x_target}: y_hat = {y_target_spline:.5f}")
print("Notice how Cubic Spline produces a smooth, continuous trajectory rath

plt.figure(figsize=(7, 4.2))
plt.plot(x_dense, y_dense, 'b-', label='Natural Cubic Spline', linewidth=2)
plt.plot(x_data, y_data, 'ro', markersize=5, label='Data Points')
plt.plot(x_target, y_target_spline, 'go', markersize=5, label=f'Spline at x=
plt.title('Natural Cubic Spline Interpolation', fontsize=12)
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

```
Cubic Spline at x = 1.5: y_hat = 2.78125
Notice how Cubic Spline produces a smooth, continuous trajectory rather than
sharp joints.
```



LAGRANGE & NEWTON'S POLYNOMIAL INTERPOLATION

Instead of piecewise polynomials, we can find a **single global polynomial** of degree $n-1$ that passes through all n data points.

Lagrange Polynomial Interpolation

The Lagrange polynomial expresses $L(x)$ as a linear combination of basis polynomials $P_i(x)$:

$$L(x) = \sum_{i=1}^{n} y_i P_i(x)$$

where each basis polynomial $P_i(x)$ is defined as:

$$P_i(x) = \prod_{j=1, j\neq i}^{n} \frac{x - x_j}{x_i - x_j}$$

Each basis polynomial has the property:

$$P_i(x_j) = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Newton's Polynomial & Divided Differences

While mathematically identical to Lagrange's polynomial, Newton's representation is far more computationally efficient because adding a new data point does not require recalculating the existing polynomial coefficients.

The Newton polynomial of order $n-1$ is written:

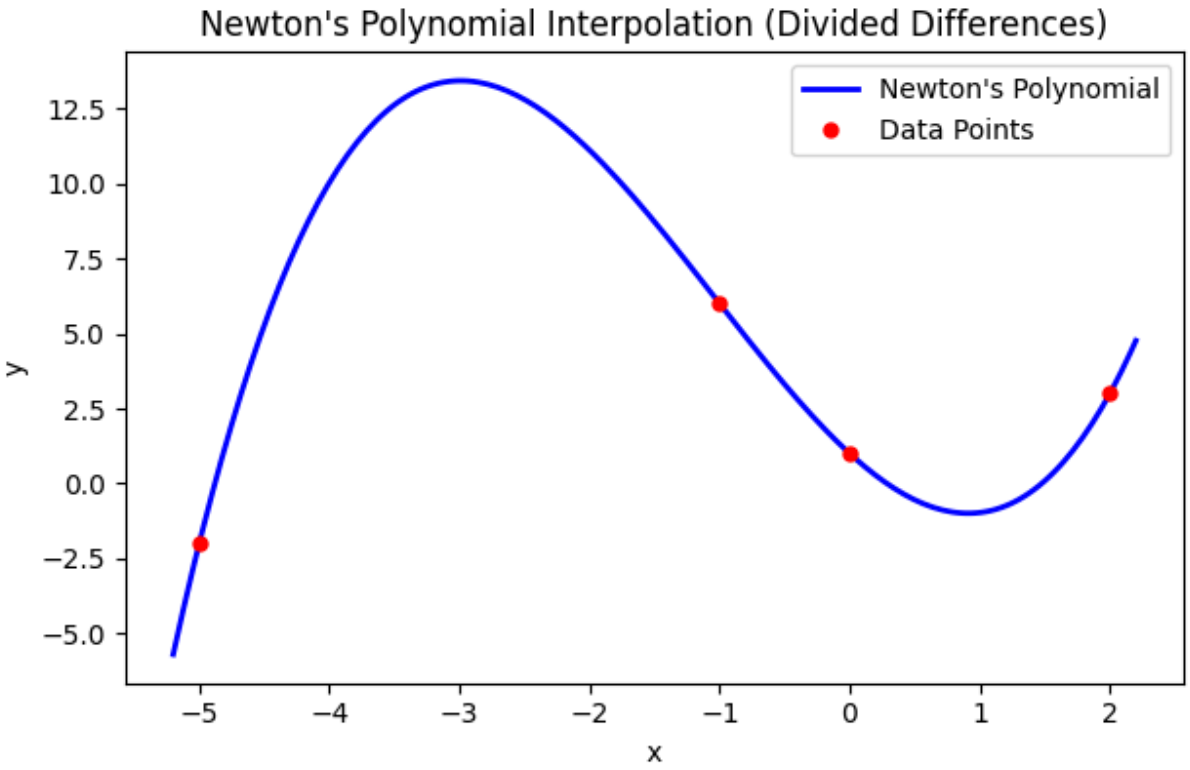
$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_n$$

The coefficients a_i are calculated using the divided differences table:

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}, \qquad f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

In [4]:
```python
# function to calculate divided differences
def divided_diff(x, y):
    """
    Calculates the divided differences table for coordinate vectors x and y.
    Returns the full table matrix where the first row contains coefficients
    """
    n = len(y)
    coef = np.zeros([n, n])
    coef[:, 0] = y  # First column is y

    for j in range(1, n):
        for i in range(n - j):
            coef[i][j] = (coef[i + 1][j - 1] - coef[i][j - 1]) / (x[i + j] -

    return coef

def newton_poly(coef, x_data, x):
    """
    Evaluates the Newton polynomial at given point.
    """
    n = len(x_data) - 1
    p = coef[n]
    for k in range(1, n + 1):
        p = coef[n - k] + (x - x_data[n - k]) * p
    return p

# example dataset from lecture slides
x_pts = np.array([-5, -1, 0, 2])
y_pts = np.array([-2, 6, 1, 3])

# calculate divided difference table and extract top row
table = divided_diff(x_pts, y_pts)
a_s = table[0, :]

# evaluate the fit
x_eval = np.linspace(-5.2, 2.2, 200)
y_eval = newton_poly(a_s, x_pts, x_eval)

plt.figure(figsize=(7, 4.2))
plt.plot(x_eval, y_eval, 'b-', label="Newton's Polynomial", linewidth=2)
plt.plot(x_pts, y_pts, 'ro', markersize=5, label='Data Points')
```

```
plt.title("Newton's Polynomial Interpolation (Divided Differences)", fontsiz
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```



ROOT FINDING

A root (or zero) of a function $f(x)$ is any value x_r such that:

$$f(x_r) = 0$$

While simple algebraic equations (like $x^2 - 9 = 0 \implies x = \pm 3$) can be solved analytically, transcendental functions like $f(x) = \cos(x) - x$ have no closed-form algebraic solution and require numerical iterative methods.

Because digital computers use finite floating-point arithmetic, an algorithm almost never lands exactly on $f(x) = 0.000000000\ldots$ Instead, an iterative method terminates when the error is smaller than a specified tolerance (`tol`).

Two common error metrics are used:

1. Function residual: $|f(x_k)| < \text{tol}$ (measures how close the output is to zero).
2. Step size: $|x_{k+1} - x_k| < \text{tol}$ (measures when successive guesses stop changing).

An inbuilt function for root finding is SciPy's `fsolve` . It accepts the function and an initial guess for the root as parameters; you can also specify the tolerance and the maximum number of iterations, but the default values are 1.49012^{-08} and 400

respectively. `fsolve` can also be used for a system of equations; simply pass arrays of functions and initial guesses. Additionally, you can pass an array of initial guesses for just one function to find multiple roots.

In [5]:
```python
from scipy.optimize import fsolve

# successful root finding on transcendental equation cos(x) - x = 0
f_cos = lambda x: np.cos(x) - x    # use lambda function definition
root_cos = fsolve(f_cos, x0=-2)
print("--- Case 1: Success (cos(x) - x = 0) ---")
print(f"Converged root: r = {root_cos[0]:.8f}")
print(f"Residual f(r):  {f_cos(root_cos)[0]:.2e}")

# failure case on f(x) = 1/x (no root exists)
f_inv = lambda x: 1.0 / x
r_inv, infodict, ier, mesg = fsolve(f_inv, x0=-2, full_output=True)
print("\n--- Case 2: Failure (1/x = 0) ---")
print(f"Result value:   {r_inv[0]:.4e}")
print(f"Residual f(r):  {f_inv(r_inv)[0]:.4e}")
print(f"Function calls: {infodict['nfev']}")
print(f"Exit message:   {mesg}")


# finding multiple roots for a function
# f(x) = (x^2 - 1)(x - 100) -> roots are -1, 1, and 100
f_cubic = lambda x: x**3 - 100 * x**2 - x + 100
guesses = [-5, 2, 80]
found_roots = [float(fsolve(f_cubic, g)[0]) for g in guesses]
print("\n--- Case 3: Multiple roots ((x^2 - 1)(x - 100) = 0) ---")
print(f"Roots of x^3 - 100x^2 - x + 100 with initial guesses {guesses}:")
print(f"Found roots: {found_roots}")
```

```
--- Case 1: Success (cos(x) - x = 0) ---
Converged root: r = 0.73908513
Residual f(r):  0.00e+00

--- Case 2: Failure (1/x = 0) ---
Result value:   -3.5205e+83
Residual f(r):  -2.8405e-84
Function calls: 402
Exit message:   The number of calls to function has reached maxfev = 400.

--- Case 3: Multiple roots ((x^2 - 1)(x - 100) = 0) ---
Roots of x^3 - 100x^2 - x + 100 with initial guesses [-5, 2, 80]:
Found roots: [-1.0000000000000022, 1.0, 100.00000000000001]
```

THE BISECTION METHOD

The bisection method of root finding is a bracketing method based on the Intermediate Value Theorem: if $f(x)$ is continuous on an interval $[a, b]$ and $f(a) \cdot f(b) < 0$ (opposite signs), then there must exist at least one root $c \in [a, b]$ such that $f(c) = 0$.

The bisection method algorithm:

1. Check that $f(a)$ and $f(b)$ have opposite signs ($f(a) \cdot f(b) < 0$).
2. Compute the midpoint: $m = \frac{a+b}{2}$.
3. If $|f(m)| < \text{tol}$, m is the accepted root.
4. If $\text{sign}(f(a)) == \text{sign}(f(m))$, the root lies in $[m, b]$; update $a \leftarrow m$. Otherwise, the root lies in $[a, m]$; update $b \leftarrow m$.
5. Repeat until interval length or residual is below tolerance.

The bisection method is very reliable: it is guaranteed to converge if initial signs differ. However, it converges linearly, so it can be slower compared to other methods.

In [6]:
```python
# function to implement the bisection method
def my_bisection(f, a, b, tol):
    """
    Approximates a root of f bounded by a and b to within tolerance tol.
    """
    # raise exception if signs at interval ends are not different
    if np.sign(f(a)) == np.sign(f(b)):
        raise ValueError(f"The scalars a={a} and b={b} do not bound a root (

    steps = 0
    while True:
        steps += 1
        m = (a + b) / 2.0     # find midpoint of current interval

        if abs(f(m)) < tol or (b - a) / 2.0 < tol:
            return m, steps   # return root if within tolerance

        if np.sign(f(a)) == np.sign(f(m)):
            a = m             # update boundary a if sign of midpoint matche
        else:
            b = m             # else update boundary b

# example from slides
f_sqrt2 = lambda x: x**2 - 2

for tol_val in [0.1, 0.01, 1e-6]:
    root_val, iterations = my_bisection(f_sqrt2, 0, 2, tol_val)
    print(f"Tolerance = {tol_val:7g} -> Root = {root_val:10.8f} in {iteratic
```

```
Tolerance =     0.1 -> Root = 1.43750000 in  5 iterations (Exact: 1.4142135
6)
Tolerance =    0.01 -> Root = 1.41406250 in  8 iterations (Exact: 1.4142135
6)
Tolerance =   1e-06 -> Root = 1.41421413 in 21 iterations (Exact: 1.4142135
6)
```

NEWTON-RAPHSON METHOD & MULTIPLE ROOTS

The Newton-Raphson method is an iterative method that uses the local tangent line (first-order Taylor series approximation) to project the next guess:

$$f(x) \approx f(x_0) + f'(x_0)(x_1 - x_0) = 0$$

Solving for x_1 yields the Newton step:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Near a root, Newton-Raphson exhibits quadratic convergence (the number of accurate decimal places roughly doubles with each iteration!) which makes it more efficient than the bisection method. However, it is very sensitive to the initial guess (may oscillate or diverge if too far from the root or converge to a different root), and can blow up/fail if $f'(x)$ approaches 0.

In [7]:
```python
# Newton-Raphson Method vs. SciPy fsolve
def my_newton(f, df, x0, tol, max_iter=100):
    """
    Iterative Newton-Raphson method for root finding.
    """
    x = x0
    for i in range(1, max_iter + 1):
        fx = f(x)    # evaluate function value at x
        if abs(fx) < tol:
            return x, i
        dfx = df(x)  # evaluate derivative value at x
        if abs(dfx) < 1e-12:
            raise ZeroDivisionError("Derivative near zero. Method failed.")
        x = x - fx / dfx
    return x, max_iter

# test on f(x) = x^2 - 2
f = lambda x: x**2 - 2
df = lambda x: 2 * x

r_newton, iters = my_newton(f, df, x0=1.5, tol=1e-6)
print(f"Newton-Raphson Root: {r_newton:.12f} in only {iters} iterations!")
print(f"Bisection required 21 iterations for the same 1e-6 precision.\n")
```

```
Newton-Raphson Root: 1.414213562375 in only 4 iterations!
Bisection required 21 iterations for the same 1e-6 precision.
```

NUMERICAL DIFFERENTIATION

THE NUMERICAL GRID & FINITE DIFFERENCE FORMULATIONS

In computational science, continuous functions are frequently represented on a numerical grid—a discrete set of evenly spaced points over a domain:

$$x_j = x_0 + j \cdot h, \quad j = 0, 1, 2, \ldots, N$$

where h is the uniform step size (grid spacing).

The fundamental definition of the derivative from calculus is:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

On a discrete grid where h cannot be taken to the infinitesimal limit, we use finite difference approximations based on neighboring grid values:

1. Forward Difference: estimating the slope at a point x_j as the slope between x_j and x_{j+1}

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_j)}{h}$$

2. Backward Difference: estimating the slope at a point x_j as the slope between x_{j-1} and x_j

$$f'(x_j) \approx \frac{f(x_j) - f(x_{j-1})}{h}$$

3. Central Difference: estimating the slope at a point x_j as the slope between x_{j-1} and x_{j+1}

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_{j-1})}{2h}$$

Taylor Series Derivation & Accuracy

To understand why central difference is superior, we can derive the error terms using Taylor series expansions around x_j:

$$f(x_j + h) = f(x_j) + hf'(x_j) + \frac{h^2}{2!}f''(x_j) + \frac{h^3}{3!}f'''(x_j) + \mathcal{O}(h^4) \quad \text{--- (Eq. 1)}$$

$$f(x_j - h) = f(x_j) - hf'(x_j) + \frac{h^2}{2!}f''(x_j) - \frac{h^3}{3!}f'''(x_j) + \mathcal{O}(h^4) \quad \text{--- (Eq. 2)}$$

Forward Difference Accuracy:

Rearranging Eq. 1 for $f'(x_j)$:

$$f'(x_j) = \frac{f(x_j + h) - f(x_j)}{h} - \frac{h}{2}f''(x_j) - \cdots = \frac{f(x_{j+1}) - f(x_j)}{h} + \mathcal{O}(h)$$

- The leading truncation error is proportional to h.
- First-order accurate $\mathcal{O}(h)$: halving h reduces the error by a factor of 2.

We get the same stucture for Backward Difference.

Central Difference Accuracy:

Subtracting Eq. 2 from Eq. 1:

$$f(x_j + h) - f(x_j - h) = 2hf'(x_j) + \frac{2h^3}{6} f'''(x_j) + \mathcal{O}(h^5)$$

Dividing by $2h$:

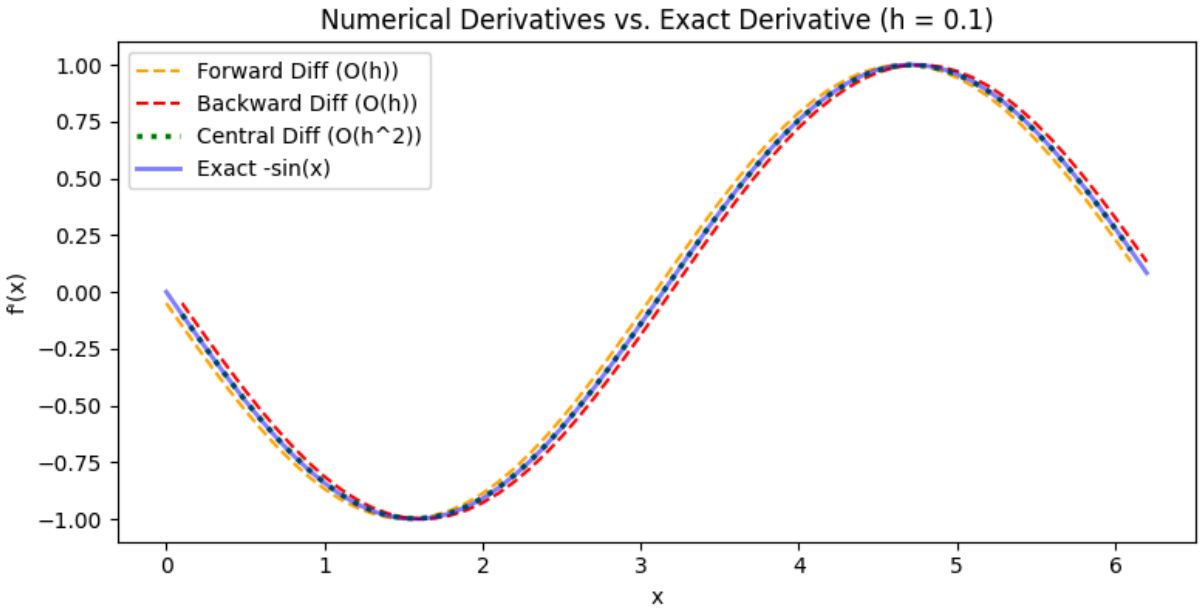
$$f'(x_j) = \frac{f(x_{j+1}) - f(x_{j-1})}{2h} - \frac{h^2}{6} f'''(x_j) - \cdots = \frac{f(x_{j+1}) - f(x_{j-1})}{2h} + \mathcal{O}(h^2)$$

- Second-order accurate $\mathcal{O}(h^2)$: halving h reduces the error by a factor of 4.

In [8]:

```
# comparing forward and central differences
h = 0.1
x_grid = np.arange(0, 2 * np.pi, h)
y_grid = np.cos(x_grid)

# forward difference: dy/dx approx (y[j+1] - y[j]) / h
forward_diff = np.diff(y_grid) / h       # np.diff finds differences betwee
x_fwd = x_grid[:-1]

# backward difference: dy/dx approx (y[j] - y[j-1]) / h
backward_diff = np.diff(y_grid) / h
x_bwd = x_grid[1:]

# central difference: dy/dx approx (y[j+1] - y[j-1]) / (2*h)
central_diff = (y_grid[2:] - y_grid[:-2]) / (2 * h)
x_cent = x_grid[1:-1]

# exact analytical derivative: d/dx(cos(x)) = -sin(x)
exact_fwd = -np.sin(x_fwd)
exact_bwd = -np.sin(x_bwd)
exact_cent = -np.sin(x_cent)

# errors
err_fwd = np.max(np.abs(exact_fwd - forward_diff))
err_bwd = np.max(np.abs(exact_bwd - backward_diff))
err_cent = np.max(np.abs(exact_cent - central_diff))

print(f"Step size h = {h}")
print(f"Max Error (Forward Difference, O(h)):    {err_fwd:.6f}")
print(f"Max Error (Backward Difference, O(h)):   {err_bwd:.6f}")
print(f"Max Error (Central Difference, O(h^2)):  {err_cent:.6f}")
print(f"Central Difference is ~{err_fwd / err_cent:.1f}x more accurate!")

# plot
plt.figure(figsize=(9, 4.2))
plt.plot(x_fwd, forward_diff, '--', label='Forward Diff (O(h))', color='orar
plt.plot(x_bwd, backward_diff, '--', label='Backward Diff (O(h))', color='re
plt.plot(x_cent, central_diff, ':', label='Central Diff (O(h^2))', color='gr
plt.plot(x_grid, -np.sin(x_grid), 'b-', label='Exact -sin(x)', alpha=0.5, li
plt.title('Numerical Derivatives vs. Exact Derivative (h = 0.1)', fontsize=1
plt.xlabel('x')
plt.ylabel("f'(x)")
plt.legend()
plt.show()
```

```
Step size h = 0.1
Max Error (Forward Difference, O(h)):    0.049984
Max Error (Backward Difference, O(h)):   0.049970
Max Error (Central Difference, O(h^2)):  0.001666
Central Difference is ~30.0x more accurate!
```



ERROR SCALING

To verify the theoretical orders of accuracy $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$, we study how the maximum absolute error scales as we systematically decrease h across several orders of magnitude:

$$h = 2^{-1}, 2^{-2}, \ldots, 2^{-18}$$

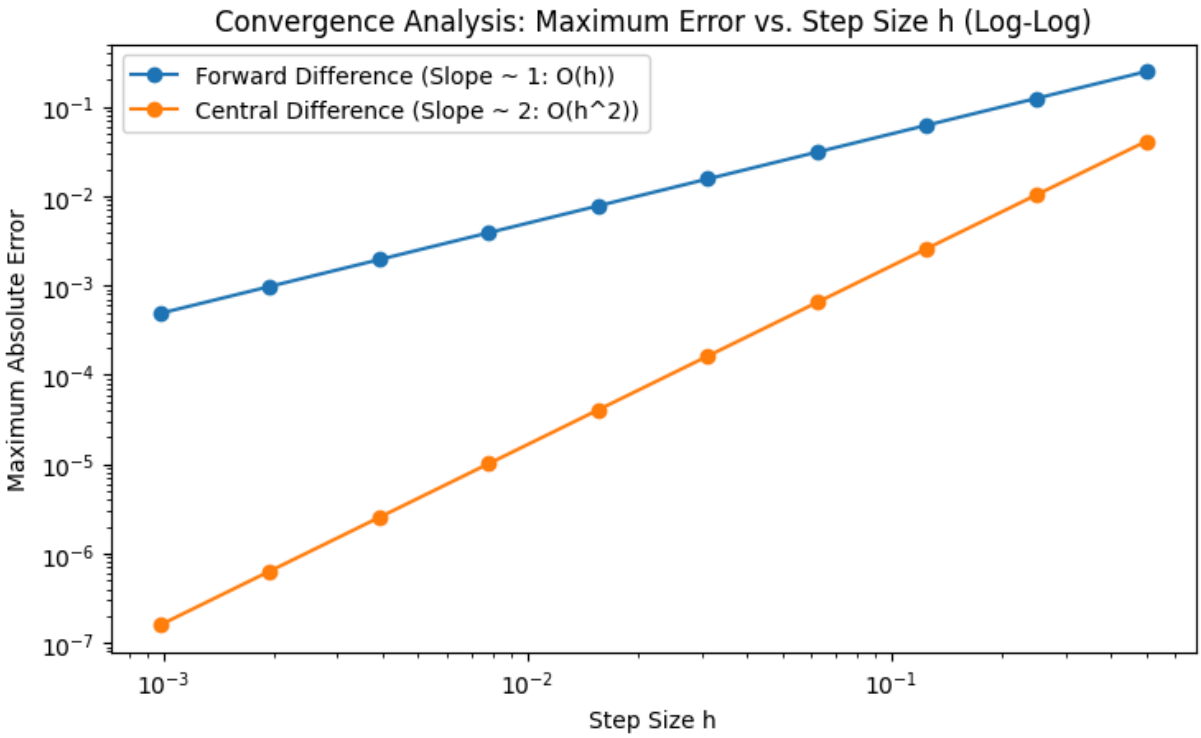
On a log-log plot:

$$\log(\text{Error}) \approx \log(C \cdot h^p) = p \log(h) + \log(C)$$

- The slope of the line reveals the convergence order p:
  - Forward difference: slope = 1
  - Central difference: slope = 2

In [9]:
```python
# log-log error analysis over diminishing step sizes h
iterations = 10
h_val = 1.0

step_sizes = []
errors_fwd = []
errors_cent = []

for i in range(iterations):
    h_val /= 2.0
    step_sizes.append(h_val)
```

```
    x = np.arange(0, 2 * np.pi, h_val)
    y = np.cos(x)

    # forward difference
    fwd = np.diff(y) / h_val
    exact_f = -np.sin(x[:-1])
    errors_fwd.append(np.max(np.abs(exact_f - fwd)))

    # central difference
    cent = (y[2:] - y[:-2]) / (2 * h_val)
    exact_c = -np.sin(x[1:-1])
    errors_cent.append(np.max(np.abs(exact_c - cent)))

# plot
plt.figure(figsize=(8.5, 4.8))
plt.loglog(step_sizes, errors_fwd, 'o-', label='Forward Difference (Slope ~
plt.loglog(step_sizes, errors_cent, 'o-', label='Central Difference (Slope ~
plt.title('Convergence Analysis: Maximum Error vs. Step Size h (Log-Log)', f
plt.xlabel('Step Size h')
plt.ylabel('Maximum Absolute Error')
plt.legend()
plt.show()
```



APPROXIMATING HIGHER-ORDER DERIVATIVES

Higher-order derivatives (such as second derivatives $f''(x)$) are fundamental in physics and engineering. We can derive a numerical differentiation approximation using Taylor series again.

Adding the forward and backward expansions (Eq. 1 and Eq. 2):

$$f(x_j + h) + f(x_j - h) = 2f(x_j) + h^2 f''(x_j) + \frac{h^4}{12} f^{(4)}(x_j) + \mathcal{O}(h^6)$$

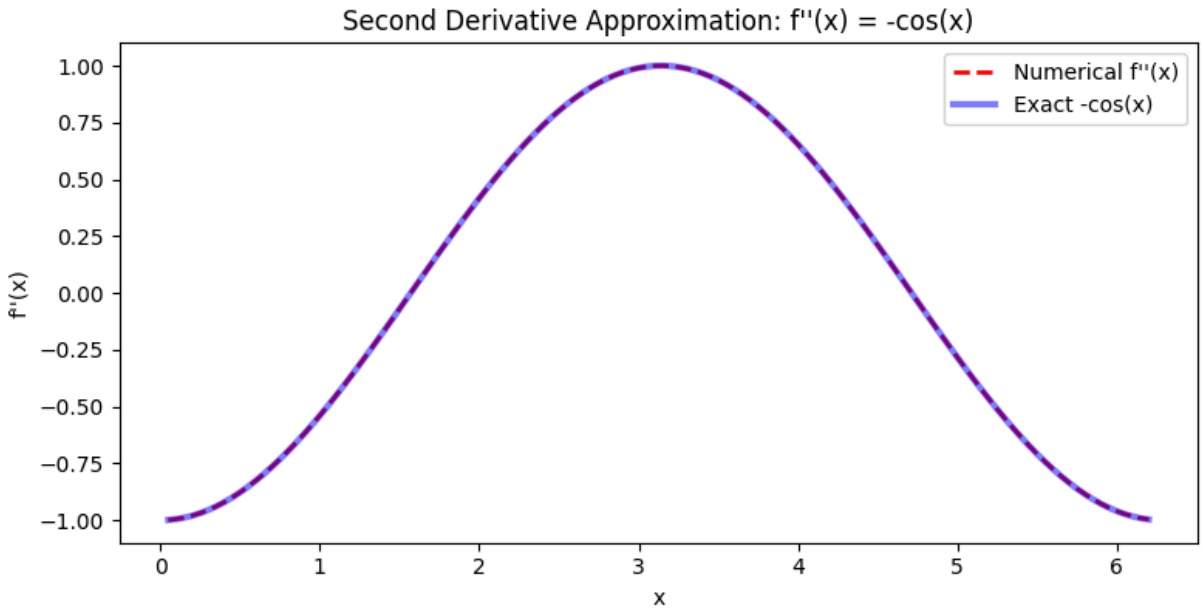
Rearranging for the second derivative $f''(x_j)$:

$$f''(x_j) = \frac{f(x_{j+1}) - 2f(x_j) + f(x_{j-1})}{h^2} + \mathcal{O}(h^2)$$

This is the standard second-order central difference formula for the second derivative, requiring three points (x_{j-1}, x_j, x_{j+1}) with truncation error $\mathcal{O}(h^2)$.

In [10]:
```
# second derivative implementation
h = 0.05
x = np.arange(0, 2 * np.pi, h)
y = np.cos(x)

# second derivative approximation: (y[j+1] - 2*y[j] + y[j-1]) / h^2
second_diff = (y[2:] - 2 * y[1:-1] + y[:-2]) / (h**2)
x_interior = x[1:-1]

# exact second derivative: d^2/dx^2(cos(x)) = -cos(x)
exact_second = -np.cos(x_interior)

max_second_err = np.max(np.abs(exact_second - second_diff))
print(f"Step size h = {h}")
print(f"Max Error in Second Derivative (O(h^2)): {max_second_err:.6e}")

plt.figure(figsize=(9, 4.2))
plt.plot(x_interior, second_diff, 'r--', label="Numerical f''(x)", linewidth
plt.plot(x_interior, exact_second, 'b-', label="Exact -cos(x)", alpha=0.5, l
plt.title("Second Derivative Approximation: f''(x) = -cos(x)", fontsize=12)
plt.xlabel('x')
plt.ylabel("f''(x)")
plt.legend()
plt.show()
```

```
Step size h = 0.05
Max Error in Second Derivative (O(h^2)): 2.083086e-04
```



SENSITIVITY TO NOISE

Real-world data from sensors, accelerometers, or optical encoders always contains measurement noise.

Consider a clean signal $f(x) = \cos(x)$ contaminated by high-frequency noise:

$$f_{\epsilon,\omega}(x) = \cos(x) + \epsilon \sin(\omega x)$$

where:

- $\epsilon = 0.01$ is a small noise amplitude (only 1% of the signal).
- $\omega = 100$ is a high angular frequency.

Taking the analytical derivative:

$$f'_{\epsilon,\omega}(x) = -\sin(x) + \epsilon \cdot \omega \cos(\omega x)$$

Look at the noise amplitude in the derivative:

$$\epsilon \cdot \omega = 0.01 \times 100 = 1.0$$

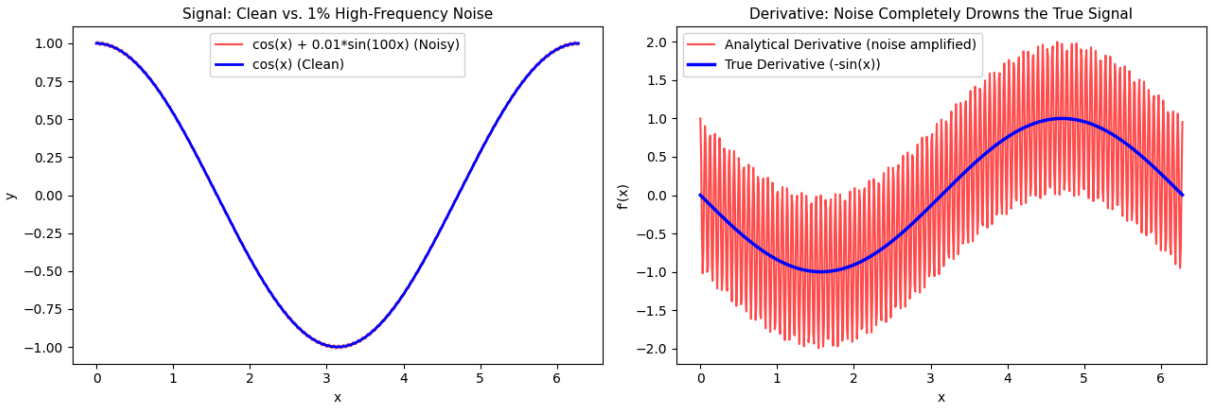
Even though the noise was barely 1% in the original data (completely invisible to the naked eye), the derivative's noise amplitude is 1.0—equal in magnitude to the actual signal.

In finite differences, the issue is even worse:

$$\frac{\Delta y_{\text{noise}}}{h}$$

As step size h becomes smaller, dividing by h amplifies any random noise Δy. Differentiation amplifies noise, and we should be wary of this limitation.

In [11]:
```
x = np.arange(0, 2 * np.pi, 0.01)
omega = 100
epsilon = 0.01

# clean signal vs. noisy signal
y_clean = np.cos(x)
y_noisy = y_clean + epsilon * np.sin(omega * x)

# analytical derivatives
dy_clean = -np.sin(x)
dy_noisy = -np.sin(x) + epsilon * omega * np.cos(omega * x)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4.5))

# plotting functions
ax1.plot(x, y_noisy, 'r-', label='cos(x) + 0.01*sin(100x) (Noisy)', alpha=0.
ax1.plot(x, y_clean, 'b-', label='cos(x) (Clean)', linewidth=2)
ax1.set_title('Signal: Clean vs. 1% High-Frequency Noise', fontsize=11)
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.legend()

# plotting derivatives
ax2.plot(x, dy_noisy, 'r-', label="Analytical Derivative (noise amplified)",
ax2.plot(x, dy_clean, 'b-', label="True Derivative (-sin(x))", linewidth=2.5
ax2.set_title("Derivative: Noise Completely Drowns the True Signal", fontsiz
ax2.set_xlabel('x')
ax2.set_ylabel("f'(x)")
ax2.legend()

plt.tight_layout()
plt.show()
```



NUMERICAL INTEGRATION

Numerical integration approximates the definite integral:

$$I = \int_a^b f(x)\,dx$$

Geometrically, the integral represents the signed area between the curve and the x-axis.

Left and Right Riemann Sums:

We divide the domain $[a, b]$ into $n - 1$ subintervals of equal width:

$$h = \frac{b - a}{n - 1}$$

- Left Riemann Sum: Rectangles calculated using the function value at the left boundary of each subinterval:

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n-2} h \cdot f(x_i)$$

- Right Riemann Sum: Rectangles calculated using the function value at the right boundary:

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{n-1} h \cdot f(x_i)$$

Single-interval error is of order $\mathcal{O}(h^2)$. Global error across all $N = (b - a)/h$ intervals is of the order $\mathcal{O}(h)$, so finer subintervals are crucial for increased accuracy.

MIDPOINT RULE

The midpoint rule significantly improves upon Riemann sums by evaluating the function at the midpoint of each subinterval to calculate the area of the rectangles:

$$y_i = \frac{x_i + x_{i+1}}{2}$$

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n-2} h \cdot f(y_i)$$

It is second-order accurate because the symmetry helps cancel out lower-order error terms in the Taylor expansion. However, this method only works if the data at midpoints is available to you.

In [12]:
```
# left/right Riemann sums vs midpoint rule
# integral of sin(x) from 0 to pi with n = 11 points (exact value = 2.0)
a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)
```

```python
# left Riemann sum
I_riemannL = h * np.sum(f[:-1])
err_riemannL = 2.0 - I_riemannL

# right Riemann sum
I_riemannR = h * np.sum(f[1:])
err_riemannR = 2.0 - I_riemannR

# midpoint rule
x_midpoints = (x[:-1] + x[1:]) / 2.0
I_mid = h * np.sum(np.sin(x_midpoints))
err_mid = 2.0 - I_mid

print(f"Grid points n = {n}, Step size h = {h:.4f}")
print(f"Left Riemann Integral:  {I_riemannL:.8f} | Error: {err_riemannL:.8f}
print(f"Right Riemann Integral: {I_riemannR:.8f} | Error: {err_riemannR:.8f}
print(f"Midpoint Rule Integral: {I_mid:.8f} | Error: {err_mid:.8f}")
print(f"Midpoint error is ~{abs(err_riemannL / err_mid):.1f}x smaller than R
```

```
Grid points n = 11, Step size h = 0.3142
Left Riemann Integral:  1.98352354 | Error: 0.01647646
Right Riemann Integral: 1.98352354 | Error: 0.01647646
Midpoint Rule Integral: 2.00824841 | Error: -0.00824841
Midpoint error is ~2.0x smaller than Riemann.
```

THE TRAPEZOID RULE

The trapezoid rule models the function on each subinterval as a straight line (linear interpolation between endpoints). Instead of calculating the area of a rectangle, we calculate the area of the trapezoid formed by this linear interpolation for each subinterval.

The area of a single trapezoid on $[x_i, x_{i+1}]$ is:

$$\text{Area}_i = \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

Summing over all subintervals gives the trapezoid rule:

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{n-2} \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

We can optimise the calculation. Notice that every interior point $x_1, x_2, \ldots, x_{n-2}$ is shared by two adjacent trapezoids and therefore counted twice, whereas the endpoints x_0 and x_{n-1} are counted only once:

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(x_0) + 2\sum_{i=1}^{n-2} f(x_i) + f(x_{n-1})\right]$$

The trapezoid rule is second-order accurate like the midpoint rule, but uses only the given grid points, making it directly applicable to experimental tabulated data where

midpoints cannot be measured. However, it is less accurate for functions with higher curvature where a linear interpolation between points is not a good approximation.

In [13]:
```
# trapezoid rule implementation for sin(x) from 0 to pi
I_trap = (h / 2.0) * (f[0] + 2 * np.sum(f[1:-1]) + f[-1])
err_trap = 2.0 - I_trap

print(f"Trapezoid Rule Integral: {I_trap:.8f}")
print(f"Trapezoid Rule Error:    {err_trap:.8f}")
```

```
Trapezoid Rule Integral: 1.98352354
Trapezoid Rule Error:    0.01647646
```

SIMPSON'S 1/3 RULE (QUADRATIC INTERPOLATION)

Instead of approximating $f(x)$ with straight line segments, Simpson's rule fits a quadratic polynomial (parabola) through sets of three consecutive points across two adjacent subintervals $[x_{i-1}, x_{i+1}]$ (width $2h$). Then it integrates the parabolic functions exactly over each subinterval.

Summing over all pairs of intervals yields Simpson's rule:

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\left[f(x_0) + 4\sum_{i\ \text{odd}} f(x_i) + 2\sum_{i\ \text{even}} f(x_i) + f(x_{n-1})\right]$$

Simpson's rule requires an odd number of grid points n (so that the number of subintervals $n-1$ is even). It is fourth-order accurate $\mathcal{O}(h^4)$ because the cubic term cancels out symmetrically in the Taylor expansion. Simpson's rule is therefore exact for all polynomials up to degree 3.

In [14]:
```
# Simpson's rule with n = 11 grid points (odd)
I_simp = (h / 3.0) * (f[0] + 4 * np.sum(f[1:-1:2]) + 2 * np.sum(f[2:-1:2]) +
err_simp = 2.0 - I_simp

print(f"Simpson's Rule Integral: {I_simp:.10f}")
print(f"Simpson's Rule Error:    {err_simp:.10f}")
print(f"Trapezoid Error:         {err_trap:.10f}")
print(f"Simpson's Rule is ~{abs(err_trap / err_simp):.0f}x more accurate tha
```

```
Simpson's Rule Integral: 2.0001095173
Simpson's Rule Error:    -0.0001095173
Trapezoid Error:         0.0164764625
Simpson's Rule is ~150x more accurate than Trapezoid on the same 11 points.
```

SCIPY INTEGRATION TOOLS (`scipy.integrate`)

In Python, `scipy.integrate` provides optimised functions for both discrete data arrays and analytical functions:

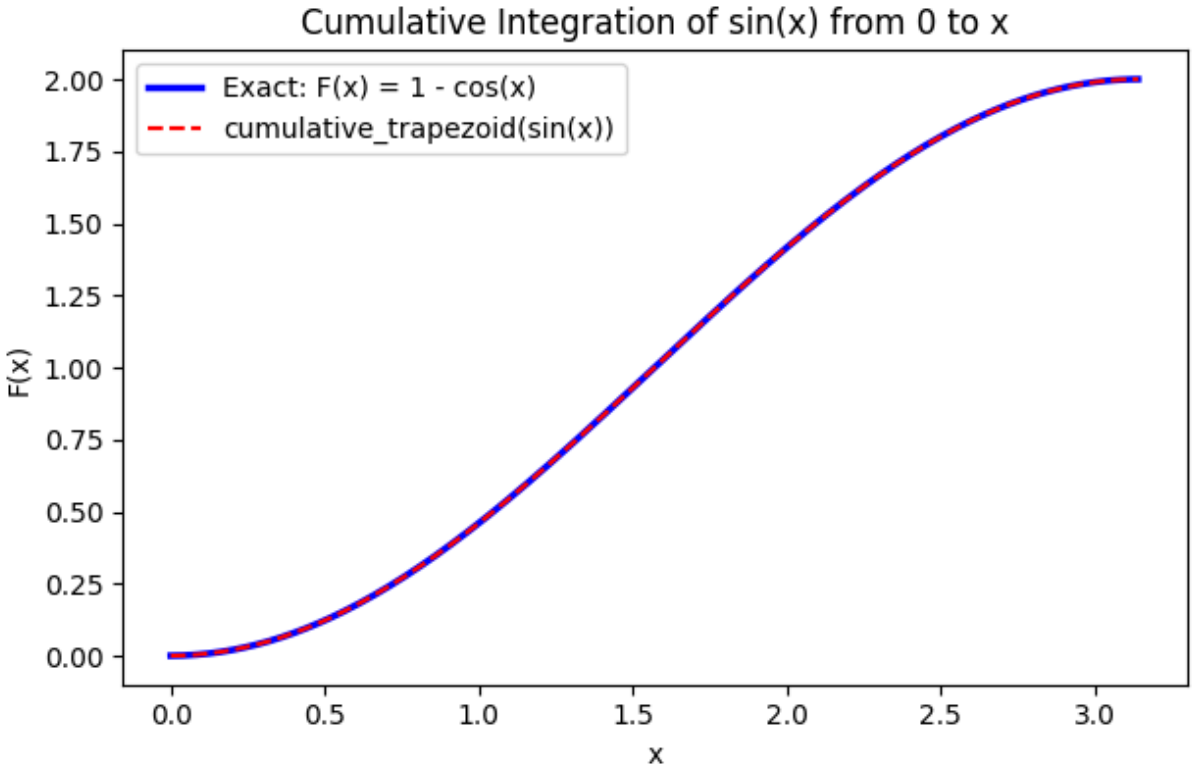
1. `trapezoid` : computes the total trapezoidal integral for given discrete arrays y and x (arguments)
2. `cumulative_trapezoid` : computes the running cumulative integral, same arguments as `trapezoid` :

$$F(X) = \int_{x_0}^{X} f(t)\,dt$$

3. `quad` : higher accuracy method, returning both the estimated integral and an upper bound on absolute numerical error; takes function and boundary values as arguments

In [15]:
```
from scipy.integrate import trapezoid
from scipy.integrate import cumulative_trapezoid
from scipy.integrate import quad

a = 0
b = np.pi
n = 11
x = np.linspace(a, b, n)
f = np.sin(x)

# discrete trapezoid integration
I_scipy_trap = trapezoid(f, x)
print(f"scipy.integrate.trapezoid: {I_scipy_trap:.8f} (Matches manual: {I_tr

# cumulative trapezoidal integration
# exact integral of sin(t) from 0 to x is: 1 - cos(x)
x_fine = np.arange(0, np.pi, 0.01)
F_exact = 1.0 - np.cos(x_fine)
F_approx = cumulative_trapezoid(np.sin(x_fine), x_fine, initial=0)

plt.figure(figsize=(7, 4.2))
plt.plot(x_fine, F_exact, 'b-', label='Exact: F(x) = 1 - cos(x)', linewidth=
plt.plot(x_fine, F_approx, 'r--', label='cumulative_trapezoid(sin(x))', line
plt.title('Cumulative Integration of sin(x) from 0 to x', fontsize=12)
plt.xlabel('x')
plt.ylabel('F(x)')
plt.legend()
plt.show()

# quad integration
I_quad, est_err = quad(np.sin, 0, np.pi)
err_quad = abs(2.0 - I_quad)
print(f"\nscipy.integrate.quad result: {I_quad:.14f}")
print(f"Exact Analytical Error:      {err_quad:.2e}")
print(f"SciPy Estimated Error Bound: {est_err:.2e}")
```

```
scipy.integrate.trapezoid: 1.98352354 (Matches manual: 1.98352354)
```




```
scipy.integrate.quad result: 2.00000000000000
Exact Analytical Error:      0.00e+00
SciPy Estimated Error Bound: 2.22e-14
```

PRACTICE

For these exercises, code the formulas manually. You may use inbuilt Python functions to confirm results (and for practice!).

1. Interpolation

Use **linear interpolation**, **cubic spline interpolation** and **Newton's polynomical interpretation** to estimate $f(3)$ from the following data and compare values:

(x)	2	4	6
(f(x))	5	9	15

2. Root Finding

Use the **bisection method** and the **Newton-Raphson method** to find a root of $f(x) = x^2 - 5$ in the interval $[2, 3]$.

3. Numerical Differentiation

Use the **central difference formula** and the **second derivative formula** with $h = 0.1$ to estimate $f'(2)$ and $f''(2)$ for $f(x) = x^2 + 3x$.

4. Numerical Integration

Use the **right Riemann sum**, **midpoint rule**, **trapezoidal rule**, and **Simpson's rule** with $n = 21$ to approximate $\int_0^2 (x^2 + 1)\,dx$.

PHYB57 · TERM TEST 2 · C1–C21 (through Nov 9)

1 · PYTHON CORE (C1–C5)

Operators: `+ - * / // % **`; 7/2=3.5, 7//2=3, −7//2=−4, 7%2=1, 2**3**2=512, −2**2=−4. **Order:** () → ** → unary − → * / // % → + − → comparisons (< <= > >= == !=) → `not` → `and` → `or`.

Types: int, float (`1e3`, `3.14`), complex (`2+5j`, `complex(2,5)`), bool, str, None. `type(x)`; `int(3.9)`=3; `True`==1, `False`==0 → (3>2)+(5>4)=2; any non-zero number is truthy. = assigns (right→left), == compares. x=x+1 updates x; after y=x+1 changing x does not change y.

math: `import math` → `math.sqrt, sin, exp, log`(=ln), `log10, pi, factorial`; help: `math.factorial?`. **Names:** letters/digits/_ , can't start with digit, case-sensitive, don't shadow built-ins. `%whos` list vars, `del x`, `%reset`.

Strings (immutable): `s='Hello World'`; `len(s)`=11; `s[0]='H'`, `s[-1]='d'`; `s[6:11]='World'` (end excluded); `s[:5]`, `s[::2]`, `s[::-1]` reverse; + concat, `'ab'*2`; `.upper() .lower() .replace('a','b') .split(',') .strip() .find('lo') .count('l'); 'lo' in s; str(3), float('2.5')`; f-string f"{x:.2f} {n:5d} {v:.3e} {3*4}".

List (mutable, any types): `L=[1,2,'a',[3,4]]`; `L.append(x), L.insert(i,x), del L[i], L.pop(), L.remove(x), L.sort(), L1+L2, len(L), x in L, L[1:3], L[::2], list('abc')`, sum(L). b=a is the same object; copy with `a.copy()` / `a[:]`.

Tuple (immutable → TypeError on assign): `t=(1,2,3), (5,)`; unpack `a,b,c=t`; `t.count(2), t[1:3]`. **Set** (unordered, unique): `set([1,2,2,3])`={1,2,3}, `set('Banana')`={'B','a','n'}; `.union .intersection .difference .symmetric_difference .issubset .issuperset .add`; empty = `set()` ({} is a dict).

Dict (key→value, keys unique & immutable: str/number/tuple, not list): `d={'a':1}; d['b']=2; d['a']; d.keys() .values() .items(); 'a' in d` tests keys; `dict([('a',1),('b',2)])`; `del d['a']`; missing key → KeyError.

NumPy: `a=np.array([[1,4,3],[9,2,7]])` → `a.shape=(2,3), a.size=6`; `np.arange(1,5,1)`=[1,2,3,4] (end excl.); `np.linspace(a,b,n)` = n pts incl. both ends, `h=(b−a)/(n−1)`; `np.zeros((2,3)), np.ones(n)`; index `a[0,1]`, row `a[1,:]`, col `a[:,0]`, `a[−1]`, mask `x[x>3]`. `+ - * / **` are element-wise (`b*d` ≠ matrix product); matrix product `np.dot(A,B)` or `A@B`; `A.T`; `np.linalg.inv(A)`. `np.sum, mean, max, abs, sqrt, exp, log, sin, cos, pi`, diff; `np.vstack([x,np.ones(len(x))]).T`; `y[:,np.newaxis]` → column; `np.random.seed(1); np.random.random(n)`.

```
def f(a, b=2):        # b has a default value
    """docstring"""
    return a+b, a*b   # multiple outputs = tuple
s, p = f(1)           # s=3, p=2 ; f(1, b=5)
sq = lambda x: x**2   # anonymous, ONE expression
sorted(L, key=lambda t: t[1])   # sort by 2nd item
def plus1(f, x): return f(x)+1  # function as argument
plus1(np.sin, np.pi/2) → 2.0 ; plus1(lambda x: x+2, 2) → 5
```

Scope: variables made inside a function are local (own memory block): `out=1; f()` that sets `out=6` inside leaves outer `out=1`. Reading a global works; assigning to it inside a function without `global n` → `UnboundLocalError` (referenced before assignment). **Nested** function: defined inside parent, callable only by parent. No `return` → returns `None`; `print` ≠ `return`. Wrong arg type → TypeError. **Recursion:** base case + self call: `def fact(n): return 1 if n<=1 else n*fact(n-1)`.

```
if temp < want-5:  s='Heat'     # first True branch runs,
elif temp > want+5: s='AC'      # rest are skipped;
else:               s='Off'     # same indentation in block
y = a if cond else b            # ternary
for i in range(1,4): n += i     # i=1,2,3 → n=6 ; range(n)=0..n-1
for c in 'banana': ...   for k,v in d.items(): ...
for i,x in enumerate(L): ...  for a,b in zip(A,B): ...
while n >= 1: n /= 2; i += 1    # n=8 → i=4, n=0.5
break → leave loop ; continue → next iteration
[x**2 for x in range(5)]              → [0,1,4,9,16]
[x for x in L if x%2==0]              # filter
[i+j for i in range(2) for j in range(3)] → [0,1,2,1,2,3]
{k: v**3 for k,v in d.items()}        # dict comprehension
```

`range(a,b,step)` end excluded. While = unknown # of iterations; condition never False → infinite loop.

2 · COMPLEXITY & PROFILING (C8)

Count basic operations vs input size n; keep highest-order term, drop constants: $4n^2+n+1$ → **O(n²)**. Growth: O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ).

Pattern	O	Pattern	O
one loop to n; iterative Fibonacci	n	loop halving n (n//=2)	log n
two nested loops to n	n^2	recursive fib f(n−1)+f(n−2)	2^n

2× computing power: exponential alg. solves n→n+1 only; polynomial O(n^c) solves $n \to 2^{1/c}\cdot n$. **NP-complete**: no known polynomial-time algorithm (Travelling Salesman, Set Cover) — basis of encryption. **Profiling:** `%time stmt` (one line, once), `%%time` (whole cell), `%timeit` (mean ± std over many loops), `cProfile/%prun` (per function), `%lprun −f func func(args)` (line_profiler, per line).

例題｜复杂度（数循环次数） `for i in range(n): for j in range(i): s+=1`。**套路**：①数最内层语句执行几次 ②按 i 求和：0+1+…+(n−1) = n(n−1)/2 ③只留最高阶、去掉系数 ⇒ **O(n²)**。若内层是 `while j>1: j//=2`（每次减半）⇒ 每轮 log n，总 O(n log n)；递归每次调用 2 次自己且 n−1 ⇒ O(2ⁿ)。

3 · NUMBER REPRESENTATION & ROUND-OFF (C9)

Base N: digits × powers of N: $121_3 = 1\cdot9+2\cdot3+1 = 16$; $147.3 = 1\cdot10^2+4\cdot10+7+3\cdot10^{-1}$. **Dec→bin:** divide by 2, read remainders bottom→top: 37→18r1→9r0→4r1→2r0→1r0→0r1 ⇒ 100101; 11=1011. **Fraction:** ×2, read integer parts top→down: .625→1.25→0.5→**1.0** ⇒ .101. Add with carry (1+1=10): 100101+10001=110110 (37+17=54). 37×17=1001110101 (629).

$2^0 \ldots 2^{10} = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024$; $2^{-1} \ldots 2^{-4} = .5, .25, .125, .0625$.

IEEE-754 double (64 bit): 1 sign s | 11 exponent e | 52 fraction f n = $(-1)^s \cdot 2^{e-1023} \cdot (1+f)$, f = $\Sigma\, b_i 2^{-i} \in [0,1)$

Decode 1 10000000010 1000…0: s=1, e=1026 → 2^3, 1+f=1.5 ⇒ −12.0. **Encode** 15 = $1111_2 = 1.111_2 \times 2^3$ → s=0, e=3+1023=1026=10000000010, f=1110…0.

e=0 → subnormal: $(-1)^s 2^{-1022}(0+f)$. e=2047 (all 1s): f=0 → ±inf, f≠0 → NaN.

max ≈ $2^{1023}(2-2^{-52}) \approx 1.8e308$; min normal $2^{-1022} \approx 2.2e{-}308$; min subnormal $2^{-1074} \approx 4.9e{-}324$. **Overflow → inf; underflow → 0.**

Gap between adjacent floats grows with |x|: `np.spacing(x)` = 2^{E-52} for $2^E \le x < 2^{E+1}$, spacing(1) = $2^{-52} \approx 2.2e{-}16$ (≈16 sig. digits); spacing(1e9) ≈ 1.2e−7. Adding less than half a gap leaves x unchanged: `1e9 + np.spacing(1e9)/3 == 1e9` → True. **Round-off error** = true value − stored approximation (binary can't represent 0.1, 1/3, π exactly). `0.1+0.2+0.3 != 0.6`; `4.9-4.845 = 0.05500000000000004`; compare floats with `abs(a−b)<tol`. **Accumulates:** 1 + 1/3 − 1/3 once = 1.0; repeated 100× → 1.0000000000000002, grows with # of repetitions. **Repeated rounding:** 4.845→4.85→4.9 but direct → 4.8.

例題｜13.25 → IEEE−754（编码五步） ①整数部分除 2 取余：13 = `1101` ②小数部分乘 2 取整：.25→0.5→1.0 ⇒ `.01` ③合并并规格化：`1101.01` = `1.10101`×2^3（小数点左移 3 位，E=3）④exponent e = E+1023 = 1026 = 10000000010 ⑤fraction = 小数点后 `10101` 后面补 0 补满 52 位；s=0。**反向解码：**读 s、e、f → $(-1)^s \cdot 2^{e-1023} \cdot (1+f)$。**检验：**−6.5 ⇒ s=1, 6.5=1.101×2^2, e=1025。

4 · FILES: TXT / CSV / PICKLE / JSON / HDF5 (C11)

`f=open(name, mode)`: `'r'` read (default; file must exist), `'w'` write (creates / overwrites), `'a'` append, `'r+'` read+write, `'w+'` write+read (discards old), `'a+'` append+read; add `b` for binary. Always `f.close()` — or use `with` (auto-close).

```
f = open('test.txt','w')
for i in range(5): f.write(f"This is line {i}\n")
f.close()
with open('test.txt','a') as f: f.write("more\n")
f = open('test.txt','r'); s = f.read(); f.close() # all → 1 str
f.readline() → one line ; f.readlines() → list of lines
for line in f: print(line.strip())     # strip() drops \n
np.savetxt('a.txt', arr, fmt='%.2f', header='Col1 Col2')
arr = np.loadtxt('a.txt')              # skips '#' header
t, y = np.loadtxt('a.txt', unpack=True)   # columns → vars
np.savetxt('d.csv', data, fmt='%.2f', delimiter=',')
d = np.loadtxt('d.csv', delimiter=',') ; d[:5,:]  # 5 rows
df = pd.read_csv('d.csv') ; df.to_csv('o.csv')    # pandas
pickle.dump(obj, open('f.pkl','wb'))
obj = pickle.load(open('f.pkl','rb'))
json.dump(d, open('f.json','w')); d=json.load(open('f.json','r'))
hf = h5py.File('st.hdf5','w'); hf['/acc/1/data'] = arr
hf['/acc/1'].attrs['dt'] = 0.04 ; hf.close()
hf = h5py.File('st.hdf5','r'); list(hf.keys()) → ['acc','gps']
hf['/acc/1/data'][:] ; hf['/acc/1'].attrs['dt']
```

Format	What / when
TXT/CSV	plain text, universal (Excel, R, MATLAB); CSV = one record per line, fields split by commas; numeric tables
Pickle	serialize (object→byte stream) any Python object, binary `'wb'/'rb'`; Python-only, version issues (Py2 file: `encoding='latin1'`) → not for sharing/long-term
JSON	text key–value like a dict (str, num, list, nested); language-independent, compact; APIs / data exchange
HDF5	binary, hierarchical: **groups** (folders) + **datasets** (arrays) + **attributes** (metadata); huge data, fast & parallel I/O; `h5py`, PyTables

5 · VISUALIZATION (C12)

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,6))
plt.plot(x, y, 'ro', label='data')  # red circles
plt.plot(x, y2, 'g--', label='fit') # 'b-' line 'g--' dash 'k:'
plt.title('T'); plt.xlabel('x'); plt.ylabel('y')
plt.legend(); plt.grid(); plt.xlim(0,5); plt.show()
plt.scatter(x,y) ; plt.loglog(x,y) ; plt.semilogy(x,y)
plt.savefig('f.png') ; plt.style.use('seaborn-poster')
from mpl_toolkits import mplot3d
ax = plt.axes(projection='3d')
ax.plot3D(x, y, z) ; ax.scatter(x, y, z, c='r') # 3-D line, scatter
X, Y = np.meshgrid(x, y); Z = np.sin(X)*np.cos(Y)
ax.plot_surface(X, Y, Z, cmap=plt.cm.cividis)
import cartopy.crs as ccrs; import cartopy.feature as cf
ax = plt.axes(projection=ccrs.PlateCarree())
ax.coastlines(); ax.gridlines(draw_labels=True)
ax.set_extent([lon0, lon1, lat0, lat1])    # zoom
ax.add_feature(cf.LAND)   # OCEAN BORDERS LAKES RIVERS
import matplotlib.animation as manimation
W = manimation.writers['ffmpeg']
writer = W(fps=15, metadata=dict(title='Movie'))
fig = plt.figure(); plt.plot(x, np.sin(x), 'b-') # background
dot, = plt.plot([], [], 'ro')              # placeholder
with writer.saving(fig, 'out.mp4', 100):  # 100 dpi
    for i in range(n):
        dot.set_data([x[i]], [y[i]]); writer.grab_frame()
```

`%matplotlib inline` static; `%matplotlib notebook` interactive (pan/zoom). `meshgrid`: X,Y have shape (len(y), len(x)). Maps: longitude −180…180 (E–W), latitude −90…90 (N–S); Cartopy replaced Basemap; Folium = interactive web maps. Animation = still frames shown rapidly: ① metadata/writer ② draw background ③ loop: update object + grab frame.

6 · LEAST-SQUARES REGRESSION (C16)

Noisy data (x_i, y_i), i=1…m. Model = linear combination of n linearly independent basis functions: $\hat{y}(x) = \Sigma_{j=1\ldots n}\, \alpha_j f_j(x)$. Residual $e_i = y_i - \hat{y}(x_i)$. Minimize total squared error **E = Σ e_i^2 = ||Y − Aβ||$_2^2$**.

A = m×n matrix, $A_{ij} = f_j(x_i)$; β = $[\alpha_1 \ldots \alpha_n]^T$; Ŷ = Aβ. Residual ⊥ col(A): $A^T(Y - A\beta) = 0$ ⇒

Normal eqs: $A^TA\,\beta = A^TY \Rightarrow \beta = (A^TA)^{-1}A^TY$ = pinv(A)·Y (A^TA invertible iff columns of A independent).

Calculus: $\partial E/\partial \alpha_k = 0 \Rightarrow \Sigma_j\, \alpha_j\, \Sigma_i\, f_j(x_i)f_k(x_i) = \Sigma_i\, f_k(x_i)\, y_i$ for k=1…n — the same system.

Line y = mx + b (f_1=x, f_2=1; A = [x | 1]; N = # points): $m = \frac{N\,\Sigma x_i y_i - \Sigma x_i\,\Sigma y_i}{N\,\Sigma x_i^2 - (\Sigma x_i)^2}$, $b = \bar{y} - m\,\bar{x} = \frac{\Sigma y_i - m\,\Sigma x_i}{N}$. **Through origin** (spring F = kx, one basis f=x): $k = \frac{\Sigma\, x_i F_i}{\Sigma\, x_i^2}$.

$A^TA = [[\Sigma x^2, \Sigma x], [\Sigma x, N]]$, $A^TY = [\Sigma xy, \Sigma y]^T$. 2×2 inverse: $\frac{1}{\text{ad−bc}}[[d, -b], [-c, a]]$.

Ex (0,1),(1,3),(2,2): N=3, Σx=3, Σy=6, Σxy=7, Σx²=5 → m=(21−18)/(15−9)=0.5, b=2−0.5·1=1.5; E = (−.5)²+1²+(−.5)² = 1.5.

解題結構｜最小二乘手算（4 步） ①列表算 N、Σx、Σy、Σxy、Σx²（先写表，不会漏项）②写 $A^TA=[[\Sigma x^2,\Sigma x],[\Sigma x,N]]$，$A^TY=[\Sigma xy,\Sigma y]^T$ ③解 2×2（消元或公式 m、b）④算残差 $e_i=y_i-(mx_i+b)$，$E=\Sigma e_i^2$（**检验**：Σe_i=0，且 e ⊥ x）。过原点 ⇒ 只有一个基函数 ⇒ $k=\Sigma xy/\Sigma x^2$。

例題｜幂律线性化 (1,3),(2,12),(4,48) 拟合 $y=bx^m$。①两边取 ln：ln y = m·ln x + ln b ②新点 (ln x, ln y)=(0, ln3),(ln2, ln3+2ln2),(2ln2, ln3+4ln2) ③斜率 = 2ln2/ln2 = **m=2**，截距 ln b=ln3 ⇒ **b=3**。指数型 $y=\alpha e^{\beta x}$ 则用 (x, ln y)：β=斜率，α=$e^{\text{截距}}$。

```
A = np.vstack([x, np.ones(len(x))]).T ; y = y[:, np.newaxis]
ATA = np.dot(A.T, A) ; ATY = np.dot(A.T, y)
alpha = np.dot(np.linalg.inv(ATA), ATY)      # direct
alpha = np.linalg.pinv(A).dot(y)             # pseudo-inverse
alpha = np.linalg.lstsq(A, y, rcond=None)[0]  # lstsq
def func(x, a, b): return a*x + b            # any model
popt, pcov = optimize.curve_fit(func, x, y, p0=[1,1])
perr = np.sqrt(np.diag(pcov))                # 1σ errors
c = np.polyfit(x, y, deg) ; np.polyval(c, x)  # highest power 1st
```

Non-linear models — log trick (linearize, fit a line, recover):

Exponential $y = \alpha e^{\beta x} \to \ln y = \beta x + \ln\alpha$: fit (x, ln y) → β = slope, α = exp(intercept).

Power law $y = bx^m \to \ln y = m \ln x + \ln b$: fit (ln x, ln y) → m = slope, b = exp(intercept).

Polynomial $y = a_n x^n + \ldots + a_0$ is linear in the a's → A columns [x^n … x 1] or `polyfit`. Low order → underfit; high order → RSS↓, R²↑ but overfitting (fits noise, wild between points). **curve_fit**: direct iterative non-linear fit, no transform needed (e.g. sums of exponentials, unknown frequency); give initial guess `p0`.

7 · INTERPOLATION (C17)

Find $\hat{f}$ with $\hat{f}(x_i) = y_i$ — passes through every point; no model assumed; for reliable data, inside the data range only.

Linear ($x_i < x < x_{i+1}$): $\hat{y}(x) = y_i + \frac{(y_{i+1}-y_i)(x-x_i)}{x_{i+1}-x_i}$. Ex x=[0,1,2], y=[1,3,2]: ŷ(1.5) = 3 + (2−3)(.5)/1 = 2.5. `f=interp1d(x,y); f(1.5)` (scipy.interpolate).

Cubic spline: piecewise cubics $S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ on $[x_i, x_{i+1}]$. n points → n−1 cubics → **4(n−1) unknowns**. Equations: $S_i(x_i)=y_i$ and $S_i(x_{i+1})=y_{i+1}$ → 2(n−1); $S'_i(x_{i+1})=S'_{i+1}(x_{i+1})$ → n−2; $S''_i(x_{i+1})=S''_{i+1}(x_{i+1})$ → n−2; + 2 boundary conditions — **natural**: $S''_1(x_1)=0$, $S''_{n-1}(x_n)=0$. Total 4(n−1) ✓. Smooth (C²).

`f=CubicSpline(x,y,bc_type='natural'); f(1.5)`.

Lagrange (one polynomial, degree n−1): $L(x) = \Sigma_i\, y_i P_i(x)$, $P_i(x) = \Pi_{j\neq i} \frac{x - x_j}{x_i - x_j}$,

$P_i(x_j) = 1$ if i=j else 0.

Ex x=[0,1,2], y=[1,3,2]: $P_1=\frac{1}{2}(x-1)(x-2)$, $P_2=-x(x-2)$, $P_3=\frac{1}{2}x(x-1)$ → L = $1\cdot P_1 + 3\cdot P_2 + 2\cdot P_3 = -1.5x^2+3.5x+1$; L(1.5)=2.875. `f=lagrange(x,y)`.

Newton: $f(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \ldots + a_n(x-x_0)\cdots(x-x_{n-1})$; $a_k = f[x_0,\ldots,x_k]$ = **top row** of divided-difference table:

$f[x_i,x_{i+1}] = \frac{y_{i+1}-y_i}{x_{i+1}-x_i}$, $f[x_i,\ldots,x_{i+k}] = \frac{f[x_{i+1},\ldots,x_{i+k}] - f[x_i,\ldots,x_{i+k-1}]}{x_{i+k}-x_i}$. Same polynomial as Lagrange; adding a point only adds one new term (old a's kept).

x	y	1st	2nd	3rd
−5	2	**(6+2)/4 = 2**	**(−5−2)/5 = −1.4**	**(2+1.4)/7 = 0.4857**
−1	6	(1−6)/1 = −5	(1+5)/3 = 2	
0	1	(3−1)/2 = 1		
2	3			

⇒ f(x) = −2 + 2(x+5) − 1.4(x+5)(x+1) + 0.4857(x+5)(x+1)x.

```
coef = np.zeros([n,n]); coef[:,0] = y
for j in range(1,n):
    for i in range(n-j):
        coef[i][j] = (coef[i+1][j-1]-coef[i][j-1]) / (x[i+j]-x[i])
a = coef[0,:]                  # evaluate (nested / Horner):
p = a[n]                       # here n = len(x)-1
for k in range(1,n+1): p = a[n-k] + (x - x[n-k])*p
```

Tut 7 (2,5),(4,9),(6,15) → f(3): linear 7; Newton a=[5, 2, 0.25] → 5+2(1)+.25(1)(−1) = 6.75; natural spline 6.8125.

解題結構｜插值題怎么答 ①先看点的个数 n，多项式次数 = n−1 ②**Lagrange**：每个 P_i 分子=其他 (x−x_i)，分母=代入 x_i，求值时检验 ΣP_i=1 ③**Newton**：画均差表，系数取**第一行**；加新点只在表底多算一斜行，旧系数不变 ④**样条**：未知数 4(n−1)；方程 = 过点 2(n−1) + 一阶导连续 (n−2) + 二阶导连续 (n−2) + 边界 2 ⑤手算最后用另一种方法验算同一点。

8 · ROOT FINDING (C19)

Root x_r: f(x_r) = 0. **Tolerance** = acceptable error; converged when error < tol. Error metrics: **|f(x)|** (how close to 0) or **$|x_{i+1} - x_i|$** (change between guesses). Pitfall: f = 1/x has no root, yet |f(x)| < tol for huge x → false convergence (fsolve stops at max # of function evaluations, `maxfev`). Always check both the error size and algorithm behaviour.

Bisection (Intermediate Value Thm: f continuous on [a,b], f(a)·f(b) < 0 ⇒ root inside):

① check opposite signs (else raise error) ② m = (a+b)/2 ③ if |f(m)| < tol → return m; if sign f(a) = sign f(m) → a = m; else b = m ④ repeat.
Always converges, but **linearly**: bracket halves each step, error after n steps ≤ (b−a)/2^n → n ≥ $\log_2$((b−a)/tol).

Ex f=x²−2 on [0,2]: m=1 (−1) → [1,2]; 1.5 (+.25) → [1,1.5]; 1.25 (−.44) → [1.25,1.5]; 1.375 (−.11) → [1.375,1.5]; 1.4375 (+.066 < 0.1 ✓). tol=0.1 → 1.4375 (5 it.); tol=.01 → 1.4140625 (8 it.); 1e−6 → 21 it.

```
def my_bisection(f, a, b, tol):
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception("a and b do not bound a root")
    m = (a + b)/2
    if np.abs(f(m)) < tol: return m
    elif np.sign(f(a)) == np.sign(f(m)):
        return my_bisection(f, m, b, tol)     # root in [m,b]
    else: return my_bisection(f, a, m, tol)   # root in [a,m]
```

Newton–Raphson (tangent line: 0 = f(x_0) + f′(x_0)(x_1−x_0)): $\mathbf{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$

Quadratic convergence near root (correct digits ≈ double each step) — faster than bisection when x_0 is close. Limits: sensitive to x_0; f′ ≈ 0 → huge/unstable step (÷0); may jump to another root, oscillate or diverge; needs f′.

Ex x²−2, x_0=1.5: x_1 = 1.5 − .25/3 = 1.41667; x_2 = 1.414216 (4 it. for 1e−6). √a: x ← ½(x + a/x).

```
def my_newton(f, df, x0, tol):
    if abs(f(x0)) < tol: return x0
    return my_newton(f, df, x0 - f(x0)/df(x0), tol)
from scipy.optimize import fsolve
f = lambda x: x**3 - 100*x**2 - x + 100    # roots −1, 1, 100
fsolve(f, [2, 80]) → [1., 100.]            # one root per guess
r,info,ier,msg = fsolve(f, x0, full_output=True) # ier=1 ok
```

解題結構｜求根题怎么答 **二分**：①验证 f(a)f(b)<0 ②列表 a,b,m,f(m) 每行一次 ③停止条件 |f(m)|<tol（注意题目用哪个判据）④迭代次数 n ≥ $\log_2$((b−a)/tol)。**Newton**：①先写 f′ ②代入 $x_{k+1}=x_k-f/f'$ ③用分数保持精确 ④$f'\approx0$ 或来回跳（如 x^3-2x+2，x_0=0 在 0↔1 循环）⇒ 不收敛。**选方法**：有区间要稳 ⇒ 二分；有导数且初值好 ⇒ Newton（快）。

9 · NUMERICAL DIFFERENTIATION (C20)

Grid $x_j = x_0 + jh$, spacing h. **Taylor:** $f(x_j \pm h) = f \pm hf' + \frac{h^2}{2}f'' \pm \frac{h^3}{6}f''' + \frac{h^4}{24}f'''' \pm \ldots$ (all at x_j)

Scheme	Formula for f′(x_j)	Leading error	Order
Forward	$[f(x_{j+1}) - f(x_j)] / h$	−(h/2) f″	O(h)
Backward	$[f(x_j) - f(x_{j-1})] / h$	+(h/2) f″	O(h)
Central	$[f(x_{j+1}) - f(x_{j-1})] / 2h$	−(h²/6) f‴	**O(h²)**
f″ central	$[f(x_{j+1}) - 2f(x_j) + f(x_{j-1})] / h^2$	−(h²/12) f⁗	**O(h²)**

Derive: forward = solve Eq(+h) for f′. Central = Eq(+h) − Eq(−h): even terms cancel → 2hf′ + (h³/3)f‴. f″ = Eq(+h) + Eq(−h): odd terms cancel → 2f + h²f″ + (h⁴/12)f⁗. O(h²): halving h cuts error by 2^p (fwd ÷2, central ÷4); on a log-log plot of error vs h the **slope = p** (log err = p log h + log C). Central & f″ formulas are exact for quadratics (f=x²+3x: f′(2)=7, f″(2)=2).

```
h = 0.1; x = np.arange(0, 2*np.pi, h); y = np.cos(x)
fwd = np.diff(y)/h      # = (y[1:]-y[:-1])/h → on x[:-1]
cen = (y[2:] - y[:-2])/(2*h)              # → x[1:-1]
d2  = (y[2:] - 2*y[1:-1] + y[:-2])/h**2  # → x[1:-1]
max_err = max(abs(-np.sin(x[:-1]) - fwd))  # ≈ 0.05 ≈ h/2
```

`np.diff` output has length n−1. cos, h=0.1: max error fwd 0.04998, central 0.00167 (≈ h²/6).

Noise: f = cos x + ε sin(ωx), ε=0.01, ω=100 → f′ = −sin x + **εω** cos(ωx): 1 % noise becomes amplitude εω = 1 = signal size. Differentiation amplifies high-frequency noise (finite diff: $\Delta y_{\text{noise}}/h$); worse for higher derivatives → smooth/fit before differentiating.

例題｜f=x³ 在 x=1，h=0.1（f(0.9)=.729, f(1)=1, f(1.1)=1.331） ①Forward (1.331−1)/.1=**3.31**（误差 .31≈(h/2)f″=.30）②Backward (1−.729)/.1=**2.71** ③Central (1.331−.729)/.2=**3.01**（误差 .01=(h²/6)f‴）④f‴: (1.331−2+.729)/.01=**6.00**。**套路**：真值 f′=3 ⇒ 误差 ⇒ 对照首项系数 ⇒ h 减半：Forward 误差 ÷2，Central ÷4。

10 · NUMERICAL INTEGRATION (C21)

n grid points $x_0 \ldots x_{n-1}$ on [a,b] → n−1 sub-intervals, **h = (b−a)/(n−1)**; $y_i = (x_i + x_{i+1})/2$.

Rule	$\int_a^b f\,dx \approx$	1 interval	Total
Left Riemann	$h\,\Sigma_{i=0}^{n-2} f(x_i)$	O(h²)	O(h)
Right Riemann	$h\,\Sigma_{i=1}^{n-1} f(x_i)$	O(h²)	O(h)
Midpoint	$h\,\Sigma_{i=0}^{n-2} f(y_i)$	O(h³)	**O(h²)**
Trapezoid	$\frac{h}{2}[f(x_0) + 2\Sigma_{i=1}^{n-2} f(x_i) + f(x_{n-1})]$	O(h³)	**O(h²)**
Simpson ⅓	$\frac{h}{3}[f(x_0) + 4\Sigma_{\text{odd } i} f(x_i) + 2\Sigma_{\text{even } i=0,n-1} f(x_i) + f(x_{n-1})]$	O(h⁵)	**O(h⁴)**

Why total = one order lower: # intervals = (b−a)/h, so N·O(h^{p+1}) = O(h^p). **Midpoint:** expand about y_i — the odd (x−y_i) term integrates to 0 by symmetry ⇒ hf(y_i) + (h^3/24)f″, needs f at midpoints (not possible with fixed tabulated data). **Trapezoid** = area (h/2)[f(x_i)+f(x_{i+1})] = average of left & right sums; uses grid data only; exact for straight lines; interior points counted twice. Error ≈ −(b−a)h²f″/12 (midpoint: + (b−a)h²f″/24, half the size, opposite sign). *(Slide C21 says trapezoid O(h) total; but T: O(h²).)*

Simpson: fit a parabola (Lagrange polynomial) through 3 points (x_{i-1}, x_i, x_{i+1}) over two sub-intervals and integrate exactly: (h/3)[f_{i-1} + 4f_i + f_{i+1}]. Needs an **odd # of points** (even # of intervals). Weights 1,4,2,4,…,2,4,1. Exact for polynomials up to degree 3 (cubic term cancels by symmetry). Error ≈ −(b−a)h⁴f‴′/180.

$\int_0^\pi \sin x\,dx = 2$, n = 11 (h = π/10): Left = Right = Trapezoid = 1.9835 (err .0165); Midpoint = 2.0082 (err −.0082); Simpson = 2.00011 (err −1.1e−4, ~150× better than trapezoid). **Tut 7:** $\int_0^2(x^2+1)dx$ = 14/3 = 4.6667, n=21, h=.1: right 4.87, mid 4.665, trap 4.67, Simpson 4.6667 (exact).

```
a, b, n = 0, np.pi, 11
h = (b-a)/(n-1); x = np.linspace(a, b, n); f = np.sin(x)
I_left  = h*sum(f[:n-1])  ;  I_right = h*sum(f[1:])
I_mid   = h*sum(np.sin((x[:n-1] + x[1:])/2))
I_trap  = (h/2)*(f[0] + 2*sum(f[1:n-1]) + f[n-1])
I_simp  = (h/3)*(f[0] + 4*sum(f[1:n-1:2])
                 + 2*sum(f[2:n-2:2]) + f[n-1])
from scipy.integrate import trapezoid, cumulative_trapezoid, quad
trapezoid(f, x)                       # old name: trapz(f, x)
F = cumulative_trapezoid(f, x, initial=0)   # running ∫ (cumtrapz)
I, err = quad(np.sin, 0, np.pi)       # (func, a, b) → value, err
```

`trapezoid/cumulative_trapezoid` take arrays (y, x); cumulative gives F(X)=$\int_a^X$ f dt at every grid point (sin → 1−cos x; without `initial=0` output has n−1 points). `quad` takes a function + limits, adaptive quadrature, returns (integral ≈ 2.0, abs-error bound ≈ 2.2e−14).

例題｜f=x³ 在 [0,2]，n=5，h=0.5（f: 0, .125, 1, 3.375, 8；真值 4） 套路：①h=(b−a)/(n−1) ②列出 f 值 ③代公式。**Left**=.5(0+.125+1+3.375)=**2.25**；**Right**=.5(.125+1+3.375+8)=**6.25**；**Trap**=.25[0+2(4.5)+8]=**4.25**（=(L+R)/2）；**Simpson**=(.5/3)[0+4(.125+3.375)+2(1)+8]=4（三次以内精确）；**Midpoint**（只能用 h=1）=1·(.125+3.375)=**3.5**。h 减半：Trap 误差 ÷4（.25→.0625，得 4.0625）。

11 · NO-CALCULATOR AIDS

√2=1.4142, √3=1.7321, √5=2.2361, √10=3.1623; π=3.1416, π/10=.31416, π^2=9.870; e=2.7183, e^2=7.389, e^{-1}=.3679. ln 2=.6931, ln 3=1.0986, ln 5=1.6094, ln 10=2.3026; $\log_2$10=3.3219; $2^{10}\approx10^3$, $2^{-52}\approx2.2e{-}16$.

θ	0	π/6	π/4	π/3	π/2	π
sin	0	.5	.7071	.8660	1	0
cos	1	.8660	.7071	.5	0	−1

Taylor about a: f(x) = Σ $\frac{f^{(n)}(a)}{n!}(x-a)^n$. e^x = 1+x+x²/2+x³/6…; sin x = x − x³/6 + x⁵/120…; cos x = 1 − x²/2 + x⁴/24… ; ln(1+x) = x − x²/2 + x³/3…; $(1+x)^p \approx 1+px$. $\log_2 x = \ln x/\ln 2$.

PHYB57 — Introduction to Scientific Computing

Term Test 2 · Mock Exam · Duration: **90 minutes** · Total: **100 marks**

Instructions

- Aids: one single-sided letter-size crib sheet. **No calculator.** Leave answers as simple fractions or decimals.
- Covers everything up to and including the Mon Nov 9 lecture (Chapters 1–5, 8, 9, 11, 12, 16, 17, 19, 20, 21).
- Show your work. For code questions, assume `import numpy as np` and `import matplotlib.pyplot as plt` are done; minor syntax slips are not penalised.

Question	1	2	3	4	5	6	7	8	Total
Topic	Python	Complexity / floats	I/O & plots	Least sq.	Interp.	Roots	Diff.	Integr.	
Marks	10	10	8	18	16	14	12	12	100
Suggested min	8	9	7	16	15	13	11	11	90

Question 1 — Python basics

10 marks

Write the exact output of each snippet (2 marks each).

(a)

```
print([i*j for i in range(1,3) for j in range(3) if j != 1])
```

(b)

```
s = 'Scientific'
print(s[2:5], s[-3:], s[::3])
```

(c)

```
n = 5
def f(x):
    n = x + 1
    return n * 2
print(f(n), n)
```

(d)

```
a = np.array([[1, 2], [3, 4]])
print(a * a)
print(np.dot(a, a))
```

(e)

```
d = {k: len(k) for k in ['a', 'bb', 'ccc']}
print(sum(d.values()), 'bb' in d, 2 in d)
```

Question 2 — Complexity and number representation

10 marks

(a) [3] Give the Big-O complexity of `f` and of `g` in terms of `n`, with a one-line justification each.

```
def f(n):                          def g(n):
    s = 0                              if n <= 1:
    for i in range(n):                     return 1
        j = n                          return g(n-1) + g(n-1)
        while j > 1:
            j = j / 2
            s += 1
    return s
```

(b) [2] An IEEE-754 double has the bit pattern (sign | exponent | fraction) `0 | 10000000001 | 0100 0000 … 0`. Which decimal number is it?

(c) [2] Write the sign bit, the 11 exponent bits and the first four fraction bits of −6.5.

(d) [3] (i) What is `np.spacing(8.0)`, as a power of 2? (ii) What does `8.0 + 2**-51 == 8.0` print, and why? (iii) In one sentence, why does `0.1 + 0.2 == 0.3` print `False`?

Question 3 — Files and visualization

8 marks

(a) [5] You have two 1-D arrays of equal length, `t` (seconds) and `y` (metres). Write code that (1) saves them as two columns of a comma-separated file `run.csv` with header `t,y`; (2) reads the file back into two arrays `t2`, `y2`; (3) plots `y2` against `t2` as red circles with axis labels, a title and a legend.

(b) [3] Choose the most appropriate format among TXT/CSV, Pickle, JSON, HDF5 and give one reason each: (i) a nested dictionary of settings that a colleague will read in MATLAB; (ii) 40 GB of sensor arrays from many stations, each with metadata such as location and sampling rate; (iii) a quick save of a Python list of tuples that only your own script will reload tomorrow.

Question 4 — Least-squares regression

18 marks

Four measurements are taken:

x	−1	0	1	2
y	0	1	3	4

(a) [3] For the model $\hat{y} = mx + b$, write the matrix A, the vectors β and Y, and compute $A^{T}A$ and $A^{T}Y$.

(b) [4] Solve the normal equations for m and b.

(c) [2] Compute the four residuals and the total squared error E.

(d) [3] Now use the one-parameter model $\hat{y} = kx$. Starting from $E(k) = \Sigma(y_i - kx_i)^2$, derive the formula for the best k and evaluate it for this data.

(e) [4] A radioactive sample follows $N(t) = N_0 e^{-\lambda t}$. The measured values of ln N are 4.0, 3.4, 3.1 at t = 0, 1, 2. Linearise the model and find λ and N_0 by least squares (leave N_0 as a power of e).

(f) [2] A classmate fits a degree-3 polynomial to the four (x, y) points in the table and reports E = 0, "so it is the best model". Explain why E = 0 and why the conclusion is wrong.

Question 5 — Interpolation

16 marks

Data: (x, y) = (1, 3), (2, 4), (4, 12).

(a) [2] Estimate y(3) by linear interpolation.

(b) [4] Write the three Lagrange basis polynomials P_1, P_2, P_3 and use them to evaluate the Lagrange polynomial at x = 3. (You do not need to expand them.)

(c) [4] Build the divided-difference table and write the Newton polynomial. Expand it to the form $ax^2 + bx + c$.

(d) [3] A fourth point (0, 10) is added. Without redoing earlier work, extend your table and write the new Newton polynomial.

(e) [3] For a cubic spline through the original 3 points: how many unknown coefficients are there? List the conditions (with how many equations each gives) that determine them for a *natural* spline.

Question 6 — Root finding

14 marks

Let $f(x) = x^2 - 3$.

(a) [4] Apply the bisection method on [1, 2] with the stopping criterion $|f(m)| < 0.1$. Tabulate a, b, m, f(m) for every iteration and state the returned root.

(b) [2] Starting from [1, 2], how many bisection iterations guarantee that the midpoint is within 10^{-3} of the true root?

(c) [3] Perform two Newton–Raphson iterations from $x_0 = 2$. Give x_1 and x_2 as fractions.

(d) [2] For $g(x) = x^3 - 2x + 2$, compute the Newton iterates x_1 and x_2 starting from $x_0 = 0$. What happens, and what limitation of the method does this illustrate?

(e) [3] Write a Python function `my_newton(f, df, x0, tol, max_iter=50)` that returns the root estimate once $|f(x)| < \text{tol}$, and raises an exception if the derivative is (nearly) zero or if it does not converge in `max_iter` steps.

Question 7 — Numerical differentiation

12 marks

x	0.9	1.0	1.1	1.2
f(x)	0.729	1.000	1.331	1.728

(a) [4] Estimate $f'(1.0)$ with the forward, backward and central difference formulas, and $f''(1.0)$ with the central formula.

(b) [3] Starting from the Taylor expansions of $f(x_j + h)$ and $f(x_j - h)$, derive the central difference formula for $f'(x_j)$ including its leading error term.

(c) [3] The data are $f(x) = x^3$. Compute the actual errors of the forward and central estimates in (a) and check them against the leading error terms. Predict both errors if h were halved to 0.05.

(d) [2] A signal is $s(x) = \sin x + 0.001 \cdot \sin(1000x)$. What is the amplitude of the noise term in $s'(x)$? What does this tell you about differentiating measured data?

Question 8 — Numerical integration

12 marks

We want $I = \int_0^2 f(x)\,dx$ from the grid values below ($n = 5$ points, $h = 0.5$).

x	0	0.5	1	1.5	2
f(x)	0	0.125	1	3.375	8

(a) [5] Compute the left Riemann sum, the right Riemann sum, the trapezoid rule and Simpson's rule.

(b) [2] Using only the values in the table, compute a midpoint-rule estimate. What sub-interval width did you have to use, and why?

(c) [3] The data are $f(x) = x^3$, so $I = 4$. (i) Why is Simpson's rule exact here? (ii) Predict the trapezoid result if h were halved to 0.25.

(d) [2] Name the `scipy.integrate` function you would use, with its arguments, to (i) integrate the table above; (ii) integrate `np.sin` from 0 to π to near machine precision. What does (ii) return?

— End of test —

参考答案与评分要点

做完再看。每问后面标了分值；计算题有过程分。

Q1 — 10

(a) **[0, 2, 0, 4]** — 外层 i=1,2，内层 j=0,2（跳过 j=1）。

(b) **ien fic Seic** — 下标 0–9 为 S c i e n t i f i c；`s[2:5]` 不含下标 5；`s[::3]` 取 0,3,6,9。

(c) **12 5** — 函数里的 `n` 是局部变量，不影响外面的 n=5。

(d) **[[1 4] [9 16]]**（`*` 是逐元素）和 **[[7 10] [15 22]]**（矩阵乘法）。

(e) **6 True False** — `in` 只查 key，2 是 value 不是 key。

Q2 — 10

(a) **f: O(n log n)** — 外层循环 n 次，内层每次把 j 减半，约 $\log_2 n$ 次。**g: O(2^n)** — 每次调用产生两个规模 n−1 的调用，共约 2^n 次。（各 1.5 分）

(b) s=0 → 正；e = 10000000001_2 = 1025 → $2^{1025-1023} = 2^2$；f = 0.01_2 = 0.25 → 1+f = 1.25。**n = 1.25 × 4 = 5.0**

(c) $6.5 = 110.1_2 = 1.101_2 \times 2^2$ → **sign = 1；exponent = 2+1023 = 1025 = 10000000001；fraction = 1010…**

(d) (i) 8 = 2^3，间隔 = $2^{3-52} = 2^{-49}$。(ii) **True**：2^{-51} 只有间隔的 1/4，小于半个间隔，舍入回 8.0。(iii) 0.1、0.2、0.3 在二进制下是无限循环小数，只能近似存储（representation / round-off error），两边的近似值不相等。

Q3 — 8

(a) 保存 2 分、读取 1.5 分、画图 1.5 分：

```
data = np.vstack([t, y]).T                    # shape (N, 2)
np.savetxt('run.csv', data, delimiter=',', header='t,y')
t2, y2 = np.loadtxt('run.csv', delimiter=',', unpack=True)   # 自动跳过 '#' 开头的 heade
plt.plot(t2, y2, 'ro', label='data')
plt.xlabel('t (s)'); plt.ylabel('y (m)'); plt.title('Run')
plt.legend(); plt.show()
```

不用 unpack 也可以：`d = np.loadtxt(...); t2 = d[:,0]; y2 = d[:,1]`。

(b) (i) **JSON**：key–value 结构、与语言无关。(ii) **HDF5**：大数据、group/dataset 分层、attributes 存元数据、读写快。(iii) **Pickle**：直接序列化任意 Python 对象，只在 Python 内使用所以兼容性不是问题。

Q4 18

(a) $A = [[-1,1],[0,1],[1,1],[2,1]]$（第一列 x，第二列 1），$\beta = [m, b]^T$，$Y = [0,1,3,4]^T$。

$\Sigma x = 2$，$\Sigma x^2 = 6$，$\Sigma y = 8$，$\Sigma xy = 0+0+3+8 = 11 \rightarrow$ **$A^TA = [[6, 2],[2, 4]]$，$A^TY = [11, 8]^T$**

(b) $6m + 2b = 11$，$2m + 4b = 8$。第一式×2 减第二式：$10m = 14 \rightarrow$ **$m = 1.4$**；$b = (8 - 2.8)/4 =$ **1.3**。

用公式验证：$m = (4 \cdot 11 - 2 \cdot 8)/(4 \cdot 6 - 2^2) = 28/20$。

(c) $\hat{y} = -0.1, 1.3, 2.7, 4.1 \rightarrow e = y - \hat{y} =$ **$0.1, -0.3, 0.3, -0.1$**；**$E = 0.01+0.09+0.09+0.01 = 0.2$**

(d) $dE/dk = -2\Sigma x_i(y_i - kx_i) = 0 \rightarrow$ **$k = \Sigma x_i y_i / \Sigma x_i^2 = 11/6 \approx 1.83$**（推导 2 分，数值 1 分）

(e) $\ln N = \ln N_0 - \lambda t$，即直线：斜率 $= -\lambda$，截距 $= \ln N_0$。$\Sigma t = 3$，$\Sigma t^2 = 5$，$\Sigma \ln N = 10.5$，$\Sigma t \cdot \ln N = 3.4 + 6.2 = 9.6$。

斜率 $= (3 \cdot 9.6 - 3 \cdot 10.5)/(3 \cdot 5 - 9) = -2.7/6 = -0.45$；截距 $= (10.5 + 0.45 \cdot 3)/3 = 3.95$。

$\lambda = 0.45$，$N_0 = e^{3.95}$（线性化 1 分，斜率 2 分，N_0 1 分）

(f) 三次多项式有 4 个参数，4 个点 → A 是 4×4 可逆方阵，曲线恰好穿过每个点（等同于插值），所以 $E = 0$。但这是 **overfitting**：它把噪声也拟合进去了，点与点之间和范围之外会剧烈偏离，E 小不代表模型好。

Q5 16

(a) 3 在 [2, 4] 内：$y = 4 + (12 - 4)(3 - 2)/(4 - 2) =$ **8**

(b) $P_1 = \dfrac{(x-2)(x-4)}{(1-2)(1-4)} = (x-2)(x-4)/3$，$P_2 = \dfrac{(x-1)(x-4)}{(2-1)(2-4)} = -(x-1)(x-4)/2$，$P_3 = \dfrac{(x-1)(x-2)}{(4-1)(4-2)} = (x-1)(x-2)/6$。

在 $x=3$：$P_1 = -1/3$，$P_2 = 1$，$P_3 = 1/3 \rightarrow L(3) = 3(-1/3) + 4(1) + 12(1/3) =$ **7**　（检查：三个基函数之和 = 1 ✓）

(c) $f[1,2] = (4-3)/(2-1) = 1$；$f[2,4] = (12-4)/(4-2) = 4$；$f[1,2,4] = (4-1)/(4-1) = 1$。

$p(x) = 3 + 1 \cdot (x-1) + 1 \cdot (x-1)(x-2) = x^2 - 2x + 4$；$p(3) = 7$ 与 (b) 一致。

(d) 新点接在表的最下面（$x_3 = 0$）：$f[4,0] = (10-12)/(0-4) = 0.5$；$f[2,4,0] = (0.5-4)/(0-2) = 1.75$；$f[1,2,4,0] = (1.75-1)/(0-1) = -0.75$。

$p_3(x) = 3 + (x-1) + (x-1)(x-2) - 0.75(x-1)(x-2)(x-4)$；检查 $p_3(0) = 4 - 0.75 \cdot (-8) = 10$ ✓。旧系数不变，这就是 Newton 形式的优点。

(e) $n = 3$ 点 → 2 段三次多项式 → **$4(n-1) = 8$ 个未知数**。条件：每段过两端点 → $2(n-1) = 4$；内部节点 $x=2$ 处一阶导连续 → 1；二阶导连续 → 1；natural 边界 $S''(1) = 0$、$S''(4) = 0 \rightarrow 2$。合计 8 ✓。

Q6 14

(a) f(1) = −2，f(2) = 1，异号 ✓。

iter	a	b	m	f(m)	更新
1	1	2	1.5	−0.75	与 f(a) 同号 → a = 1.5
2	1.5	2	1.75	+0.0625	|f(m)| < 0.1 → 停

返回 1.75（真值 $\sqrt{3} \approx 1.732$）。

(b) 第 n 次的中点与根的距离 $\le (b-a)/2^n = 1/2^n \le 10^{-3} \to 2^n \ge 1000 \to$ **n = 10**（$2^{10} = 1024$）。

(c) $f' = 2x$。$x_1 = 2 - 1/4 =$ **7/4**；$f(7/4) = 49/16 - 3 = 1/16$，$f' = 7/2 \to x_2 = 7/4 - (1/16)/(7/2) = 7/4 - 1/56 =$ **97/56 ≈ 1.7321**。两步已有 4 位有效数字（二次收敛）。

(d) $g(0) = 2$，$g'(0) = -2 \to x_1 = 0 - 2/(-2) =$ **1**；$g(1) = 1$，$g'(1) = 1 \to x_2 = 1 - 1 =$ **0**。在 0 和 1 之间无限循环，永不收敛：Newton 法对初值敏感，不保证收敛。

(e)

```
def my_newton(f, df, x0, tol, max_iter=50):
    x = x0
    for i in range(max_iter):
        if abs(f(x)) < tol:
            return x
        if abs(df(x)) < 1e-12:
            raise ZeroDivisionError("derivative near zero")
        x = x - f(x) / df(x)
    raise Exception("did not converge")
```

评分：更新公式 1，收敛判断 1，两个异常 1。递归写法也可以。

Q7 12

(a) h = 0.1。Forward：(1.331 − 1)/0.1 = **3.31**；Backward：(1 − 0.729)/0.1 = **2.71**；Central：(1.331 − 0.729)/0.2 = **3.01**；$f'' \approx (1.331 - 2 + 0.729)/0.01 =$ **6.00**。

(b) $f(x_j+h) = f + hf' + (h^2/2)f'' + (h^3/6)f''' + \ldots$；$f(x_j-h) = f - hf' + (h^2/2)f'' - (h^3/6)f''' + \ldots$

相减：$f(x_j+h) - f(x_j-h) = 2hf' + (h^3/3)f''' + O(h^5) \to$ **$f'(x_j) = [f(x_{j+1}) - f(x_{j-1})]/(2h) - (h^2/6)f''' + \ldots$，$O(h^2)$**

(c) 真值 $f'(1) = 3$，$f'' = 6x = 6$，$f''' = 6$。Forward 误差 = 0.31，首项 $(h/2)f'' = 0.05 \cdot 6 = 0.30$ ✓。Central 误差 = 0.01，首项 $(h^2/6)f''' = 0.01$ ✓（x^3 没有更高阶导数，所以正好相等）。

$h \to 0.05$：**forward ≈ 0.15（减半，O(h)；精确值 0.1525），central = 0.0025（变为 1/4，$O(h^2)$）**。

(d) $s' = \cos x + 0.001 \cdot 1000 \cdot \cos(1000x) \to$ **噪声幅度 = 1**，和信号本身一样大。原信号里只有 0.1% 的高频噪声，求导后被放大 ω 倍：数值微分会放大高频噪声，对实测数据求导前要先平滑或拟合。

Q8

(a) Left = 0.5(0 + 0.125 + 1 + 3.375) = **2.25**；Right = 0.5(0.125 + 1 + 3.375 + 8) = **6.25**；
Trapezoid = (0.5/2)[0 + 2(0.125 + 1 + 3.375) + 8] = 0.25 × 17 = **4.25**（也等于 (L+R)/2）；
Simpson = (0.5/3)[0 + 4(0.125 + 3.375) + 2(1) + 8] = (1/6)(24) = **4**。（L、R 各 1，T 1，S 2）

(b) 中点必须是表里有的点，所以只能用宽度 h = 1 的两个区间 [0,1]、[1,2]，中点 0.5 和 1.5：M = 1·(0.125 + 3.375) = **3.5**。用 h = 0.5 的话需要 x = 0.25, 0.75… 处的值，表里没有——这是 midpoint rule 用于离散数据的局限。

(c) (i) Simpson 对三次及以下多项式精确：误差首项 $\propto h^4 f''''$，而 x^3 的四阶导为 0（三次项因对称性抵消）。(ii) 梯形法整体误差 $O(h^2)$：现在误差 0.25，h 减半后变为 1/4 即 0.0625 → **T ≈ 4.0625**。

(d) (i) **`trapezoid(f, x)`**（旧名 `trapz`；传入数组）。(ii) **`quad(np.sin, 0, np.pi)`**，返回元组（积分值 ≈ 2.0，绝对误差估计 ≈ 2e−14）。